



东邪的机器学习之旅

作者: dx

时间: 2025, 8, 19

版本: 1.0

自定义: 信息



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

目录

第 1 章 机器学习基础理论	1
1.1 基础名词与概念	1
1.2 监督学习	2
1.3 评估模型	3
1.3.1 分类模型评估	4
第 2 章 基础工具层	6
第 3 章 线形回归-预测线性值	7
第 4 章 逻辑回归-分类	8
4.1 分类方法	11
4.1.1 LDA	12
4.1.2 马氏距离	13
4.1.3 ROC 与 AUC	14
4.1.4 QDA	15
第 5 章 控制过拟合	17
第 6 章 发现隐藏结构	18
6.1 PCA	18
6.2 协方差矩阵理解	18
6.3 SVD 奇异值分解	19
6.3.1 为什么要把矩阵转化为向量乘积	20
6.3.2 为什么还需要 SVD 分解	20
6.3.3 如何寻找正交化基底	21
6.3.4 奇异值分解计算方法	22
6.3.5 奇异值分解也可以满足不满秩的情况	22
6.3.6 为什么可以去掉小的 σ 项	23
第 7 章 工程基础	26
第 8 章 训练前的准备	27
第 9 章 训练前的设计	28
第 10 章 训练传统模型	29
10.1 回归树	29
10.1.1 回归树的具体计算	30
10.2 装代法	32
10.3 随机森林	33
10.4 boosting 提升法	33
10.4.1 GBRT	35
10.4.2 XGBoosting	36

10.4.3 LightGBM	38
第 11 章 训练神经网络	43
第 12 章 调优与上线意识	44
第 13 章 QR 分解	45
13.1 转置子空间等知识回顾	46
13.2 最小二乘法	48
13.3 QR 分解操作	51
13.4 QR 分解迭代法-数值分析	53
第 14 章 LLL 格基	56
14.1 LLL 最短向量的意义	59
14.2 LLL 的时间复杂度	62
第 15 章 Attention Is All You Need	64
15.1 Abstract	64
15.2 Introduction	64
15.3 Background	64
15.4 Model Architecture	65
15.4.1 Encoder and Decoder Stacks	65
15.4.2 Attention	65
15.4.2.1 Scaled Dot-Product Attention	66
15.4.2.2 Multi-Head Attention	66
15.4.2.3 Applications of Attention in our Model	66
15.4.3 Position-wise Feed-Forward Networks	67
15.4.4 Embeddings and Softmax	67
15.4.5 Positional Encoding	67
15.5 Why Self-Attention	67
15.6 Training	68
15.6.1 Training Data and Batching	68
15.6.2 Hardware and Schedule	68
15.6.3 Optimizer	69
15.6.4 Regularization	69
15.7 Results	69
15.7.1 Machine Translation	69
15.7.2 Model Variations	69
15.7.3 English Constituency Parsing	70
15.8 Conclusion	70
第 16 章 《Deep Neural Networks for YouTube Recommendations》中文版	71
16.1 摘要	71
16.2 引言	71
16.3 系统概述	71
16.4 候选生成	72
16.4.1 将推荐建模为分类	72

16.4.2	模型架构	72
16.4.3	异质信号	73
16.4.4	标签与上下文选择	73
16.4.5	特征与深度实验	73
16.5	排序	74
16.5.1	特征表示	74
16.5.2	期望观看时长建模	75
16.5.3	隐藏层实验	75
16.6	结论	75
第 17 章	Deep Neural Networks for YouTube Recommendations	77
17.1	Abstract	77
17.2	Introduction	77
17.3	System Overview	78
17.4	Candidate Generation	78
17.4.1	Recommendation as Classification	78
17.4.2	Model Architecture	79
17.4.3	Heterogeneous Signals	79
17.4.4	Label and Context Selection	80
17.4.5	Experiments with Features and Depth	80
17.5	Ranking	81
17.5.1	Feature Representation	81
17.5.2	Modeling Expected Watch Time	82
17.5.3	Experiments with Hidden Layers	83
17.6	Conclusions	83
第 18 章	XGBoost: A Scalable Tree Boosting System	84
18.1	ABSTRACT	84
18.2	1. INTRODUCTION	84
18.3	2. TREE BOOSTING IN A NUTSHELL	84
18.3.1	2.1 Regularized Learning Objective	84
18.3.2	2.2 Gradient Tree Boosting	85
18.3.3	2.3 Shrinkage and Column Subsampling	85
18.4	3. SPLIT FINDING ALGORITHMS	85
18.4.1	3.1 Basic Exact Greedy Algorithm	85
18.4.2	3.2 Approximate Algorithm	86
18.4.3	3.3 Weighted Quantile Sketch	86
18.4.4	3.4 Sparsity-aware Split Finding	86
18.5	4. SYSTEM DESIGN	86
18.5.1	4.1 Column Block for Parallel Learning	86
18.5.2	4.2 Cache-aware Access	86
18.5.3	4.3 Blocks for Out-of-core Computation	87
18.6	5. RELATED WORKS	87
18.7	6. END TO END EVALUATIONS	87
18.7.1	6.1 System Implementation	87

18.7.2	6.2 Dataset and Setup	87
18.7.3	6.3 Classification	87
18.7.4	6.4 Learning to Rank	88
18.7.5	6.5 Out-of-core Experiment	88
18.7.6	6.6 Distributed Experiment	88
18.8	7. CONCLUSION	88
第 19 章	《XGBoost: A Scalable Tree Boosting System》中文版	89
19.1	摘要	89
19.2	1. 引言	89
19.3	2. 树提升简述	89
19.3.1	2.1 正则化学习目标	89
19.3.2	2.2 梯度树提升	89
19.3.3	2.3 收缩与列采样	90
19.4	3. 分裂查找算法	90
19.4.1	3.1 基本精确贪婪算法	90
19.4.2	3.2 近似算法	90
19.4.3	3.3 加权分位数草图	90
19.4.4	3.4 稀疏感知分裂查找	91
19.5	4. 系统设计	91
19.5.1	4.1 用于并行学习的列块	91
19.5.2	4.2 缓存感知访问	91
19.5.3	4.3 用于外存计算的块	91
19.6	5. 相关工作	91
19.7	6. 端到端评估	91
19.7.1	6.1 系统实现	91
19.7.2	6.2 数据集与实验设置	91
19.7.3	6.3 分类	92
19.7.4	6.4 学习排序	92
19.7.5	6.5 外存实验	92
19.7.6	6.6 分布式实验	92
19.8	7. 结论	92
第 20 章	LightGBM: A Highly Efficient Gradient Boosting Decision Tree	93
20.1	Abstract	93
20.2	1 Introduction	93
20.3	2 Preliminaries	93
20.3.1	2.1 GBDT and Its Complexity Analysis	93
20.3.2	2.2 Related Work	94
20.4	3 Gradient-based One-Side Sampling	94
20.4.1	3.1 Algorithm Description	94
20.4.2	3.2 Theoretical Analysis	94
20.5	4 Exclusive Feature Bundling	94
20.6	5 Experiments	95
20.6.1	5.1 Overall Comparison	95

20.6.2	5.2 Analysis on GOSS	95
20.6.3	5.3 Analysis on EFB	95
20.7	6 Conclusion	96
第 21 章	《LightGBM: A Highly Efficient Gradient Boosting Decision Tree》中文版	97
21.1	摘要	97
21.2	1 引言	97
21.3	2 预备知识	97
21.3.1	2.1 GBDT 及其复杂度分析	97
21.3.2	2.2 相关工作	97
21.4	3 基于梯度的单边采样	98
21.4.1	3.1 算法描述	98
21.4.2	3.2 理论分析	98
21.5	4 互斥特征捆绑	98
21.6	5 实验	98
21.6.1	5.1 整体比较	99
21.6.2	5.2 GOSS 分析	99
21.6.3	5.3 EFB 分析	99
21.7	6 结论	99
第 22 章	模型	100
22.1	BPR 模型	100
22.1.1	WBPR	101
第 23 章	基础知识	102
23.1	attention	104

第1章 机器学习基础理论

- 1 《An Introduction to Statistical Learning with Python》
 - └ = 给机器学习建立主干：从数据、模型、误差到泛化
 - └ 0 基础工具层（不是目的，是理解所有模型的公共语言）
 - └ 监督学习 / 无监督学习
 - └ 明确特征 X 、标签 y 、训练集、测试集、预测目标
 - └ 损失函数与泛化误差
 - └ 训练误差低不等于模型好，关键是新数据上表现好
 - └ 交叉验证与重采样
 - └ 用数据切分估计模型未来表现，是调参和选模型的底座
 - └ 能力一预测连续数值 → 回归
 - └ 线性回归
 - └ 正则化
 - └ 非线性扩展
 - └ 能力二判断样本类别 → 分类
 - └ 逻辑回归
 - └ LDA / QDA
 - └ SVM
 - └ 能力三控制过拟合 → 模型选择
 - └ 偏差-方差权衡
 - └ 调参
 - └ 评估指标
 - └ 能力四发现隐藏结构 → 无监督学习
 - └ PCA
 - └ K-Means
 - └ 层次聚类

1.1 基础名词与概念

定义 1.1 (可约误差与不可约误差)

预测误差 = 可约误差 + 不可约误差。设真实关系 $Y = f(X) + \epsilon$ ， $\hat{f}(X)$ 为模型估计：

- 可约误差： $f(X) - \hat{f}(X)$ ，可通过改进模型降低。
- 不可约误差： ϵ （随机噪声），无法消除。

 **笔记** [Bias-Variance 分解] 预测误差分解为：

$$\mathbb{E}(Y - \hat{f})^2 = \underbrace{\text{Bias}^2 + \text{Variance}}_{\text{reducible}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

其中 $\sigma^2 = \text{Var}(\epsilon)$ 为噪声。

东邪的 tip 1.1

误差的期望就是可约误差 + 不可约误差也就是偏差平方 + 方差 + 不可约误差对吗

例题 1.1 可约误差 可约误差 $f(X) - \hat{f}(X)$ 是模型没学到真实规律造成的误差。

例如，真实关系 $f(X) = 3X + 5$ ，但你学到 $\hat{f}(X) = 2X + 10$ ，则 $f - \hat{f}$ 即为模型误差。

降低可约误差的三种方式：

- 换更好的模型：线性回归 $\rightarrow$ 神经网络
- 增加训练数据：数据更多 $\Rightarrow \hat{f} \rightarrow f$
- 调参数：正则化强度、树深度、学习率

东邪的 tip 1.2

可约误差就是模型不够强，不可约误差就是真实世界会出现某些随机情况

表 1.1: 预测与推断的区别

	预测 (Prediction)	推断 (Inference)
目标	预测未来或未知结果	理解变量之间的规律
问题	$X \rightarrow Y$	X 和 Y 的关系
关注	预测准确率	参数、显著性、因果关系
例子	预测股票涨跌	分析哪些因素影响股票价格
常用方法	机器学习、深度学习	回归分析、假设检验

东邪的 tip 1.3

线形模型是模型的名称最小二乘是构建线形模型的方法



笔记 精度越高可解释性越弱，比如线形模型。推断一般用欠柔性的方法方便解释关系，预测一遍用柔性方法但是柔性方法就是容易过拟合。

1.2 监督学习

定义 1.2 (监督学习)

监督学习 (supervised learning) 是一类利用有标签数据集 (labeled dataset) 训练模型的机器学习方法。数据集表示为 $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ ，其中 N 为样本总数， $\mathbf{x}_i$ 为第 i 个样本的特征向量 (feature vector)， y_i 为对应的标签 (label)。监督学习算法 (supervised learning algorithm) 利用上述数据集生成一个模型，以样本的特征向量作为输入，输出用于判断该样本标记的信息。



例如，一个癌症预测模型可以利用某患者的特征向量，输出该患者患有癌症的概率。就是每一个数据是知道是癌症或者不是癌症的

定义 1.3 (非监督学习)

有别于监督学习，非监督学习 (unsupervised learning) 只需要包含无标签样本的数据集，表示为 $\{(\mathbf{x}_i)\}_{i=1}^N$ 。非监督学习算法 (unsupervised learning algorithm) 所产生的模型 (model) 同样接受一个特征向量 $\mathbf{x}$ 为输入信息，并通过数学变换使其变成另外一个对其他任务更有用的向量或数值。



笔记 非监督学习的三类典型任务举例如下：

- 聚类 (clustering)：对电商平台的用户按消费行为自动分组，模型输出每个用户所属的类簇标记，无需事先告知“应分几类”。
- 降维 (dimensionality reduction)：将每位用户由上千维的行为特征向量压缩为二维向量，便于可视化分析用户分布。
- 异常值检测 (outlier detection)：对每笔信用卡交易输出一个实数值，该值越大表示该笔交易与“正常”交易差异越大，从而识别潜在的欺诈行为。

定义 1.4 (半监督学习)

半监督学习 (semi-supervised learning) 可利用掺杂着有标签和无标签样本的数据集进行学习, 通常情况下无标签样本的数量远超过有标签样本。半监督学习算法 (semi-supervised learning algorithm) 的功能和原理与监督学习算法大同小异, 区别在于算法可以利用大量无标注的样本学得更好的模型。

定义 1.5 (强化学习)

在强化学习 (reinforcement learning) 中, 我们假设机器“生活”在一个环境中, 并可以感知当前环境的状态 (state), 状态表示为特征向量。在每个状态下, 机器可以通过执行不同动作而获取相应的奖励, 同时进入另一个环境状态。强化学习的目的是学习选择行动的策略 (policy), 使得机器在长期交互过程中累计获得的奖励最大化。

 **笔记** 强化学习的典型应用场景举例如下:

- 游戏对战: AlphaGo以棋盘局面作为状态, 以落子位置作为动作, 通过不断与自身对弈获取胜负奖励, 最终学得超越人类的博弈策略。
- 自动驾驶: 车辆以道路环境 (车速、障碍物位置等) 作为状态, 以加速、制动、转向作为动作, 通过安全行驶里程作为奖励信号, 学习最优驾驶策略。
- 推荐系统: 平台以用户历史行为作为状态, 以推送内容作为动作, 以用户点击或停留时长作为奖励, 持续优化内容推荐策略。

 **笔记** 分类 (classification) 与回归 (regression) 的核心区别在于输出值的类型:

- 分类: 输出为离散值, 即样本所属的类别标签。例如, 判断一封邮件是否为垃圾邮件 (是/否), 或识别手写数字属于 0 到 9 中的哪一类。
- 回归: 输出为连续实数值。例如, 根据房屋面积、地段等特征向量预测房价, 或根据历史数据预测某股票次日的收盘价格。

 **笔记** 浅度学习 (shallow learning) 与深度学习 (deep learning) 的区别主要体现在模型结构的复杂程度:

- 浅度学习: 模型结构较简单, 通常只有一层或少数几层变换, 例如逻辑回归、支持向量机 (SVM)、决策树等。此类模型对特征工程依赖较强, 需要人工设计输入特征。
- 深度学习: 模型由多层非线性变换堆叠而成 (即深层神经网络), 能够自动从原始数据中逐层提取抽象特征, 无需大量人工特征设计。典型应用包括图像识别、语音识别和自然语言处理。

一般而言, 深度学习在数据量充足时表现更优, 而浅度学习在小样本或可解释性要求较高的场景下仍具有重要价值。

东邪的 tip 1.4

选方差当惩罚函数一个是因为绝对值函数不平滑, 一个是可以惩罚偏差

1.3 评估模型

东邪的 tip 1.5

定量用回归分析定性用分类问题

 **笔记** 拟合效果检验用测试的均方差检验模型的好坏越小越好

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2$$

 **笔记** Test MSE的 Bias-Variance 分解

对测试点 x_0 , 期望测试 MSE 分解为三项:

$$\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \underbrace{\text{Var}(\hat{f}(x_0))}_{\text{Variance}} + \underbrace{[\text{Bias}(\hat{f}(x_0))]^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible Error}}$$

$$\text{Test MSE} = \text{Variance} + \text{Bias}^2 + \text{Irreducible Error}$$

其中 $\text{Variance} + \text{Bias}^2$ 为可约误差, $\text{Var}(\epsilon)$ 为不可约误差 (数据固有噪声, 无法消除)。

东邪的 tip 1.6

偏差是用简单模型模拟复杂世界的误差, 方差是用不同训练集估计 f 函数估计的改变量. 我们想让 MSE 最小就是要让偏差和方差尽可能的都小但是偏差方差下降的曲线是不一样的所以我们需要权衡这个。

定义 1.6 (Training Error Rate)

分类模型在训练集上的错误比例:

$$\text{Error Rate} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i)$$

其中 $I(\cdot)$ 为示性函数, 预测错误取 1, 否则取 0。



1.3.1 分类模型评估

定义 1.7 (贝叶斯分类器 Bayes Classifier)

贝叶斯分类器将每个样本分到条件概率最大的类别:

$$\hat{f}(x_0) = \arg \max_k P(Y = k | X = x_0)$$

其中 $P(Y = k | X = x_0)$ 为给定特征 x_0 时样本属于类别 k 的后验概率。

贝叶斯分类器是理论上最优的分类器, 其测试误差率称为贝叶斯误差率 (Bayes Error Rate), 是所有分类器能达到的最小误差下限。



东邪的 tip 1.7

通过设定一个概率比如大于百分之 50 选 A 小于百分之五十选 B 这条等于 50 的分界线就是贝叶斯决策边界分类本质是化几条线把不是一个类的隔开

笔记 贝叶斯分类器举例

假设用湿度、风速预测天气。新样本: 湿度 90%, 风速 5 km/h。

模型计算三类天气的后验概率:

$$P(\text{下雨} | \text{湿度} = 90\%, \text{风速} = 5) = 0.7$$

$$P(\text{阴天} | \text{湿度} = 90\%, \text{风速} = 5) = 0.2$$

$$P(\text{晴天} | \text{湿度} = 90\%, \text{风速} = 5) = 0.1$$

贝叶斯分类器取概率最大的类别:

$$\hat{f}(x_0) = \arg \max_k P(Y = k | X = x_0) = \text{下雨}$$

即给定当前特征 X , 分别计算属于每个类别 k 的概率, 哪个最大就分到哪一类。

定义 1.8 (KNN 分类器)

K 近邻分类器 (K-Nearest Neighbors, KNN) 是一种非参数分类方法。对于测试点 x_0 :

1. 在训练集中找到与 x_0 距离最近的 K 个样本 (邻居)
2. 统计这 K 个邻居的类别分布
3. 将 x_0 分到邻居中出现频率最高的类别

$$\hat{f}(x_0) = \arg \max_k \sum_{i \in N_K(x_0)} I(y_i = k)$$

其中 $N_K(x_0)$ 为 x_0 的 K 个最近邻构成的集合, $I(\cdot)$ 为示性函数。



 笔记 KNN 分类器举例

假设用年龄、收入预测是否购买产品, 训练集如下:

年龄	收入 (万)	购买
25	30	是
35	50	是
22	20	否
28	45	是
23	25	否

新样本 x_0 : 年龄 26, 收入 35 万。取 $K = 3$, 计算欧氏距离:

$$d(26, 25) = \sqrt{(26 - 25)^2 + (35 - 30)^2} \approx 5.1 \quad (\text{是})$$

$$d(26, 35) = \sqrt{(26 - 35)^2 + (35 - 50)^2} \approx 17.5 \quad (\text{是})$$

$$d(26, 22) = \sqrt{(26 - 22)^2 + (35 - 20)^2} \approx 15.5 \quad (\text{否})$$

$$d(26, 28) \approx 10.0 \quad (\text{是})$$

$$d(26, 23) \approx 10.0 \quad (\text{否})$$

最近邻: $K = 3 \rightarrow$ 距离最近的三个样本为 (25, 30, 是)、(23, 25, 否)、(28, 45, 是)。

邻居住类分布: 是 $\times 2$, 否 $\times 1$ 。

$$\hat{f}(x_0) = \arg \max_k \{\text{是: } 2, \text{ 否: } 1\} = \text{是}$$

故 KNN 将 x_0 预测为“购买”。

东邪的 tip 1.8

通过 neighbors 决定你是谁人以类聚物以群分, 第一个 K 也就一共选几个最近的邻居

第 2 章 基础工具层

第 3 章 线形回归-预测线性值

第4章 逻辑回归-分类

定义 4.1 (逻辑回归——解决把结果映射到 01 之间以及把向量乘成数字)

逻辑回归 (logistic regression) 是一种用于二分类任务的监督学习算法。给定样本的特征向量 $\mathbf{x} \in \mathbb{R}^n$, 逻辑回归通过 **sigmoid** 函数将线性组合 $\mathbf{w}^\top \mathbf{x} + b$ 映射到区间 $(0, 1)$, 输出样本属于正类的概率:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

其中 $\mathbf{w} \in \mathbb{R}^n$ 为权重向量, $b \in \mathbb{R}$ 为偏置项。模型参数通常通过最大化对数似然函数 (即最小化交叉熵损失) 估计得到。预测时, 以 $\hat{y} \geq 0.5$ 判为正类, 否则判为负类。



笔记 逻辑回归的似然函数

给定 n 个样本 (x_i, y_i) , 逻辑回归的似然函数为:

$$L(\beta_0, \beta_1) = \prod_{i: y_i=1} p(x_i) \prod_{i': y_{i'}=0} (1 - p(x_{i'}))$$

其中 $p(x) = P(Y = 1 | X = x) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$ 。

最大似然估计 (MLE) 找一组 β_0, β_1 最大化 L , 即让“所有样本的真实标签同时出现”的概率最大。

举例: 3 个样本 $(x_1, 1), (x_2, 0), (x_3, 1)$ 的似然:

$$L = p(x_1) \cdot (1 - p(x_2)) \cdot p(x_3)$$

直观理解: 正样本的预测概率 $p(x)$ 尽量大, 负样本的预测概率 $1 - p(x)$ 尽量大, 两者乘积最大时得到最优参数。

定义 4.2 (Log-Odds (对数几率))

Odds (几率) 为发生与不发生概率之比: $\text{odds} = \frac{p}{1-p}$ ($p = 0.8$ 时 $\text{odds} = 4$, 即发生可能性是不发生的 4 倍)。

Log-Odds (对数几率) 即对 odds 取自然对数: $\log\left(\frac{p}{1-p}\right)$ 。



笔记 逻辑回归的核心

逻辑回归不是直接让概率 p 等于线性函数, 而是让 log-odds 等于线性函数:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x$$

东邪的 tip 4.1

也就是吸烟的人比不吸烟的人肺癌概率是 15 倍, 要求线性模型的生活也就是要把概率映射到实数轴的时候用 logit 要把数字映射到概率的时候用 logistic

笔记 Logit vs Logistic: 互为反函数

- **Logit** 函数: 概率 $\rightarrow$ 实数

$$\text{logit}(p) = \log \frac{p}{1-p}, \quad p \in (0, 1) \rightarrow (-\infty, +\infty)$$

作用: 将约束在 $(0, 1)$ 的概率映射到全体实数, 使得线性函数可以拟合。

- **Logistic 函数**：实数 $\rightarrow$ 概率

$$p = \frac{e^z}{1 + e^z}, \quad z \in (-\infty, +\infty) \rightarrow (0, 1)$$

作用：将线性组合 $z = \beta_0 + \beta_1 x$ 映射回合法的概率值。

- 关系：两者互为反函数，logistic 是 logit 的逆变换。

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x \iff p = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

逻辑回归就是在线性回归外层套了一个 logistic 函数，将实数输出映射到 $[0, 1]$ 。

定义 4.3 (哑变量 (Dummy Variable))

将 K 个类别的分类变量转化为 $K - 1$ 个 0/1 数值变量：

$$D_j = \begin{cases} 1, & \text{样本属于第 } j \text{ 类} \\ 0, & \text{否则} \end{cases}, \quad j = 1, \dots, K - 1$$

第 K 类作为参照组， $K - 1$ 个哑变量全为 0。



东邪的 tip 4.2

所以哑变量就是把一个“是什么”的问题，拆成 $K - 1$ 个“是不是？”的问题。也就是是的时候他对应的那个系数起作用也就乘 1 不然的话就不起作用也就乘 0

例题 4.1 哑变量：从设计到预测的完整流程

场景：预测是否购买，特征有学历和收入。

1. 设计哑变量并展示完整数据（参照组 = 高中）

学历	D_1 (本科)	D_2 (研究生)	D_3 (高铁司机)	收入	购买
高中	0	0	0	30	0
本科	1	0	0	50	1
研究生	0	1	0	40	1
高铁司机	0	0	1	60	0

你知道： D_1, D_2, D_3 分别对应三种学历。模型看到：四个数值列 (D_1, D_2, D_3 , 收入)，仅此而已。

2. 模型训练学出系数：

$$\log \frac{p}{1-p} = \underbrace{-12}_{\beta_0} + \underbrace{2.5}_{\beta_1} D_1 + \underbrace{3.0}_{\beta_2} D_2 + \underbrace{(-1.5)}_{\beta_3} D_3 + \underbrace{0.2}_{\beta_4} \times \text{收入}$$

模型仍不知道 D_1 是本科，只知道这一列等于 1 时加上 2.5 误差变小。

3. 你来解释

- $\beta_1 = 2.5$ ：本科购买 odds 是高中生的 $e^{2.5} \approx 12.2$ 倍
- $\beta_3 = -1.5$ ：高铁司机购买 odds 是高中生的 $e^{-1.5} \approx 0.22$ 倍（更低）
- $\beta_4 = 0.2$ ：收入每增加 1 单位，odds 乘以 $e^{0.2} \approx 1.22$

4. 预测新样本（本科，收入 55）

$$D_1 = 1, D_2 = 0, D_3 = 0, \text{收入} = 55$$

$$\log \frac{p}{1-p} = -12 + 2.5 \times 1 + 0.2 \times 55 = 1.5$$

$$p = \frac{e^{1.5}}{1 + e^{1.5}} \approx 0.82$$

模型不知道他是本科生，只是机械代入公式算出 82%。你知道他是本科生，所以你念出那句话：“本科生有 82% 的概率购买”。

东邪的 tip 4.3

这个里面其实有两个模型一个是用欠款 *balance* 预测违约率一个是通过其他的指标预测 *balance*

例题 4.2 哑变量：ISLR 信用卡欠款预测

场景：预测信用卡欠款 (*Balance*)，特征包括收入、限额、是否学生。

1. 你设计哑变量 (参照组 = 非学生)

Student(D_1)	收入	限额	欠款
1	14.9	3601	333
0	10.6	2646	403
0	15.4	4722	12
1	16.5	2873	482

只有 2 个类别 (学生/非学生)，所以只需 $K - 1 = 1$ 个哑变量。 $D_1 = 1$ 为学生。

2. 模型训练学出系数：

$$\widehat{\text{欠款}} = \underbrace{441}_{\beta_0} + \underbrace{(-66.5)}_{\beta_1} D_1 + \underbrace{0.013}_{\beta_2} \times \text{收入} + \underbrace{0.0015}_{\beta_3} \times \text{限额}$$

模型不知道 D_1 是“是否学生”，只知道第 1 列等于 1 时减去 66.5 误差更小。

东邪的 tip 4.4

p 值在探究 A 和 B 到底有没有关系的时候很重要，假设实验也是 $H_0: A$ 和 B 无关 (也就是系数为 0)， H_1 就是有关。

- 系数 (Coefficient)：表示变量的影响方向和大小。*balance* 每增加 1 单位，log-odds 增加 0.0055。
- 标准误差 (Standard Error)：衡量系数估计的稳定性。标准误差越小，估计越可靠。
- z 统计量 (z -statistic)：系数与其标准误差的比值，衡量系数离 0 有多远。
- p 值 (p -value)：在 $\beta = 0$ 的零假设下，观察到当前 z 统计量的概率。 p 值越小，变量越显著。

例题 4.3 混淆现象：学生、信用卡欠款与违约率

数据：ISL 的 Default 数据集。响应变量为

$$\text{default} \in \{\text{No}, \text{Yes}\},$$

解释变量包括信用卡欠款余额 *balance*、收入 *income* 和学生身份 *student*。

模型一：只用 *balance* 预测违约率

$$\log \left(\frac{P(\text{default} = \text{Yes})}{1 - P(\text{default} = \text{Yes})} \right) = \beta_0 + \beta_1 \text{balance} \Rightarrow \beta_1 > 0.$$

结论：信用卡欠款余额越高，违约概率越高。

模型二：只用 *student* 预测违约率

$$\log \left(\frac{P(\text{default} = \text{Yes})}{1 - P(\text{default} = \text{Yes})} \right) = \beta_0 + \beta_1 \text{student} \Rightarrow \beta_1 > 0.$$

结论：只看学生身份时，学生的违约概率看起来更高。

模型三：同时控制 *balance*、*income* 和 *student*

$$\log \left(\frac{P(\text{default} = \text{Yes})}{1 - P(\text{default} = \text{Yes})} \right) = \beta_0 + \beta_1 \text{balance} + \beta_2 \text{income} + \beta_3 \text{student} \Rightarrow \beta_3 < 0.$$

结论：在欠款余额和收入相同的条件下，学生反而比非学生更不容易违约。

为什么符号反转？

- *balance* 越高，违约概率越高；
- 学生群体平均 *balance* 更高；

- 因此只看 *student* 时，模型把 *balance* 的影响也归给了学生身份；
- 控制 *balance* 后，*student* 的条件效应变为负。

教训：边际关系不等于条件关系。一个变量在简单模型中可能为正，加入关键控制变量后可能变为负，这就是混淆现象。

东邪的 tip 4.5

这里欠款不是违约而是花呗待还款。所以不是学生更大概率会违约而是学生倾向于更多欠款，欠款多的违约率高，如果欠款一样多学生的违约率反而少。这个就是我们得注意要控制什么变量，比如探讨学生这个身份会不会引起违约我们就得保证在欠款收入一样的时候比较

笔记 多类逻辑回归

二分类逻辑回归只适用于响应变量只有两类的情况，例如

$$Y \in \{0, 1\}.$$

但实际问题中，响应变量可能有多于两类。例如急诊病人可能被诊断为

$$Y \in \{\text{中风}, \text{服药过量}, \text{癫痫发作}\}.$$

这类问题不能简单把类别编码为 1, 2, 3，因为这些类别没有天然顺序，数字大小会错误地暗示类别之间存在大小关系和距离关系。

当响应变量有 $K > 2$ 个无序类别时，可以使用多类逻辑回归 (multinomial logistic regression)。其基本思想是：先选一个类别作为基准类 (baseline)，然后分别建模其他类别相对于基准类的对数几率。

若选择第 K 类作为基准类，则对 $k = 1, \dots, K - 1$ ，有

$$\log \left(\frac{P(Y = k | X = x)}{P(Y = K | X = x)} \right) = \beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p.$$

因此，每一个非基准类别都会有一组自己的系数。模型最终输出 K 个类别的概率，并且这些概率之和为 1：

$$\sum_{k=1}^K P(Y = k | X = x) = 1.$$

基准类的选择不影响最终预测概率和分类结果，但会影响系数的解释。若选择“癫痫发作”为基准类，则“中风”对应的系数表示：某个特征增加一单位时，中风相对于癫痫发作的对数几率如何变化。

东邪的 tip 4.6

多远回归得选一个基准线，算出相对概率再乘基准概率得到最终概率

定义 4.4 (Softmax)

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad \text{如 } [3, 1, 0] \xrightarrow{e^z} [20.1, 2.7, 1] \xrightarrow{\text{归一化}} [0.84, 0.11, 0.05]$$

指数 e^z 放大分数差距，实现“带竞争的归一化”，使得分最高的 token 获得大部分注意力权重。

东邪的 tip 4.7

softmax 就归一化只不过为了突出重要信息先变成 e 的指数次方再归一

4.1 分类方法

- 逻辑回归 (判别式模型)：

$$P(Y = k | X = x)$$

直接建模——给定特征 x ，样本属于第 k 类的概率。

- **生成式分类模型**：先对每个类内部建模：

$$P(X = x | Y = k)$$

再用贝叶斯公式反推类别概率：

$$P(Y = k | X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)}$$

其中：

- π_k ：第 k 类的先验概率（该类原本有多常见）
- $f_k(x)$ ：第 k 类内部特征 x 的概率密度函数
- $P(Y = k | X = x)$ ：最终想要的分类概率

4.1.1 LDA

东邪的 tip 4.8 (什么时候用 LDA)

- **3 类以上**：LDA 直接支持多分类，只需对每个类算 $\delta_k(x)$ 再取最大即可。逻辑回归的多分类需要 one-vs-rest 或 softmax，实现更复杂。
- **特征大致正态**：LDA 假设每类特征服从正态分布，当数据满足这一条件时，LDA 的估计是理论最优的。
- **样本量不大**：LDA 只需估计每类均值 $\hat{\mu}_k$ 和共享协方差 $\hat{\Sigma}$ ，在小样本下比逻辑回归的 MLE 更稳定，不会出现不收敛的问题。

定义 4.5 (线性判别分析 (LDA))

线性判别分析 (Linear Discriminant Analysis, LDA) 是一种生成式分类方法。设

$$Y \in \{1, 2, \dots, K\}, \quad X \in \mathbb{R}^p.$$

LDA 假设每一类中的特征服从多元正态分布，且各类共享同一个协方差矩阵：

$$X | Y = k \sim N(\mu_k, \Sigma).$$

其分类规则为：

$$\hat{Y} = \arg \max_k P(Y = k | X = x).$$

等价地，LDA 选择使下列判别函数最大的类别：这个其实就是 $1/2$ 码氏距离然后把相同的项笑掉大牙了然后再加一个先验概率

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k.$$



例题 4.4 LDA 直观例子 用身高、体重判断三类人：篮球运动员、普通人、马拉松运动员。

LDA 先估计每类均值 μ_k 和共享协方差 Σ ：

$$\mu_{\text{篮球}} = \begin{pmatrix} \text{平均身高} \\ \text{平均体重} \end{pmatrix}, \mu_{\text{普通}} = \begin{pmatrix} \text{平均身高} \\ \text{平均体重} \end{pmatrix}, \mu_{\text{马拉松}} = \begin{pmatrix} \text{平均身高} \\ \text{平均体重} \end{pmatrix}$$

$$\Sigma = \begin{pmatrix} \text{Var(身高)} & \text{Cov(身高, 体重)} \\ \text{Cov(体重, 身高)} & \text{Var(体重)} \end{pmatrix}$$

新样本 $x_0 = (190, 88)^T$ 来了，LDA 算三类的判别得分：

$$\delta_k(x_0) = x_0^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k$$

判给得分最高的类: $\hat{Y} = \arg \max_k \delta_k(x_0)$ 。

直观理解: x_0 离哪个类的中心最近 (经 Σ^{-1} 加权), 就分给谁。

证明 新样本 x 属于第 k 类的概率取决于它与 μ_k 的距离 (经 Σ^{-1} 加权):

$$-\frac{1}{2}(x - \mu_k)^\top \Sigma^{-1}(x - \mu_k) = -\frac{1}{2}x^\top \Sigma^{-1}x + x^\top \Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k$$

其中 $-\frac{1}{2}x^\top \Sigma^{-1}x$ 与 k 无关 (x 固定, Σ 共享), 分类时删去, 剩下:

$$\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k$$

这正是 LDA 的线性判别函数。

东邪的 tip 4.9

线性来自共享协方差矩阵。“共享协方差矩阵”意思是: 不同类别的特征分布, 可以有不同的均值, 但假设它们有相同的形状和方向。他的最后得分是样本和 k 类匹配度 + 对第 k 类位置的修正 + 原本样本中第 k 类出现的概率。 x 特征向量转置乘协方差矩阵的逆是消除相关性和方差

4.1.2 马氏距离

定义 4.6 (马氏距离)

设 $x \in \mathbb{R}^p$ 是一个样本点, $\mu \in \mathbb{R}^p$ 是中心点, Σ 是协方差矩阵。则 x 到 μ 的马氏距离定义为

$$d_M(x, \mu) = \sqrt{(x - \mu)^\top \Sigma^{-1}(x - \mu)}.$$

等价地, 平方马氏距离为

$$d_M^2(x, \mu) = (x - \mu)^\top \Sigma^{-1}(x - \mu).$$



笔记 距离不能只看绝对差多大, 还要看这个方向上本来波动大不大、变量之间有没有相关性。

因此, 在 LDA 中, Σ^{-1} 的作用是用数据自身的波动规律来衡量样本 x 距离类别中心 μ_k 有多远。

定义 4.7 (协方差矩阵)

设 $X = (X_1, \dots, X_p)^\top$ 为 p 维随机向量, 其协方差矩阵定义为:

$$\Sigma = \begin{pmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_p) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_p, X_1) & \text{Cov}(X_p, X_2) & \cdots & \text{Var}(X_p) \end{pmatrix}$$

其中每个元素为:

$$\Sigma_{ij} = \text{Cov}(X_i, X_j) = \frac{1}{n-1} \sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)$$

简言之, 协方差矩阵就是把每对特征的协方差算出来, 按 (i, j) 位置排成一个 $p \times p$ 方阵。对角线上是各特征的方差, 非对角线上是特征间的协方差。



东邪的 tip 4.10 (拿到一个矩阵就要分解它)

协方差矩阵的特征分解 (谱分解):

$$\Sigma = Q\Lambda Q^\top$$

- Q : 特征向量组成的正交矩阵, 表示新坐标系的方向。在二维情况下, Q 的两列就是椭圆的长轴和短轴方向。
- $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_p)$: 特征值组成的对角矩阵, 表示各方向上的方差大小。特征值越大, 该方向上的数据波动越大。

直观理解: Σ 描述了一个 p 维椭球。 Q 告诉你这椭球朝哪个方向转, Λ 告诉你在每个方向上的半径 (拉伸) 有多大。

 笔记 Σ^{-1} 的加权意义

特征独立时:

$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{pmatrix}, \quad \Sigma^{-1} = \begin{pmatrix} 1/\sigma_1^2 & 0 \\ 0 & 1/\sigma_2^2 \end{pmatrix}$$

$$d_M^2(x, \mu) = \frac{(x_1 - \mu_1)^2}{\sigma_1^2} + \frac{(x_2 - \mu_2)^2}{\sigma_2^2}$$

方差大的特征权重小, 方差小的特征权重大。

特征相关时:

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{pmatrix}, \quad \Sigma^{-1} = \frac{1}{\sigma_1^2 \sigma_2^2 - \sigma_{12}^2} \begin{pmatrix} \sigma_2^2 & -\sigma_{12} \\ -\sigma_{12} & \sigma_1^2 \end{pmatrix}$$

$$d_M^2 = \frac{1}{1 - \rho^2} \left[\frac{(x_1 - \mu_1)^2}{\sigma_1^2} - 2\rho \frac{(x_1 - \mu_1)(x_2 - \mu_2)}{\sigma_1 \sigma_2} + \frac{(x_2 - \mu_2)^2}{\sigma_2^2} \right]$$

其中 $\rho = \frac{\sigma_{12}}{\sigma_1 \sigma_2}$ 。 Σ^{-1} 不仅按方差加权, 还通过协方差结构去除冗余信息。

4.1.3 ROC 与 AUC**东邪的 tip 4.11**

对于信用卡以及癌症检测抱着宁可错杀一千也不能放走一个的想法需要把原来百分之 50 可能是坏人我们就判定为坏人现在我们要设置成百分 20 是坏人我们就判定他是坏人

- **灵敏度 (Sensitivity)**: $P(\hat{Y} = 1 | Y = 1) = \frac{TP}{TP + FN}$, 衡量模型找出“违约”样本的能力。
- **特异度 (Specificity)**: $P(\hat{Y} = 0 | Y = 0) = \frac{TN}{TN + FP}$, 衡量模型辨出“未违约”样本的能力。

定义 4.8 (ROC 曲线)

对阈值 t , 定义 $TP(t) = \sum_i 1(s_i \geq t \wedge y_i = 1)$, $FP(t) = \sum_i 1(s_i \geq t \wedge y_i = 0)$, 则

$$TPR(t) = \frac{TP(t)}{TP(t) + FN(t)}, \quad FPR(t) = \frac{FP(t)}{FP(t) + TN(t)}.$$

以 $FPR(t)$ 为横轴、 $TPR(t)$ 为纵轴, 遍历 t 绘制所得曲线为 **ROC 曲线**。



东邪的 tip 4.12

ROC 曲线：纵轴是 TPR（灵敏度/逮捕率），横轴是 FPR（1 - 特异性）。也就错杀率

- TPR 就是灵敏度，也就是衡量能不能给坏的都找全了。- 特异性是能不能给好人不错杀了。- (0,0) 的时候：一个坏人找不到，一个好人不错杀。- (1,1) 的时候：好人坏人通杀。

定义 4.9 (AUC)

AUC 是 ROC 曲线下的面积： $AUC = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x)) dx$ 。等价于随机正样本得分大于随机负样本的概率： $AUC = P(s_+ > s_-)$ 。取值范围 $[0.5, 1]$ 。



笔记选哪个阈值完全不影响 AUC。

AUC 衡量的只有一件事：模型的排序能力。

- 不管你在 ROC 曲线上选 (0.1, 0.9) 还是 (0.3, 0.98)，AUC 都是同一个值。
- 因为 AUC 是整条曲线下的面积，把所有阈值都算进去了。
- 它只在乎“癌患者的评分是不是普遍高于正常人”。

打个比方：

- AUC = 高考成绩排名——在全校里你排什么位置（排序能力）。
- 阈值 = 你选择“多少分算及格”——这完全由老师（业务方）根据实际情况来定。

AUC 高只说明模型“排名排得好”，但不能告诉你“切在哪里最好”——那是业务的事。

东邪的 tip 4.13

$AUC = \sum_{\text{正常人可能的分数}} \text{该分数出现的概率} \times \text{患病者分数比它高的概率}$

4.1.4 QDA**定义 4.10 (QDA (Quadratic Discriminant Analysis) 二次判别分析)**

设 $x | y = k \sim \mathcal{N}(\mu_k, \Sigma_k)$, $k = 0, 1$ 。贝叶斯决策规则为

$$\hat{y}(x) = \arg \max_k \delta_k(x),$$

其中判别函数

$$\delta_k(x) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k) + \log \pi_k.$$

$\delta_k(x)$ 包含 x 的二次项（因 $\Sigma_0 \neq \Sigma_1$ ），故决策边界为二次曲面。若 $\Sigma_0 = \Sigma_1$ 则退化为 LDA。

**东邪的 tip 4.14**

相求后验不好求先验乘条件概率密度，然后为了把乘变成 + 就取了 log 第一项其实就是概率密度的系数，第二项是 exp 里面的。第 k 类的条件密度就是第 k 类的高斯概率密度

- $-\frac{1}{2} (x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k)$ ：马氏距离的平方的一半（带负号），衡量 x 离第 k 类中心的远近。
- $-\frac{1}{2} \log |\Sigma_k|$ ：第 k 类分布的“体积/扩散程度惩罚”。协方差矩阵的行列式越大，分布越分散，该惩罚项越小。
- $\log \pi_k$ ：先验概率的对数。

东邪的 tip 4.15

QDA 的判别函数来自对整个高斯密度取对数。因此指数项中的马氏距离平方会保留下来，而密度前面的归一化系数也会变成对数项，例如 $|\Sigma_k|^{-1/2}$ 会变成 $-\frac{1}{2} \log |\Sigma_k|$ 。

- **核心假设**: LDA 假设各类协方差矩阵相等 $\Sigma_0 = \Sigma_1 = \Sigma$; QDA 允许 $\Sigma_0 \neq \Sigma_1$ 。
- **决策边界**: LDA 为线性 (超平面); QDA 为二次 (椭圆、抛物线等曲面)。
- **参数数量**: LDA 需估计 $p(p+1)/2$ 个协方差参数; QDA 需估计 $p(p+1)$ 个, 约为 LDA 的两倍。
- **偏差-方差权衡**: LDA 偏差更高但方差更低, 小样本下更稳定; QDA 偏差更低但方差更高, 需足够数据支撑。
- **适用场景**: LDA 适于样本量较小或各类协方差相近的情形; QDA 适于样本量充足且各类散布矩阵差异明显的情形。

东邪的 tip 4.16

正态分布里面 $\exp$ 里面的东西恰好就是马氏距离乘-1/2

- **用 LDA 的情形**:
 - 样本量较小, 不足以稳定估计各类的协方差矩阵。
 - 各类的协方差矩阵大致相等 (可通过 Box's M 检验验证)。
 - 追求模型简洁、可解释性强, 或决策边界已知为线性。
- **用 QDA 的情形**:
 - 样本量充足, 各类样本数远大于特征数 p 。
 - 各类的散布程度差异明显 (如一类很紧凑, 另一类很分散)。
 - 决策边界明显非线性, 且线性模型欠拟合。

定义 4.11 (朴素贝叶斯 (Naive Bayes))

对 $x = (x_1, \dots, x_p)$, 贝叶斯定理结合条件独立假设:

$$P(y = k | x) \propto P(y = k) \prod_{j=1}^p P(x_j | y = k).$$

朴素假设: 给定 y , 各特征条件独立。决策规则为

$$\hat{y}(x) = \arg \max_k \left[\log P(y = k) + \sum_{j=1}^p \log P(x_j | y = k) \right].$$

$P(x_j | y = k)$ 依特征类型选择高斯分布 (连续)、多项分布 (计数) 或伯努利分布 (二值)。



第 5 章 控制过拟合

第 6 章 发现隐藏结构

6.1 PCA

定义 6.1 (Principal Component Analysis)

主成分分析 (PCA) 是一种无监督学习算法，用于降维并识别数据中最具信息量的特征。

它的本质思想是：假设数据关系具有线性结构，并将其投影到一个由正交向量组成的子空间中，使得方差最大化。这样的向量称为主成分，它们决定了数据变化最大（信息量最高）的方向。

另一种等价的解释是：PCA 可以看作一种线性投影，它最小化了原始点与其投影点之间的均方根 (RMS) 距离。

PCA 的工作原理

首先，特征矩阵需要进行中心化处理，以确保第一主成分对应的是数据的最大变化方向，而不仅仅是平均值。通常，求主成分的方法有两种主要思路：第一，协方差矩阵的特征值与特征向量分解。协方差矩阵反映了不同变量之间的线性相关程度。该矩阵的特征向量给出了数据方差最大的方向，而特征值则表示该方向上的方差大小。与某个特征向量对应的特征值刻画了该向量对解释数据方差的贡献。特征值越大，该主成分的重要性就越高。通常，只选择能够解释预设方差比例（如 95%）的主成分。

6.2 协方差矩阵理解

定义 6.2 (3.3.1)

设 $(X_1, X_2, \dots, X_n)$ 是 n 维随机变量，若 $E(X_1), E(X_2), \dots, E(X_n)$ 存在，则称

$$(E(X_1), E(X_2), \dots, E(X_n))$$

为 $(X_1, X_2, \dots, X_n)$ 的数学期望，简称期望。

固定 i, j ，若

$$E([X_i - E(X_i)][X_j - E(X_j)])$$

存在，则称其为 X_i 与 X_j 的协方差 (covariance)，记作 $\text{Cov}(X_i, X_j)$ ，即

$$\text{Cov}(X_i, X_j) = E([X_i - E(X_i)][X_j - E(X_j)]).$$

易知， $E(X_i)$ 和 $E(X_j)$ 存在是 $\text{Cov}(X_i, X_j)$ 存在的必要条件。

若对 $i, j = 1, 2, \dots, n$ ，所有 $\text{Cov}(X_i, X_j)$ 均存在，则称 n 阶方阵

$$\Sigma = (b_{ij})_{n \times n}, \quad b_{ij} = \text{Cov}(X_i, X_j)$$

为 $(X_1, X_2, \dots, X_n)$ 的协方差矩阵 (covariance matrix)。

东邪的 tip 6.1 (方差与协方差总结)

1. 定义

$$\text{Var}(X) = E[(X - E[X])^2], \quad \text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

2. 数学关系

$$\text{Var}(X) = \text{Cov}(X, X)$$

方差是协方差的特例。

3. 性质比较

性质	方差 $\text{Var}(X)$	协方差 $\text{Cov}(X,Y)$
定义	单变量的离散程度	两变量的联动程度
符号	≥ 0	可正、可负、或为 0
几何意义	数据点围绕均值的分布宽度	点云的倾斜方向、线性关系
特殊情况	$\text{Var}(X) = 0 \iff X$ 为常数	$\text{Cov}(X,Y) = 0 \nRightarrow$ 独立

里面的 $E(\cdot)$ 就是期望算子，它的作用就是“取加权平均”。情景：两门考试成绩。设三名学生的语文与数学成绩分别为 A: $X = 50, Y = 55$, B: $X = 60, Y = 65$, C: $X = 70, Y = 75$ 。

1. 算平均值（期望）：

$$E[X] = \frac{50 + 60 + 70}{3} = 60, \quad E[Y] = \frac{55 + 65 + 75}{3} = 65.$$

2. 偏离均值：

$$\text{A: } X - 60 = -10, Y - 65 = -10;$$

$$\text{B: } X - 60 = 0, Y - 65 = 0;$$

$$\text{C: } X - 60 = 10, Y - 65 = 10.$$

3. 偏离乘积：

$$\text{A: } (-10)(-10) = 100, \quad \text{B: } 0 \cdot 0 = 0, \quad \text{C: } 10 \cdot 10 = 100.$$

4. 取平均（期望）得到协方差：

$$\text{Cov}(X,Y) = E[(X - E[X])(Y - E[Y])] = \frac{100 + 0 + 100}{3} = \frac{200}{3} \approx 66.7.$$

结果解读：协方差为正，说明语文高的一般数学也高（正相关）；若为负则一高一低（负相关）；若接近 0，则两者几乎无线性关系。协方差矩阵就是把所有不同的 X_i 和 Y_j 的关联性在一个矩阵中记录出来。

东邪的 tip 6.2

进一步来说来说减法是把所有的 x 轴上的点左移或者右移， Y 轴的点上移或者下移，两个合起来的操作就是把坐标系切换成了以他们均值为原点的坐标系。他们的乘积就是每一个坐标和原点围成矩形的面积，加权的方式可以让你觉得哪一部分的重要凸显，比如说我希望进缩小极大偏离平均值的那项对整体的影响我就可以给他加一个小的权重让他生成的矩阵面积缩小。

- 协方差不仅是“面积”，还是一种趋势的代数平均。
- 如果绝大多数点落在“同向象限”（右上、左下），那么平均面积大于 0 $\rightarrow$ 正相关。
- 如果大多数点落在“反向象限”（右下、左上），平均面积小于 0 $\rightarrow$ 负相关。
- 如果点云分布比较均匀，正负面积互相抵消 $\rightarrow$ 协方差接近 0。

协方差的几何意义有助于理解其方向与强度，可参见下列视频：<https://www.bilibili.com/video/BV1UWYfz2EKZ>

6.3 SVD 奇异值分解

第二，数据矩阵的奇异值分解 (SVD)。奇异值分解将任意矩阵表示为三个矩阵的乘积：左奇异矩阵 U 、奇异值对角矩阵 S 、以及右奇异矩阵 V 。其中，奇异值是协方差矩阵特征值的平方根（这就是为什么需要先对数据做中心化），右奇异矩阵 V 对应于协方差矩阵的特征向量，而左奇异矩阵 U 则是原始数据在主成分上的投影。因此，SVD 也可以提取出主成分，而且无需显式计算协方差矩阵。与特征分解方法相比，这种方式更高效、更数值稳定，尤其是在特征间存在强相关性、导致协方差矩阵条件数较差的情况下。特别地，scikit-learn 的 PCA 实现采用的就是这种方法，不过还结合了一些额外的技巧。

东邪的 tip 6.3 (特征值与奇异值)

奇异值与特征值在定义和几何意义上既有联系又有区别。对于方阵 $A \in \mathbb{R}^{n \times n}$, 若存在 λ 使得 $Av = \lambda v$ ($v \neq 0$), 则 λ 称为 A 的特征值; 而对于任意矩阵 $A \in \mathbb{R}^{m \times n}$, 若存在 σ 使得 $A^T A v = \sigma^2 v$ ($v \neq 0$), 则 σ 称为 A 的奇异值, 即奇异值是 $A^T A$ 的特征值的非负平方根。因此奇异值永远非负, 而特征值可能为正或负。进一步地, 如果 A 是对称矩阵, 则奇异值等于特征值的绝对值。例如 $A = \begin{bmatrix} -3 & 0 \\ 0 & 2 \end{bmatrix}$ 的特征值为 $-3, 2$, 而奇异值为 $3, 2$ 。

在几何意义上, 特征值描述了矩阵在某些特定方向上对向量的缩放因子, 而奇异值则表示矩阵将单位球拉伸为椭球时的半轴长度, 刻画的是整体的伸缩尺度。换句话说, 奇异值等于 $A^T A$ 的特征值平方根, 而在对称矩阵的情形下, 奇异值就是特征值的绝对值。

6.3.1 为什么要把矩阵转化为向量乘积

一个 $m \times n$ 的矩阵一共有 mn 个单元, 但是一列加一行只有 $m+n$ 个分量。为了快速处理矩阵, 往往需要将其转化为分量乘积的形式。简单来说, 一个图像可以视为一个大型矩阵。要完美复制它, 我们需要 $8mn$ 比特的信息。

高分辨率电视通常有 $m = 1080$ 与 $n = 1920$, 每秒通常有 24 帧, 而且你可能喜欢看彩色画面 (3 个颜色通道)。因此总共需要传输

$$3 \times 8 \times (1080 \times 1920 \times 24) = 3 \times 8 \times 48,470,400$$

比特每秒。这太昂贵而且不可行, 传输器也跟不上表演的速度。

当压缩做得很好时, 你几乎看不出与原始图像的差别。图像的边缘 (即灰度的突然改变) 是最难压缩的部分。如果每个 a_{ij} 都是无关的随机数字, 那么压缩根本不可能成功。我们完全依赖这样的事实: 邻近像素通常具有相似的灰度值。当你跨越边缘时, 会产生一个突跳。现实世界的图像中边缘无处不在, 所以动画相比静止图像更容易压缩。

6.3.2 为什么还需要 SVD 分解

Singular Value Decomposition,

例题 6.12 “王牌旗”法国国旗 A , 意大利国旗 A , 德国国旗 A^T

$$A = \begin{bmatrix} a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} a & a & c & c & e & e \end{bmatrix}$$

国旗有 3 个颜色但是秩还是 1。假设这 36 个单元保持秩 1 的模式 $A = u_1 v_1^T$, 它们甚至可以全部不相同。但是当秩变成 $r = 2$ 时, 我们需要 $u_1 v_1^T + u_2 v_2^T$ 。秩是几就需要几个非线性相关的矩阵, 每个矩阵都可以写成行乘列

例题 6.23 嵌入方形

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

A 的 1 与 0 可以是 1 的方块与 0 的方块。秩仍然是 2, 我们仍然只需要两个项 $u_1 v_1^T + u_2 v_2^T$ 。一个 6×6 图像可以由 24 个数字压缩而来; 一个 $N \times N$ 图像 (N^2 个数字) 可以压缩成 4 个向量 u_1, u_2, v_1, v_2 的 $4N$ 个数字。

注意：这并不是来自 SVD 的选择！例如 $u_1 = (1, 1), u_2 = (1, 0)$ 没有正交； $u_1 = (1, 1)$ 与 $v_2 = (0, 1)$ 也没有正交。SVD 会保证按照顺序选择秩 1 片段。

东邪的 tip 6.4 (正交化的好处)

唯一性：正交基保证分解几乎唯一，不会乱选。

简洁几何意义：任何矩阵 = 旋转 + 拉伸 + 再旋转。

数值稳定：正交基避免冗余，误差不放大。

最优近似：保留前几个奇异值就得到最小误差的低秩近似。

6.3.3 如何寻找正交化基底

ATA 是半正定而且对称的他天然自带一组正交基，AAT 同理我们这样就得倒了两组正交基。

来自 SVD 的选择

$$AA^T u_i = \sigma_i^2 u_i, \quad A^T A v_i = \sigma_i^2 v_i, \quad A v_i = \sigma_i u_i$$

$$AA^T A v_i = \sigma_i^2 A v_i \Rightarrow A v_i \text{ 是 } AA^T \text{ 的特征向量, 对应特征值 } \sigma_i^2.$$

因此 $A v_i = u_i$ ，但此时的 u_i 还不是单位向量，需要计算 $A v_i$ 的长度：

$$\|A v_i\|^2 = (A v_i)^T (A v_i) = v_i^T (A^T A) v_i = v_i^T (\sigma_i^2 v_i) = \sigma_i^2 \|v_i\|^2 = \sigma_i^2.$$

$$\Rightarrow \|A v_i\| = \sigma_i, \quad u_i = \frac{1}{\sigma_i} A v_i.$$

东邪的 tip 6.5

因为特征向量乘一个系数还是特征向量所以要看清到底指的那个，比如这里要求的就是单位化的。单位化的一大好处就是满秩正交矩阵乘自己的转置等于单位矩阵可以消除项。

由 $A v_i = \sigma_i u_i$ 可得

$$A v_i v_i^T = (A v_i) v_i^T = \sigma_i u_i v_i^T,$$

它表示 A 在第 i 个方向上的秩一部分：该矩阵将任意输入向量在 v_i 方向上的分量提取出来，并映射到 u_i 方向上，因而秩为 1（若 $\sigma_i = 0$ 则为 0）。

整体的式子如下

$$A V V^T = U \Sigma V^T, \quad V V^T = I, \quad \Rightarrow A = U \Sigma V^T.$$

其中 Σ 是一个对角矩阵

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r).$$

$$A = \begin{bmatrix} u_1 & u_2 \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix} \begin{bmatrix} v_1^T \\ v_2^T \end{bmatrix} \quad \text{或更简单的} \quad A \begin{bmatrix} v_1 & v_2 \end{bmatrix} = \begin{bmatrix} \sigma_1 u_1 & \sigma_2 u_2 \end{bmatrix}.$$

σ 矩阵在中间，是因为矩阵运算中是后一个矩阵对前一个矩阵作用产生变换，这里 σ 是为了得到 u 的系数（即缩放），所以要放在 U 矩阵的前面。

6.3.4 奇异值分解计算方法

SVD的 u 's 称为左奇异向量 (AA^T 的单位特征向量), v 's 称为右奇异向量 ($A^T A$ 的单位特征向量), σ 's 是奇异值, 就是 AA^T 与 $A^T A$ 的相同特征值的平方根:

在范例 3 (嵌入方形) 中, 下列是对称矩阵 AA^T 与 $A^T A$:

$$AA^T = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad A^T A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}.$$

它们的行列式是 1, 所以 $\lambda_1 \lambda_2 = 1$, 迹 (对角线的和) 是 3:

$$\det \begin{bmatrix} 1 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = \lambda^2 - 3\lambda + 1 = 0,$$

得到

$$\lambda_1 = \frac{3+\sqrt{5}}{2}, \quad \lambda_2 = \frac{3-\sqrt{5}}{2}.$$

λ_1, λ_2 的平方根是

$$\sigma_1 = \frac{\sqrt{5}+1}{2}, \quad \sigma_2 = \frac{\sqrt{5}-1}{2},$$

且 $\sigma_1 \sigma_2 = 1$ 。

最接近 A 的秩 1 矩阵是 $\sigma_1 u_1 v_1^T$, 误差只有 $\sigma_2 \approx 0.6$, 是最佳可能。

AA^T 与 $A^T A$ 的正交单位特征向量是:

$$u_1 = \begin{bmatrix} 1 \\ \sigma_1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} \sigma_1 \\ -1 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ -\sigma_1 \end{bmatrix},$$

全部除以 $\sqrt{1 + \sigma_1^2}$ 。

像通常有满秩, 但是他们确实有低效率的 (low effective) 秩, 这就表示: 很多奇异值很小, 可以被设定成零。我们传输的是低秩的近似压缩时我们丢弃小的 σ 不会严重损失图像的品质,

东邪的 tip 6.6 (对角化的好处)

1. 运算简化: $A^k = P D^k P^{-1}$, 复杂矩阵运算化为标量运算。
2. 结构清晰: 在特征向量基下, 矩阵只表现为伸缩 (按特征值缩放)。
3. 理论方便: 如解微分方程、研究马尔可夫链, 化为一维问题。
4. 易于分析: 长期行为由最大特征值主导, 可用于近似与压缩。

奇异值分解是线性代数的一个亮点, A 的任意 $m \times n$ 矩阵, 方形或矩形, 它的秩是 r 。我们会对角化 A , 但不是用 $X^{-1} A X$, X 里面的特征向量有三个大问题: 他们通常不是正交, 不一定有足够的特征向量, 而且 $Ax = x$ 要求 A 是方形。 A 的奇异向量以完美的方式解决了全部的问题。

6.3.5 奇异值分解也可以满足不满秩的情况

(使用向量 u 's 与 v 's 产生 4 个基础子空间的基底:)

- | | |
|-----------------------|--------------------------|
| $u_1, \dots, u_r$ | 是列空间的一组正交单位基底 |
| $u_{r+1}, \dots, u_m$ | 是左零空间 $N(A^T)$ 的一组正交单位基底 |
| $v_1, \dots, v_r$ | 是行空间的一组正交单位基底 |
| $v_{r+1}, \dots, v_n$ | 是零空间 $N(A)$ 的一组正交单位基底 |

这些 v' s 与 u' s 负责 A 的行空间与列空间, 我们还有 $n-r$ 个 v' s 与 $m-r$ 个 u' s, 分别来自零空间 $N(A)$ 与左零空间 $N(A^T)$ 。他们自动与前面 v' s 与 u' s 正交 (因为整个零空间是正交)。我们现在引入 V 与 U 的所有 v' s 与 u' s, 所以矩阵变成方形, 我们仍然有 $AV = U\Sigma$ 。

$$(m \times n)(n \times n) \quad AV = U\Sigma \quad (m \times m)(m \times n)$$

$$A \begin{bmatrix} v_1 & \cdots & v_r & \cdots & v_n \end{bmatrix} = \begin{bmatrix} u_1 & \cdots & u_r & \cdots & u_n \end{bmatrix} \begin{bmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & \sigma_r \end{bmatrix} \quad (3)$$

新的 Σ 是 $m \times n$, 它就是方程式 (2) 的 $r \times r$ 矩阵加上 $m-r$ 个额外零行以及 $n-r$ 个零列, 真正的改变在于 U 与 V 的形状。这些是方形矩阵且 $V^{-1} = V^T$, 所以 $AV = U\Sigma$ 变成 $A = U\Sigma V^T$, 这就是奇异值分解。

我可以把 V^T 的行乘来自 $U\Sigma$ 的列 $u_i\sigma_i$:

$$\text{SVD: } A = U\Sigma V^T = \sigma_1 u_1 v_1^T + \cdots + \sigma_r u_r v_r^T. \quad (4)$$

然后我们把 σ 以及后面按降序排列最大的排前面。

范例 1. 什么时候

$$A = U\Sigma V^T \quad (\text{奇异值分解})$$

与

$$A = X\Lambda X^{-1} \quad (\text{特征值分解})$$

相等?

解: 矩阵 A 需要具有正交特征向量, 才能使得 $X = U = V$ 。如果 $\Lambda = \Sigma$, 则 A 还需要满足特征值 $\lambda \geq 0$ 。因此 A 必须是一个正半定 (或正定) 对称矩阵。

只有在这种情况下:

$$A = X\Lambda X^{-1} = Q\Lambda Q^T$$

才能与

$$A = U\Sigma V^T$$

一致。

6.3.6 为什么可以去掉小的 σ 项

例: 矩阵 A 的奇异值稳定性

原矩阵

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

—

1. 未去掉最后一行时:

$$A^T A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 9 \end{bmatrix}, \quad \Rightarrow \sigma = 3, 2, 1.$$

2. 去掉最后一行后：得到 3×3 矩阵

$$A' = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}, \quad A'^T A' = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 4 \end{bmatrix},$$

其特征值为 1, 4, 9, 对应奇异值仍为

$$\sigma = 3, 2, 1.$$

3. **结论：**去掉 A 的零行（或零列）不会改变非零奇异值。原因是：零行/零列只会在 $A^T A$ 或 AA^T 中产生额外的 0 特征值，因此不会影响已有的非零奇异值。

命题 6.1 (SVD 的几何意义)

设矩阵 $A \in \mathbb{R}^{m \times n}$ 的奇异值分解为

$$A = U \Sigma V^T,$$

则有以下几何解释：

1. A 可以分解为旋转-拉伸-旋转的组合；
2. A 将单位圆上的向量 x 变换到椭圆上的 Ax ；
3. A 的算子范数为

$$\|A\| = \sigma_1,$$

其中 σ_1 是最大的奇异值，表示最大的增长因子

$$\max_{x \neq 0} \frac{\|Ax\|}{\|x\|};$$

4. A 的极分解为

$$A = QS, \quad Q = UV^T, \quad S = V \Sigma V^T;$$

5. A 的伪逆为

$$A^+ = V \Sigma^+ U^T,$$

它可将列空间中的 Ax 映射回行空间中的 x 。

定理 6.1 (基于 SVD 的 PCA 构造过程如下：)

1. 首先进行数据中心化，如果没有指定主成分的数量，则主成分数至少在样本数和特征数之间确定；
2. 然后对中心化后的数据矩阵应用奇异值分解 (SVD)；
3. 将 `svd_flip_vector` 方法应用于矩阵 U ，该方法在矩阵 U 的每一列中找到最大模元素，提取其符号，并用这些符号乘以矩阵 U ，以保证输出结果的确定性；
4. 每个主成分的解释方差计算为相应奇异值平方除以 $n_{\text{samples}} - 1$ 。然后根据下式计算变换后的数据，主成分的数量取决于所需的保留数量：

$$X_{\text{new}} = X \cdot V = U \cdot S \cdot V^T \cdot V = U \cdot S$$

定理 6.2 (PCA 的附加特性)

(1) 解释方差系数：每个主成分的解释方差系数可以通过变量 `explained_variance_ratio_` 获得，表示数据集在每个主成分轴上的方差比例。

(2) 数据恢复：使用 `inverse_transform()` 方法进行数据恢复，其本质是对 PCA 投影进行逆变换：

$$X_{\text{recovered}} = X_{d-\text{proj}} W_d^T,$$

其中 W_d^T 是前 d 个主成分所组成的矩阵。数据的恢复会存在与丢弃的方差量成比例的损失；原始数据与恢复数据之间的距离平方的平均值称为重构误差。

(3) 增量 PCA：实现为 `IncrementalPCA` 类，允许在处理大规模数据集时更加高效，可以将数据拆分成小块并逐步存储在内存中进行训练。

(4) 随机 PCA：通过设置参数 `svd_solver='randomized'` 来实现，使用随机算法快速计算近似的 d 个主成分。该方法基于随机投影的假设，即数据在投影到低维子空间时仍能较好地保持其结构和性质。但方法的精确度可能较低。

(5) 核 PCA：实现为 `KernelPCA` 类，允许使用核函数进行复杂的非线性投影。类似于支持向量机 (SVM)，其本质是将原始空间中的非线性问题转化为高维特征空间中的线性边界。



定理 6.3 (PCA 的替代方法)

尽管主成分分析 (PCA) 是最常用的降维算法之一，但在一些情况下，根据数据类型不同，也存在一些更为合适的替代方法：

6.3.6.0.1 LLE (Locally Linear Embedding) 局部线性嵌入是一种通过邻居点的线性组合来表示每个数据点，并在低维空间中恢复这些组合的算法。这样可以保持数据的非线性几何结构，在某些对全局性质要求不高的任务中十分有用。然而，该方法计算复杂度较高，并且可能对噪声敏感。

6.3.6.0.2 t-SNE (t-Distributed Stochastic Neighbor Embedding) t-SNE 是一种将高维空间中点之间的相似性转化为概率的算法，并试图最小化高维与低维空间中概率分布的差异。t-SNE 在可视化高维数据时非常有效，但它可能会扭曲数据的整体结构，因为它只考虑点在原始空间中的局部邻近性，而忽略了线性依赖关系。

6.3.6.0.3 UMAP (Uniform Manifold Approximation and Projection) 统一流形近似与投影是一种适用于数据可视化的算法，其思想是数据位于某个可以用邻居图近似的同质流形上。该方法考虑了数据的整体结构，相比 t-SNE 更能适应不同类型的数据，并且在处理噪声与异常值时表现更好。

6.3.6.0.4 Autoencoders 自编码器是一类基于神经网络的模型，通过训练编码器将输入数据转换为低维表示，再通过解码器从该表示中恢复原始数据。自编码器不仅可以用于数据压缩与降维，还可用于去噪和特征提取等多种任务。



第7章 工程基础

2 《Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow》

- └─ = 把机器学习从"会解释"推进到"能做项目"
 - └─ 0 工程工具层（把算法接到真实数据上）
 - └─ 数据清洗
 - └─ Pipeline
 - └─ 训练/验证/测试切分
 - └─ 能力一做一个完整机器学习项目 → 端到端流程
 - └─ 明确业务目标
 - └─ 构造特征
 - └─ 建立基线
 - └─ 能力二训练传统模型 → sklearn 实战
 - └─ 线性模型 / SVM
 - └─ 决策树 / 随机森林
 - └─ 集成学习 / Boosting
 - └─ 能力三训练神经网络 → 深度学习入门
 - └─ MLP
 - └─ CNN
 - └─ RNN / Attention
 - └─ 能力四调优与上线意识 → 让模型能用
 - └─ Grid Search / Random Search
 - └─ 学习曲线与误差分析
 - └─ 保存模型、复现实验、监控效果

第 8 章 训练前的准备

第 9 章 训练前的设计

第 10 章 训练传统模型

10.1 回归树

定义 10.1 (回归树)

回归树是一种非参数化的监督学习方法，用于预测连续型输出变量。它通过递归地划分输入特征空间，并在每个划分区域内用常数值（通常是该区域训练样本的均值）作为预测，从而构建一个分段常数函数近似目标函数。

建立回归树的过程大致可以分为两步：

1. 将预测变量空间（即 $X_1, X_2, \dots, X_p$ 的可能取值构成的集合）分割成 J 个互不重叠的区域 $R_1, R_2, \dots, R_J$ 。
2. 对落入区域 R_j 的每个观测值做同样的预测，预测值等于 R_j 上训练集的响应值的简单算术平均。

举个例子，若在第一步中得到两个区域 R_1 和 R_2 ， R_1 上训练集的平均响应值为 10， R_2 上训练集的平均响应值为 20。那么，对于给定的观测值 $X = x$ ，若 $x \in R_1$ ，则预测值为 10；若 $x \in R_2$ ，则预测值为 20。

现在详细说明上面的第一步。如何构建区域 $R_1, R_2, \dots, R_J$ ？理论上，区域的形状是任意的，但出于模型简化和增强可解释性的考虑，这里将预测变量空间划分成高维矩形，或称盒子（box）。划分区域的目标是找到使模型的残差平方和（RSS）最小的矩形区域 $R_1, R_2, \dots, R_J$ 。

RSS 的定义为

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2.$$

上式中的 $\hat{y}_{R_j}$ 是第 j 个矩形区域中训练集的平均响应值。

东邪的 tip 10.1 (回归树的原理)

回归树相当于一个预处理，先把具有相同规律的点的区域找出来，然后与其他区域区别开。当我们要预测的时候只需要判断点落在那个区域然后预测他为该区域平均数即可。RSS 的大小就是显示我们的刀法是不是精准，就是所有区域的误差加起来。用每一个区域的点减去该区域的平均值的平方和是为了放大误差。回归树的本质可以概括为：

- 回归树 = 把点分区块，每块用一个小模型（常数函数）解释数据；
- 它是一个分段常数函数的几何直观。

回归树的直观理解

1. **数据点视角** 假设你有一堆数据点 (x_i, y_i) ，比如 x 是房子的面积、房间数， y 是房价。把这些点放到坐标系里，就是数据分布的样子。
 2. **划分区域** 回归树做的事就是：不断选择一个特征（例如“面积”），找一个切分点（例如 120 平方米），把数据分成两个区域。然后在每个区域里，再继续切（比如在“房间数”上切一刀），直到区域足够“纯净”。最终数据空间被切成一个个矩形/盒子（box）。
 3. **区域内的模型** 在每个区域 R_j 内，用一个常数模型来解释数据：就是该区域里所有 y 的平均值。所以区域越“纯”，这个平均值越能代表里面的点。预测时：一个新点 x 落在哪个区域，就直接输出该区域的平均 y 。
- 举个例子

假如我们用回归树预测房价：

- 切分 1：面积 ≤ 120 平 $\Rightarrow$ 走左边区域，否则走右边。
- 切分 2（左区域里）：房间数 $\leq 2 \Rightarrow$ 预测 100 万，否则预测 150 万。
- 切分 3（右区域里）：面积 $> 200 \Rightarrow$ 预测 300 万，否则预测 220 万。

结果就是把整个房子数据分成几块，每块用一个平均值解释。

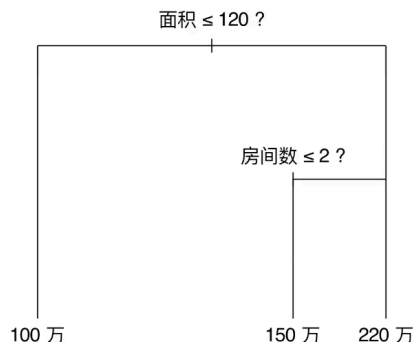


图 10.1: 回归树的结构示意图：每个切分对应一个特征阈值，叶子节点给出预测值。

这也就是为什么叫树。

10.1.1 回归树的具体计算

遗憾的是，要想考虑将特征空间划分为 J 个矩形区域的所有可能性，在计算上是不可行的。因此一般采用一种自上而下 (top-down) 的贪婪 (greedy) 方法：递归二叉分裂 (recursive binary splitting)。

定义 10.2 (递归二叉分裂)

递归二叉分裂 (Recursive Binary Splitting) 是一种用于构建回归树和分类树的贪婪算法。其基本思想是：

1. 从整个特征空间（即所有训练样本的集合）出发，选择一个特征 X_j 及其切分点 s ；
2. 将样本划分为两个矩形区域：

$$R_1(j, s) = \{x \mid x_j \leq s\}, \quad R_2(j, s) = \{x \mid x_j > s\};$$

3. 选择使得残差平方和 (RSS) 最小化的分裂：

$$\min_{j, s} \left\{ \sum_{x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2 \right\},$$

其中 $\hat{y}_{R_m}$ 表示区域 R_m 内训练样本响应的均值。

4. 在每个新生成的区域内，重复上述步骤，直到满足停止条件（如区域样本数过小或树的深度达到上限）。

该方法称为“递归”，因为分裂过程会不断重复；称为“二叉”，因为每次分裂都将一个区域划分为两个子区域；称为“贪婪”，因为每次分裂只考虑当前最优，而不是全局最优。



东邪的 tip 10.2 (贪婪算法-递归二叉分裂)

贪婪算法 (Greedy Algorithm)：在每一步选择中，都作出在当前看来最优的局部选择，而不从整体上进行考虑。‘自上而下’指的是从树的顶端（在此处，所有观测值都属于同一个空间）。开始依次分裂预测变量空间，每个分裂点都会产生两个新的分支。“贪婪”意味着在建立树的每一步中，最优 (best) 分裂的确定仅限于这一步进程，而不是针对全局去选择那些能够在未来过程中构建出更好树的分裂点。二叉分裂法就是一旦他分割了一次，以后的分割都是在分割好的区域继续分割，不会从两个区域里面个拿出一部分产生一个新的区域即使这样会更好。

命题 10.1 (防止过拟合想要全局最优)

上述方法会在训练集中取得良好的预测效果，却很有可能造成数据的过拟合，导致在测试集上效果不佳。原因在于这种方法产生的树可能过于复杂。一个分裂点更少、规模更小（区域 $R_1, R_2, \dots, R_J$ 的个数更少）的树会有更小的方差和更好的可解释性（以增加微小偏差为代价）。

针对上述问题，一种可能的解决办法是：仅当分裂使残差平方和 RSS 的减小量超过某阈值时，才分裂树节点。这种策略能生成较小的树，但可能产生短视的问题：一些起初看起来不值得的分裂在后面可能产生非常好的分裂，也就是说，在下一步中，RSS 大幅减小。

因此，更好的策略是生成一个很大的树 T_0 ，然后通过剪枝 (prune) 得到子树 (sub-tree)。

定义 10.3 (代价复杂性剪枝)

代价复杂性剪枝 (Cost Complexity Pruning, 又称最弱链接剪枝 weakest link pruning) 是一种用于从原始大树 T_0 中选择最优子树的策略。该方法不是考虑所有可能的子树，而是通过引入调整参数 $\alpha \geq 0$ ，在子树的复杂性和训练误差之间进行权衡。

对于给定的子树 T ，其代价复杂度定义为：

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|,$$

其中 $|T|$ 表示子树 T 的终端节点数， R_m 表示第 m 个终端节点对应的区域， $\hat{y}_{R_m}$ 是区域 R_m 内训练集的平均响应值。

当 $\alpha = 0$ 时， $C_\alpha(T)$ 仅度量训练误差，最优子树为原始大树 T_0 ；随着 α 增大，终端节点数较多的子树将因复杂性受到惩罚，使得更小的子树可能成为 $C_\alpha(T)$ 的最优解。因此，通过调节 α 并最小化 $C_\alpha(T)$ ，我们可以在模型复杂度和泛化能力之间找到平衡，从而选出合适的子树。

也就是最开始的大树太复杂了想砍掉一些枝桠，但是砍的太多就预测的太不准了，砍完枝桠的 RSS 就是 C_α 前面的部分。为了鼓励结构尽可能简单，后面加入 T 复杂度的权重，权重值为 α 。权重值越高说明越倾向简答的结构。

东邪的 tip 10.3

这是一种二次优化的思想，每当想引入新的维度的指标的时候都可以加入带权重的数。这正是正则化 / 多目标优化的思想。

例题 10.1 算法 8.1 建立回归树

1. 利用递归二叉分裂在训练集中生成一个大树，只有当终端节点包含的观测值个数低于某个最小值时才停止。
2. 对大树进行代价复杂性剪枝，得到一系列最优子树，子树是 α 的函数。
3. 利用 K 折交叉验证选择 α 。具体做法是将训练集分为 K 折。对所有 $k = 1, 2, \dots, K$ ，有：
 - (a) 对训练集上所有不属于第 k 折的数据重复步骤 1 和步骤 2，得到与 α 一一对应的子树。
 - (b) 求出上述子树在第 k 折上的均方预测误差。
 上述操作结束后，每个 α 会有相应的 K 个均方预测误差，对这 K 个值求平均，选出使平均误差最小的 α 。
4. 找出选定的 α 值在步骤 2 中对应的子树即可。

定义 10.4 (分类树)

分类树 (classification tree) 和回归树十分相似，它们的区别在于：分类树被用于预测定性变量而非定量变量。也就是研究是什么，而不是是多少的问题。在回归树中，对一给定观测值，响应预测值取它所属的终端节点内训练集的平均响应值。与之相对，对分类树来说，给定观测值被预测为它所属区域内的训练集中最常出现的类 (most commonly occurring class)。

分类树的构造过程和回归树也很像。与回归树一样，分类树采用递归二叉分裂。但在分类树中，RSS 无法作为确定二叉分裂点的准则。一个很自然的替代指标是分类错误率 (classification error rate)。既然要将给定区域内的观测值都分到此区域的训练集上最常出现的类中，那么分类错误率的定义为：此区域的训练集中非最常见

类所占的比例。其表达式如下：

$$E = 1 - \max_k (\hat{p}_{mk}), \quad (8.5)$$

上式中的 $\hat{p}_{mk}$ 表示第 m 个区域的训练集中的第 k 类所占比例。但事实上，分类错误率这一指标在构建树时不够敏感。在实践中，另外两个指标更有优势。

定义 10.5 (基尼系数)

基尼系数 (Gini index) 用于度量分类树节点的纯度，其定义为

$$G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk}), \quad (8.6)$$

其中， $\hat{p}_{mk}$ 表示第 m 个区域的训练集中第 k 类所占的比例。

基尼系数可以理解为 K 个类别的总方差。如果所有 $\hat{p}_{mk}$ 的取值都接近于 0 或 1，则基尼系数会很小。因而，基尼系数常被视为度量节点纯度 (purity) 的指标：若基尼系数较小，就意味着该节点包含的观测值几乎都来自同一类。也是 Bernoulli 方差

- p 表示某个类别的占比；
- $1 - p$ 表示“其他类别的占比”；
- $p(1 - p)$ 则反映了一个类与其他类的对立程度：当分布均衡时 (p 和 $1 - p$ 接近)，该值最大；当一边占绝对优势时，该值趋近于 0。

熵 (Entropy) 也可以用来替代基尼系数作为分类树分裂指标，其定义为

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}, \quad (8.7)$$

其中， $\hat{p}_{mk}$ 表示第 m 个区域的训练集中第 k 类所占的比例。

由于 $0 \leq \hat{p}_{mk} \leq 1$ ，可知 $0 \leq -\hat{p}_{mk} \log \hat{p}_{mk}$ 。显然，如果所有 $\hat{p}_{mk}$ 的取值都接近于 0 或 1，则熵值接近于 0。因此，与基尼系数类似，如果一个节点的纯度较高，则熵值较小。事实上，基尼系数和熵在数值上是相当接近的。

在建立分类树的过程中，无论基尼系数还是熵，通常都用于度量某个分裂点的分类质量。与分类错误率相比，这两种方法对节点纯度更敏感。上述三种方法都可以用于树的剪枝，但若希望追求较高的预测准确性，最好选择分类错误率。

10.2 装代法

定义 10.6 (装袋法 (Bagging))

在一般情况下难以取得多个训练集，因此可以使用自助法 (Bootstrap) 作为替代，即从某一个单一的训练集中重复取样，生成 B 个不同的自助抽样训练集。然后在第 b 个自助抽样训练集上拟合模型并得到预测值 $\hat{f}^{*b}(x)$ 。最后对所有预测值取平均，得到装袋法预测函数：

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

决策树 (Decision Tree) 通常具有较高的方差 (high variance)，这意味着如果将同一个训练集随机拆分成两部分，分别建立回归树，结果可能会差别很大。相比之下，像线性回归这类“简单模型”通常方差较小，在不同的数据集上结果差别不会太大。为了降低这种高方差，统计学习里常用一种通用技巧，称为自助法聚集 (bootstrap aggregation)，简称 bagging。具体做法是：从训练集里反复有放回地抽样 (bootstrap)，得到多个子集；在每个子集上分别建立一个模型（如决策树）；最后把这些模型的预测结果通过平均或投票的方式组合起来。这样可以显著降低模型的方差，提高泛化性能。装袋法可以显著改善回归模型的预测效果，尤其对决策树十分有效。具体

做法是：利用 B 个自助抽样训练集分别构造 B 棵回归树，再对其预测结果取平均。这些树通常根深叶茂且未剪枝，每棵树方差高但偏差小，通过平均可有效降低方差。事实证明，成百上千棵树的组合能大幅提升预测精度。

在分类问题中，装袋法同样适用。对于一个给定测试样本，记录 B 棵树的预测类别，再采用**多数投票**(majority vote)，即将出现频率最高的类别作为整体预测结果。

定义 10.7 (袋外误差估计)

装袋法可通过袋外 (out-of-bag, OOB) 样本直接估计测试误差，而无需交叉验证。在自助抽样中，每棵树约使用训练样本的三分之二，剩余三分之一即为该树的 OOB 样本。对某个观测值，可用未包含它的约 $B/3$ 棵树进行预测，并将结果求平均（回归）或多数投票（分类），得到该观测值的 OOB 预测。综合所有样本即可计算 OOB 均方误差或分类误差，从而得到装袋法测试误差的有效估计。事实证明，当 B 足够大时，OOB 误差与留一法交叉验证误差近似等价。



即使装袋法树的集合比单个树解释数据要困难得多，也可以用 RSS（针对装袋法回归树）或基尼系数（针对装袋法分类树），对各预测变量的重要性做出整体概括。

东邪的 tip 10.4

bagging 和 pasting 的区别是 bagging 要把东西放回去，pasting 不用

10.3 随机森林

定义 10.8 (随机森林)

随机森林 (random forest) 通过对树做去相关处理，实现了对装袋法树的改进。在随机森林中，需要对自助抽样训练集建立一系列决策树，这与装袋法类似。不过，在建立这些决策树时，每个分裂点不再考虑全部的 p 个预测变量，而是从中随机抽取 m 个预测变量作为候选。该分裂点所用的预测变量只能从这 m 个候选变量中选择。在每个分裂点处都要重新抽取 m 个预测变量，通常 $m \approx \sqrt{p}$ 。也就是说，每个分裂点所考虑的预测变量个数约等于总预测变量数的平方根。例如，在 Heart 数据集中共有 13 个预测变量，则取 $m = 4$ 。



这个目的就为了防止用装袋法生成的树都长的太相似。比如说都会选择最重要的变量作为顶部分裂点。问题是，与对不相关的量求平均相比，对许多高度相关的量求平均带来的方差减小程度是无法与前者相提并论的。具体而言，在这种情况下，这意味着装袋法与单个树相比不会带来方差的明显降低。但是随机森林因为是随机取出 m 个预测变量作为候选，那么最强的变量可能都不在里面，这样的话次强变量等等都可能作为顶部分裂点等塑造出不一样的树。

- 10 个人都参考同一个答案抄作业 $\Rightarrow$ 他们错的地方一样，平均结果还是错的。
- 10 个人各自独立思考 $\Rightarrow$ 有人错有人对，平均结果更接近真值。

分类聚类然后神经网络把深度学习结合到 Matlab 上边做边学

10.4 boosting 提升法

定义 10.9 (提升法 (Boosting))

提升法 (boosting, 最初被称为假设提升) 是指可以将几个弱学习器组合成一个强学习器的任意集成方法。大多数提升法的总体思想是循环训练预测器，每一次都对其前序预测器做出一些改正。常见的提升法包括 AdaBoost (Adaptive Boosting) 和梯度提升 (Gradient Boosting)。



东邪的 tip 10.5

Boosting 的核心问题是：如何把多个只比随机猜好一点的弱学习器，通过顺序训练和加权组合，提升成一个预测能力强的模型。

定义 10.10 (AdaBoost)

AdaBoost 是一种提升方法，其核心思想是在训练新的预测器时，更多地关注前序预测器欠拟合或分类错误的训练实例。具体而言，算法会先训练一个基础分类器，并用它对训练集进行预测；随后增加被错误分类样本的相对权重，再用更新后的权重训练下一个分类器。如此反复迭代，使后续分类器逐渐专注于前面模型难以处理的样本。



$$\text{初始化样本权重: } w_i^{(1)} = \frac{1}{m}, \quad \text{加权错误率: } r_t = \sum_{i=1}^m w_i^{(t)} \mathbf{1}(h_t(x_i) \neq y_i) \quad (10.1)$$

$$\text{弱分类器权重: } \alpha_t = \eta \log \frac{1-r_t}{r_t}, \quad \text{样本权重更新: } w_i^{(t+1)} = \frac{w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \quad (10.2)$$

$$\text{最终强分类器: } H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right), \quad \hat{y}(x) = \arg \max_k \sum_{\substack{t=1 \\ h_t(x)=k}}^T \alpha_t \quad (10.3)$$

东邪的 tip 10.6

AdaBoost 的核心思想是：提高上一轮被错误分类样本的权重，使后续弱学习器更加关注并修正这些 bad case。提高准确率高的人的话语权。只有第一轮是真正的错误概率后面的都是加权后的错误概率每一轮的优化目标就是加权错误概率最小这个也就是让机器学习的方向。

笔记 在 AdaBoost 中，第 j 个弱学习器的权重可以写为

$$\alpha_j = \eta \log \frac{1-r_j}{r_j},$$

其中， α_j 表示第 j 个弱学习器在最终集成模型中的投票权重或可信度； η 表示学习率，用来控制每个弱学习器对整体模型的影响程度； r_j 表示第 j 个弱学习器在当前样本权重下的加权错误率； $1-r_j$ 表示其加权正确率； $\frac{1-r_j}{r_j}$ 表示该弱学习器正确分类相对于错误分类的优势比。 r_j 越小，说明弱学习器表现越好，此时 α_j 越大，它在最终模型中的话语权也越大。

东邪的 tip 10.7

AdaBoost 就像一个军师训练多个谋士。每一轮都会训练一个新谋士，表现越好的谋士，在最终决策中的投票权越高；而上一轮被判错的案件，会在下一轮变得更重要，使新的谋士更倾向于学习如何处理这些 bad case。

笔记 η 是人为设定的学习率超参数，用来控制每一轮弱学习器权重的缩放幅度。 η 越小，模型每轮更新越保守，通常需要更多弱学习器； η 越大，模型学习越快，但更容易过拟合。

东邪的 tip 10.8

我这样理解对不对 Ada boost 先用弱分类器训练得到一个模型和他的权重，然后再调整坏的权重然后用另一个弱分类器训练，然后此时两个训练器了。然后最后的决策是他们两商量着做

定义 10.11 (符号函数)

符号函数 $\text{sign}(z)$ 用于判断实数 z 的符号，定义为

$$\text{sign}(z) = \begin{cases} +1, & z > 0, \\ 0, & z = 0, \\ -1, & z < 0. \end{cases}$$

在 AdaBoost 中，它用于将多个弱分类器的加权投票结果转化为最终类别。

**东邪的 tip 10.9**

由于弱分类器优化的加权错误率所以给 badcase 更高的权重就会尽量保证这个是对的

- 二分类：使用符号函数判断加权和的正负， $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$ 。
- 多分类：统计每个类别的加权投票，选择权重和最大的类别， $\hat{y}(x) = \arg \max_k \sum_{h_t(x)=k}^T \alpha_t$ 。

10.4.1 GBRT**定义 10.12 (梯度提升回归树 (Gradient Boosted Regression Tree, GBRT))**

GBRT 是一种基于 Boosting 思想的集成学习算法。它通过迭代训练多棵回归树，使每一棵新树拟合当前模型在损失函数上的负梯度方向，并将这些弱学习器累加形成最终的强预测模型。

**东邪的 tip 10.10**

- 梯度是由函数对各个变量的偏导数组成的向量。
- 梯度方向表示函数值上升最快的方向。
- 负梯度方向表示函数值下降最快的方向。

东邪的 tip 10.11

本质就是先把大方向弄正确，每一步都是把新的大方向弄正确，然后慢慢补细节。这个就是第一步先都预测常数使得残差最小，然后由真实值-预测值算出残差，然后用残差训练一个新模型，新模型的指标就是残差最小这个也就是负梯度方向，然后新的函数乘上一个学习率加到前一个函数后面。

$$\text{初始化模型: } F_0(x) = \arg \min_{\gamma} \sum_{i=1}^m L(y_i, \gamma)$$

$$\text{负梯度/伪残差: } r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

$$\text{拟合负梯度: } h_m = \arg \min_h \sum_{i=1}^m (r_{im} - h(x_i))^2$$

$$\text{模型更新: } F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^J \gamma_{jm} \mathbf{1}(x \in R_{jm})$$

东邪的 tip 10.12

- 损失函数 $L(y_i, F(x_i))$: 衡量真实值 y_i 与当前模型预测值 $F(x_i)$ 之间的误差，GBRT 每一轮都围绕损失函数下降来更新模型。
- 初始模型 $F_0(x)$: GBRT 的初始预测函数，通常是一个常数；在平方误差损失下，它等于所有真实值的平均值。
- 负梯度/伪残差 r_{im} : 第 m 轮中第 i 个样本需要修正的方向；在平方误差损失下， $r_{im} = y_i - F_{m-1}(x_i)$ ，也就是当前残差。

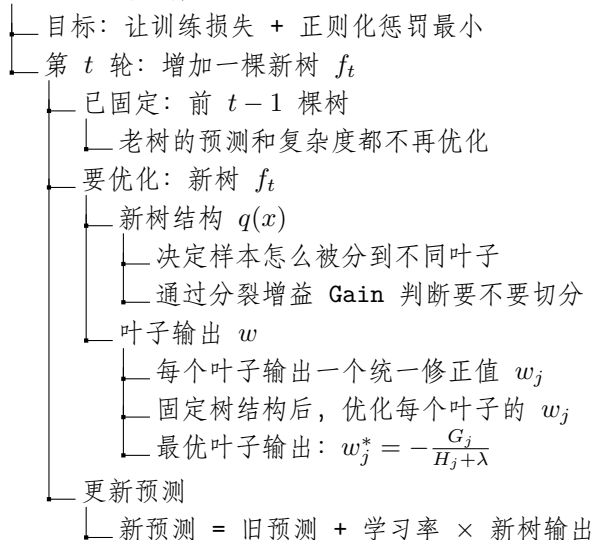
- 叶子节点：决策树中样本经过一系列划分后最终落到的终点区域。
- R_{jm} ：第 m 棵树中第 j 个叶子节点对应的样本区域。
- γ_{jm} ：第 m 棵树中第 j 个叶子节点的输出值，表示落入该叶子节点的样本统一需要修正的大小。

东邪的 tip 10.13

每个叶子节点本质上对应一组样本区域，通常需要注意三件事：该叶子包含哪些样本、这些样本当前的残差/负梯度是多少，以及该叶子的输出值 γ 是多少。其中真正用于更新模型的是叶子节点的输出值 γ 。如果某个样本落入当前叶子节点管辖的区域，那么该样本在本轮相对于上一版模型的修正量就是 $\nu\gamma$ ，也就是“学习率 ν 乘以叶子输出值”。

东邪的 tip 10.14

叶子节点可以理解决策树分叉路径的末端，是模型真正给出输出值的位置。样本从根节点进入树后，会根据每个分裂条件不断向下选择路径，最终落到某个叶子节点；该叶子节点的输出值 γ_{jm} 就是这棵树对该样本给出的修正量。

10.4.2 XGBoosting**XGBoost 总目标****定义 10.13 (XGBoost)**

XGBoost (Extreme Gradient Boosting) 是一种高效、可扩展的梯度提升决策树 (Gradient Boosting Decision Tree, GBDT) 算法实现。它通过逐轮训练多棵决策树，使每一棵新树修正前面模型的预测误差，并将所有树的输出累加形成最终预测模型。

相比普通 GBDT，XGBoost 在目标函数中加入了正则化项，并利用一阶梯度和二阶梯度信息来优化每一轮的树结构与叶子节点权重，因此通常具有更强的泛化能力、更好的过拟合控制能力和更高的训练效率。♣

东邪的 tip 10.15

XGBoost 他优化的是损失函数是训练损失 + 正则化，L2 正则

 **笔记** [优化函数]

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

东邪的 tip 10.16

训练损失就是：模型在训练样本上预测错了多少。比如回归任务里，常用平方误差。训练目标就是整个目标函数最小

 笔记 [正则项]

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

- γ : 控制叶子节点数量的惩罚强度。 γ 越大，模型越不愿意继续分裂出新的叶子节点，树会更简单。
- λ : 控制叶子节点输出值大小的惩罚强度。 λ 越大，叶子输出值 w 会被压得越小，模型每一轮更新更加保守。
- T : 这棵树最终有几片叶子，也就是有几个终端节点。
- w : 一棵树所有叶子节点输出值组成的向量。

例如一棵树有 3 个叶子节点，输出分别为 0.8, -0.3, 0.1，则

$$w = [0.8, -0.3, 0.1], \quad \|w\|^2 = 0.8^2 + (-0.3)^2 + 0.1^2 = 0.74.$$

 笔记 [所有树对 x_i 的输出之和就是预测]

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

 笔记 [加入第 t 棵树的优化函数。]

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

这个其实就是前半部分就是加入 f_t 的以后新的预测与真实的求损失函数，后面的其实就是加入新树的代价，也就是新树的复杂度。因为老树的复杂度已经是固定的了。

 笔记 XGBoost 在第 t 轮中，用二阶泰勒展开近似加入新树后的损失函数：

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

其中， g_i 是一阶梯度，表示当前预测应该往哪个方向修正； h_i 是二阶梯度，表示损失函数在当前位置的曲率信息； $f_t(x_i)$ 是新树对第 i 个样本给出的修正量。

这一步的作用是把原本不好直接优化的损失函数，近似成关于 $f_t(x_i)$ 的二次形式，从而方便后面求叶子节点最优权重和分裂增益。

例题 10.2 梯度如何影响计算 以平方误差 $l(y_i, \hat{y}_i) = (\hat{y}_i - y_i)^2$ 为例，对 $\hat{y}_i$ 求导得到一阶梯度 $g_i = 2(\hat{y}_i - y_i)$ 。若 $y_i = 10, \hat{y}_i = 6$ ，则 $g_i = 2(6 - 10) = -8$ ，负梯度 $-g_i = 8$ ，说明预测值应该往上调。

但新树不是给每个样本单独记修正量，而是把相似样本分到同一个叶子，并输出统一修正值。例如样本 1 的负梯度是 8，样本 3 的负梯度是 2，若它们落入同一叶子，则叶子可能输出折中值 5。当学习率 $\eta = 0.1$ 时，实际更新为 $0.1 \times 5 = 0.5$ ，所以样本 1 和样本 3 都加 0.5。因此，梯度只告诉模型修正方向和需求强弱，最终改多少由叶子输出、学习率和正则化共同决定。

东邪的 tip 10.17

本质上就是把 $f_t(x_i)$ 当成了 Δx ，然后损失函数是固定的比如这个例子里面就是平方误差。他的二阶导就是 2

 笔记 二次函数求最小值：对 w_j 求导，令导数等于 0：

$$G_j + (H_j + \lambda)w_j = 0, \quad w_j^* = -\frac{G_j}{H_j + \lambda}.$$

这就是叶子节点最优输出值。

 **笔记** [分裂增益 Gain] XGBoost 用分裂增益判断一个节点是否值得继续切分：

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma$$

其中, G_L, H_L 是左子节点的一阶、二阶梯度之和, G_R, H_R 是右子节点的一阶、二阶梯度之和, G, H 是父节点的一阶、二阶梯度之和。 λ 控制叶子权重的 L2 正则, γ 是新增叶子的复杂度惩罚。

如果 $Gain > 0$, 说明这一刀能让目标函数下降, 值得切分; 如果 $Gain \leq 0$, 说明这一刀不划算, 不切。

10.4.3 LightGBM

LightGBM 总目标

- 目标: 在保持 GBDT 精度的同时, 让训练更快、内存更省
- 本质: 仍然是 GBDT / GBM
 - 每一轮增加一棵新树
 - 新树仍然拟合当前模型的负梯度 / 残差
 - 仍然通过分裂增益判断节点怎么切分
- 核心瓶颈: 普通 GBDT 找分裂太慢
 - 对每个特征都要扫描大量样本
 - 复杂度大致受 $\#data \times \#feature$ 影响
 - 数据越大、特征越多, 训练越慢
- 加速一: Histogram 直方图算法
 - 连续特征先离散成 bin
 - 不再枚举所有连续取值
 - 只在有限个 bin 上找切分点
 - 构建直方图复杂度: $O(\#data \times \#feature)$
 - 找切分点复杂度: $O(\#bin \times \#feature)$
- 加速二: GOSS, 减少样本数量
 - 按梯度绝对值排序样本
 - 大梯度样本: 模型还没学好, 信息量大
 - 小梯度样本: 模型已经学得比较好, 信息量小
 - 保留前 $a \times 100\%$ 的大梯度样本
 - 从剩余小梯度样本中随机采样 $b \times 100\%$
 - 对采到的小梯度样本乘上权重 $\frac{1-a}{b}$
 - 目的: 补偿被丢掉的小梯度样本
 - 尽量不改变原始数据分布
 - 用近似增益 $\tilde{V}_j(d)$ 代替全量增益 $V_j(d)$
- 加速三: EFB, 减少特征数量
 - 利用稀疏特征的互斥性
 - 很多 one-hot 特征不会同时非零
 - 可以把互斥特征打包成一个 bundle
 - 特征数从 $\#feature$ 变成 $\#bundle$
 - 复杂度从 $O(\#data \times \#feature)$ 降到 $O(\#data \times \#bundle)$
- 最终效果
 - GOSS 处理样本多的问题
 - EFB 处理特征多的问题
 - Histogram 让分裂搜索更快
 - 所以 LightGBM = 更轻、更快的 GBDT 实现

东邪的 tip 10.18

$V_j(d)$ 是全量样本上的真实分裂增益； $\tilde{V}_j(d)$ 是 GOSS 采样后的近似分裂增益。

定义 10.14 (原始分裂增益 $V_j(d)$)

$V_j(d)$ 表示在第 j 个特征上，以 d 为切分点时的分裂增益：

$$V_j(d) = \frac{1}{n} \left[\frac{\left(\sum_{x_i: x_{ij} \leq d} g_i \right)^2}{n_l^j(d)} + \frac{\left(\sum_{x_i: x_{ij} > d} g_i \right)^2}{n_r^j(d)} \right].$$

它用全量样本计算该切分点的收益，是 GOSS 近似分裂增益 $\tilde{V}_j(d)$ 要近似的对象。

**定义 10.15 (核心公式：GOSS 近似分裂增益 Gradient-based One-Side Sampling)**

GOSS 发生在抉择要不要分叉的时候。在 GOSS 中，先保留梯度绝对值最大的前百分 a 的样本，记为集合 A ；再从剩余的小梯度样本中随机采样百分之 b ，记为集合 B 。然后用下面的近似分裂增益代替全量数据上的分裂增益：

$$\tilde{V}_j(d) = \frac{1}{n} \left[\frac{\left(\sum_{x_i \in A_l} g_i + \frac{1-a}{b} \sum_{x_i \in B_l} g_i \right)^2}{n_l^j(d)} + \frac{\left(\sum_{x_i \in A_r} g_i + \frac{1-a}{b} \sum_{x_i \in B_r} g_i \right)^2}{n_r^j(d)} \right].$$

其中： A_l 就是 left 梯度前百分之 a 的点， A_r 就是右边前百分之 a 点，第 j 个特征在切分点 d 上的分裂增益。比如在年龄这个特征 25 分裂的收益， n 是当前这个节点里的样本总数，左边的样本数就是 $n_l^j(d)$ 个，右边的样本数就是 $n_r^j(d)$ 个。

这里 g_i 表示第 i 个样本的梯度； a 表示保留的大梯度样本比例； b 表示从小梯度样本中采样的比例； $\frac{1-a}{b}$ 是小梯度样本的放大系数，用来补偿没有被采到的小梯度样本。

**东邪的 tip 10.19**

因为是和的平方，个数从原来的 n 变成了 n^2 所以最外面还得再除一次 n

东邪的 tip 10.20

- GOSS 变快，是因为它不用让所有小梯度样本都参与分裂增益计算。
- 大梯度样本：模型还没学好，信息量大，所以全部保留。
- 小梯度样本：模型已经学得比较好，信息量小，所以只抽一部分。
- 被抽中的小梯度样本：乘上一个放大系数，代表那些没被抽中的小梯度样本。



笔记 这个长的像顺序统计量，前百分之 a 本来就是顺序统计量的表述。这个 v_j 就是从原来每一个都要取每一个权重都一样变成了取一部分。

例题 10.3

d 表示某个特征上的切分点。

例如，如果特征是年龄，切分点是 25，那么：

年龄 ≤ 25 走左边， 年龄 > 25 走右边。

东邪的 tip 10.21 ($\frac{1-a}{b}$ 的无偏补偿)

GOSS 只采样一部分小梯度样本，所以直接求和会低估小梯度样本的总体贡献。乘上 $\frac{1-a}{b}$ 后，有

$$\mathbb{E} \left[\frac{1-a}{b} \sum_{x_i \in B} g_i \right] = \sum_{x_i \in A^c} g_i.$$

因此， $\frac{1-a}{b}$ 的作用是让采样小梯度的加权和成为全体小梯度梯度和的无偏估计。

定义 10.16 (Histogram 直方图算法)

Histogram 直方图算法是 LightGBM 用来加速分裂搜索的方法。它先将连续特征离散成有限个 bin，然后在训练决策树时不再枚举所有连续取值作为切分点，而只在 bin 的边界上寻找最佳切分点。

因此，构建直方图的复杂度为

$$O(\#data \times \#feature),$$

寻找切分点的复杂度为

$$O(\#bin \times \#feature).$$

由于通常有 $\#bin \ll \#data$ ，所以 Histogram 可以显著加快分裂点搜索。

**定义 10.17 (EFB: 互斥特征捆绑)**

EFB (Exclusive Feature Bundling) 利用高维稀疏特征的互斥性，把多个几乎不会同时非零的特征合并成更少的 bundle。

设原始特征数为 $\#feature$ ，捆绑后的特征组数为 $\#bundle$ ，且通常有

$$\#bundle \ll \#feature.$$

因此，直方图构建复杂度可以从

$$O(\#data \times \#feature)$$

降低为

$$O(\#data \times \#bundle).$$

**东邪的 tip 10.22**

- Histogram: 把每个连续特征的取值离散成若干个 bin。目的：减少每个特征上的候选切分点数量，也就是减少 d 的数量。
- EFB: 把多个互斥的稀疏特征打包成一个 bundle。目的：减少有效特征数量，也就是减少 j 的数量。

原来要搜索： 所有特征 $j \times$ 所有切分点 d

Histogram 后： 所有特征 $j \times$ 少量 bin 切分点 d

EFB 后： 少量 bundle $j \times$ 少量 bin 切分点 d

例题 10.4 EFB 合并互斥 one-hot 特征 假设广告/推荐场景中有城市 one-hot 特征：

北京, 上海, 深圳, 广州.

同一用户同一时刻只属于一个城市，因此这些特征互斥。例如

$$\text{北京} = 1 \Rightarrow \text{上海} = \text{深圳} = \text{广州} = 0.$$

原始特征为：

用户	北京	上海	深圳	广州
A	1	0	0	0
B	0	1	0	0
C	0	0	1	0
D	0	0	0	1

EFB 可将其捆绑成一个特征 城市_bundle:

$$\text{北京} \mapsto 1, \quad \text{上海} \mapsto 2, \quad \text{深圳} \mapsto 3, \quad \text{广州} \mapsto 4.$$

捆绑后：

用户	城市_bundle
A	1
B	2
C	3
D	4

因此，特征数从 4 降为 1，直方图构建复杂度从

$$O(\#data \times 4) \text{ 降为 } O(\#data \times 1).$$

东邪的 tip 10.23 (不是一类合并成一类而是只要互斥就可以放在一类)

EFB 可以理解为训练前的“合并同类项”：它把许多很少同时出现的稀疏特征列，打包成更少的 bundle 特征。

但 EFB 的合并依据不是语义类别，例如并不是因为

北京, 上海, 深圳, 广州

都属于城市，所以才合并。

它真正关心的是特征之间是否互斥，即：

$$x_a \neq 0 \Rightarrow x_b \approx 0.$$

如果两个特征几乎不会同时非零，就可以放入同一个 bundle。bundle 也就是特征束也就是在训练前先简化分化，分类以后改变映射关系从而矩阵的维度

 **笔记**于是，直方图不只是统计每个桶里的样本数，还会把同一个桶中的梯度累加起来：

桶	样本数	$\sum g_i$
桶 1	1	-2
桶 2	2	$1 + 3 = 4$
桶 3	1	-1
桶 4	1	2

这张按桶汇总样本数和梯度信息的统计表，就是 LightGBM 中所说的直方图。

定理 10.1 (LightGBM 的 Leaf-wise 生长策略)

LightGBM 采用 leaf-wise 的树生长策略。在每一步分裂时，它不会像 level-wise 那样按层同时扩展所有叶子节点，而是优先选择当前分裂增益最大的叶子节点继续分裂。

因此，leaf-wise 通常可以更快降低训练损失，但也更容易生成较深的局部路径，从而带来过拟合风险。实际训练中通常需要通过

`num_leaves`, `max_depth`

等参数控制树的复杂度。



 **笔记** [LightGBM 常用复杂度控制参数] Leaf-wise 生长会优先分裂增益最大的叶子节点，因此训练损失下降快，但也更容易长出过深的局部路径。实际训练中通常用以下参数控制模型复杂度：

- `num_leaves`: 控制一棵树最多有多少个叶子节点，是 leaf-wise 中最重要的复杂度参数。
- `max_depth`: 限制树的最大深度，防止某些路径过深。
- `min_data_in_leaf`: 限制每个叶子节点至少包含多少样本，防止叶子过小导致过拟合。
- `learning_rate`: 控制每棵新树对模型的修正幅度，越小越保守，通常需要更多树。
- `lambda_l1`, `lambda_l2`: 分别表示 L1 和 L2 正则化，用来惩罚模型复杂度。

东邪的 tip 10.24

EFB 的关键不只是“把互斥特征打包”，还包括两个细节：冲突率 (**conflict rate**) 和图着色 (**graph coloring**)。实际数据中，并不是所有特征都能做到完全互斥。LightGBM 允许少量特征冲突，只要冲突率足够低，这些特征仍然可以被放进同一个 bundle。

为了找到合适的 bundle，LightGBM 会把特征看成图中的节点。如果两个特征不能互斥，就在它们之间连边。这样，寻找最少 bundle 的问题就可以转化为图着色问题：相互冲突的特征不能使用同一种颜色，同一种颜色对应一个 bundle。实际算法中通常使用贪婪策略近似求解。

第 11 章 训练神经网络

第 12 章 调优与上线意识

第 13 章 QR 分解

定义 13.1 (正交单位向量组)

向量组 $\mathbf{q}_1, \dots, \mathbf{q}_n$ 是正交单位, 若

$$\mathbf{q}_i^T \mathbf{q}_j = \begin{cases} 0, & i \neq j \text{ (正交向量)}, \\ 1, & i = j \text{ (单位向量: } \|\mathbf{q}_i\| = 1) \end{cases}.$$

一个有正交列的矩阵都将被记为特征字母 Q 。



矩阵 Q 很容易处理, 因为

$$Q^T Q = I.$$

在矩阵语言中再一次说明: 若列向量 $\mathbf{q}_1, \dots, \mathbf{q}_n$ 正交单位, 则 Q 的列正交单位; 注意 Q 不一定需要是方阵 (列正交即可, 称为半正交矩阵)

矩阵 Q 是方阵时, 若满足

$$Q^T Q = I,$$

则有 $Q^T = Q^{-1}$, 也就是说转置矩阵等于逆矩阵。

进一步说明: 当 Q 是矩形矩阵时, 只要 $Q^T Q = I$, 此时 Q^T 只是 Q 的左逆矩阵。若 Q 是方阵, 我们还可以得到

$$Q Q^T = I,$$

因此 Q^T 既是左逆也是右逆, 即为 Q 的双边逆矩阵。在这种情形下, Q 的行向量也像列向量一样正交归一, 我们称这样的方阵 Q 为正交矩阵。

东邪的 tip 13.1 (正交化的好处需要牢记)

只有弄清楚了我们分解的目的才能聚焦在分解的手段

一个置换矩阵是一个方阵, 它的每一行、每一列里都有且仅有一个元素是 1, 其余元素全是 0。

换句话说, 置换矩阵是单位矩阵 I 的行或列经过某种重新排列 (permutation) 得到的矩阵。置换矩阵都是正交矩阵

例题 13.1 反射 若 u 是任意的单位向量, 令

$$Q = I - 2uu^T.$$

注意 uu^T 是一个矩阵, 而 $u^T u$ 是一个数字。由于 $\|u\|^2 = 1$, 可得

$$Q^T = I - 2uu^T = Q, \quad Q^T Q = I - 4uu^T + 4uu^T uu^T = I. \quad (2)$$

因此, 反射矩阵 $I - 2uu^T$ 对称而且正交。如果把它平方, 你会得到单位矩阵:

$$Q^2 = Q^T Q = I.$$

这意味着通过镜子反射两次会回到原始状态, 就好像 $(-1)^2 = 1$ 一样。注意在式 (2) 的 $4uu^T uu^T$ 中, 利用了 $u^T u = 1$ 。

东邪的 tip 13.2

正交矩阵还能避免求逆矩阵的麻烦。

旋转会保有每个向量的长度，反射 (reflections) 也是，置换 (permutations) 也是，用任何正交矩阵去乘也一样——长度与角度没有改变。

证明：

$$\|Qx\|^2 = \|x\|^2,$$

因为

$$(Qx)^T(Qx) = x^T Q^T Q x = x^T I x = x^T x.$$

定理 13.1 (正交矩阵保持长度与内积)

若 Q 有正交单位列 (即 $Q^T Q = I$)，它会保持向量的长度不变：

$$\text{对于每个向量 } x, \quad \|Qx\| = \|x\|. \quad (3)$$

同时， Q 也会保持点积：

$$(Qx)^T(Qy) = x^T Q^T Q y = x^T y.$$

这应用了 $Q^T Q = I$ ！



13.1 转置子空间等知识回顾

定理 13.2 (微积分与转置)

让我插入一点点微积分，这是极其重要的，否则我不会在讲线性代数时停下来。(实际上，这里是针对函数 $x(t)$ 的线性代数。) 矩阵在这里变成了一个导数运算： $A = \frac{d}{dt}$ 。要找到这个特殊 A 的转置 (更准确地说是伴随算子 adjoint)，我们需要先定义函数 $x(t)$ 与 $y(t)$ 的内积。

$$x^T y = (x, y) = \int_{-\infty}^{\infty} x(t) y(t) dt.$$

因此，内积从有限维的求和 $\sum x_k y_k$ 变成了函数的积分。

—

矩阵的转置满足

$$(Ax, y) = (x, A^T y).$$

对于 $A = \frac{d}{dt}$ ，我们有

$$(Ax, y) = \int_{-\infty}^{\infty} \frac{dx}{dt} y(t) dt = \int_{-\infty}^{\infty} x(t) \left(-\frac{dy}{dt}\right) dt = (x, A^T y).$$

这里用了分部积分法 (integration by parts)。在这个过程中，导数从第一个函数 $x(t)$ 转移到第二个函数 $y(t)$ ，并且会产生一个负号。这告诉我们：**导数的转置是它的相反数**。

—

因此，导数算子是反对称的 (antisymmetric)：

$$A = \frac{d}{dt}, \quad A^T = -\frac{d}{dt}.$$

对称矩阵满足 $S^T = S$ ，而反对称矩阵满足 $A^T = -A$ 。线性代数中也包含微分与积分 (见第 8 章)，因为它们都是线性算子。



套用分部积分公式：

$$\int u'(t) v(t) dt = \left[u(t) v(t) \right]_{-\infty}^{\infty} - \int u(t) v'(t) dt.$$

这里令

$$u(t) = x(t), \quad u'(t) = \frac{dx}{dt}, \quad v(t) = y(t).$$

那么:

$$\int \frac{dx}{dt} y(t) dt = \left[x(t) y(t) \right]_{-\infty}^{\infty} - \int x(t) \frac{dy}{dt} dt.$$

如果假设边界项 $\left[x(t) y(t) \right]_{-\infty}^{\infty} = 0$ (比如 $x(t), y(t)$ 在无穷远处衰减到 0, 或者它们满足合适的边界条件), 那么就得到:

$$\int_{-\infty}^{\infty} \frac{dx}{dt} y(t) dt = - \int_{-\infty}^{\infty} x(t) \frac{dy}{dt} dt.$$

这就是书里写的:

$$\int_{-\infty}^{\infty} \frac{dx}{dt} y(t) dt = \int_{-\infty}^{\infty} x(t) \left(-\frac{dy}{dt} \right) dt.$$

它说明了: 导数算子的伴随 (转置) 就是带一个负号的导数算子。

命题 13.1

微分的反对称也可以应用到中心差分矩阵:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad \text{转置成} \quad A^T = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = -A$$

前向差分矩阵的转置会变成反向差分矩阵, 相当于乘 -1 。在微分方程式中, 第二导数 (加速度) 是对称的, 第一导数 (正比于速度的阻尼) 是反对称的。

$A^T A$ 的含义 1. 代数性质

- 对称性: $(A^T A)^T = A^T A$;
- 半正定性:

$$x^T (A^T A) x = (Ax)^T (Ax) = \|Ax\|^2 \geq 0,$$

因此特征值均为非负。

2. 几何意义 $A: \mathbb{R}^n \rightarrow \mathbb{R}^m$, 而 $A^T A: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 。它的作用可以理解为: 先通过 A 把向量映射到输出空间, 再通过 A^T 拉回输入空间。因此 $A^T A$ 定义了一种新的 “长度度量”:

$$\|x\|_A^2 := x^T (A^T A) x = \|Ax\|^2.$$

3. 最小二乘中的作用正规方程

$$A^T A \hat{x} = A^T b$$

说明 $A^T A$ 扮演了 Gram 矩阵的角色, 它把 “残差最小化” 的几何条件转化为代数方程。

4. 数据分析中的意义 $A^T A$ 又称 Gram 矩阵或协方差矩阵 (标准化后)。它记录了列向量之间的内积关系:

$$(A^T A)_{ij} = a_i^T a_j,$$

并且是 PCA (主成分分析) 等方法的核心。

总结: $A^T A$ 既是对称半正定的 Gram 矩阵, 反映了列向量的相关性, 又是最小二乘与 PCA 的核心工具。

定理 13.3 (转置与内积的意义)

$x^T y$ 是一个数字, xy^T 是一个矩阵.

在量子力学中常写作 $\langle x|y \rangle$ (内积) 与 $|x\rangle\langle y|$ (外积)。

宇宙可能是被线性代数管理的, 下面三个例子对于内积有不同的意义:

来自力学 功 (work) = (位移)(力) $= x^T f$,

来自电路 热损失 = (压降)(电流) $= e^T y$,

来自经济 收入 = (数量)(价格) $= q^T p$.

**定义 13.2 (列空间、行空间与零空间)**

还要再看设 $A \in \mathbb{R}^{m \times n}$ 。

- 列空间 (Column Space)

$$\mathcal{C}(A) = \{ Ax : x \in \mathbb{R}^n \} \subseteq \mathbb{R}^m.$$

它由 A 的列向量张成, 表示所有可能的输出 Ax 。几何意义: A 把 $\mathbb{R}^n$ 中的向量映射到 $\mathbb{R}^m$, 结果必定落在列空间中。

- 行空间 (Row Space)

$$\mathcal{C}(A^T) = \{ A^T y : y \in \mathbb{R}^m \} \subseteq \mathbb{R}^n.$$

它由 A 的行向量张成, 等价于 A^T 的列空间。几何意义: 行空间包含了 A 对输入向量 x 所进行的“线性约束”的所有可能组合。也就是 $Ax=b$ 里面每一行 x 的系数限制 x 之间的关系。

- 零空间 (Null Space)

$$\mathcal{N}(A) = \{ x \in \mathbb{R}^n : Ax = 0 \}.$$

它是所有被 A 映射到零向量的输入向量的集合。几何意义: 零空间表示了“失效方向”——这些方向上的输入完全被 A 消去。

**命题 13.2 (三者对比)**

- 列空间 $\mathcal{C}(A) \subseteq \mathbb{R}^m$, 行空间 $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$, 零空间 $\mathcal{N}(A) \subseteq \mathbb{R}^n$ 。
- 维数关系:

$$\dim \mathcal{C}(A) = \dim \mathcal{C}(A^T) = \text{rank}(A).$$

设 $A \in \mathbb{R}^{m \times n}$, 那么

$$\dim \mathcal{C}(A) + \dim \mathcal{N}(A) = n.$$

- 正交关系: 零空间 $\mathcal{N}(A)$ 与行空间 $\mathcal{C}(A^T)$ 正交; 零空间 $\mathcal{N}(A^T)$ 与列空间 $\mathcal{C}(A)$ 正交。



13.2 最小二乘法

$Ax = b$ 经常是无解, 一般的理由是方程式太多。矩阵 A 的行比列多, 方程式的个数大于未知数的个数 ($m > n$)。这 n 个列只生成了 m 维空间的一小部分, 除非所有的量测值都很完美, 否则 b 会落在 A 的列空间之外。消元法得到不可能存在的方程式然后停止, 但我们不能停止, 因为量测值必然包含杂讯。

重复: 我们不可能一直把误差 $e = b - Ax$ 降为零; 当 $e = 0$ 时, x 是 $Ax = b$ 的确切解。当 e 的长度尽可能地小, $\hat{x}$ 就是“最小二乘解” (least squares solution)。本段的目标是计算并使用 $\hat{x}$, 许多实际问题都需要它。

前一段强调投影 p , 本段强调最小二乘解 $\hat{x}$; 二者通过 $p = A\hat{x}$ 相关联。基本方程式仍然是正规方程: $A^T Ax = A^T b$ 。下面给出一个非正式的得到“正规方程式”的办法:

 **笔记** 当 $Ax = b$ 无解时, 左乘 A^T 然后求解

$$A^T Ax = A^T b.$$

我们希望用一条直线

$$y \approx ax + b$$

来拟合一组点。将参数写成向量

$$\theta = \begin{bmatrix} a \\ b \end{bmatrix},$$

并将数据点 (x_i, y_i) 写成矩阵形式:

$$A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

于是所有观测点可以统一写作

$$A\theta \approx y,$$

即

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax_1 + b \\ ax_2 + b \\ \vdots \\ ax_m + b \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

由于一般情况下方程组无解, 我们改为最小二乘问题:

$$\hat{\theta} = \arg \min_{\theta} \|A\theta - y\|^2,$$

其解为正规方程

$$\hat{\theta} = (A^T A)^{-1} A^T y.$$

东邪的 tip 13.3 (数值分析课上关于最小二乘法的原理)

这是经典的最小二乘误差函数:

$$E(a_0, a_1) = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2.$$

用导数方法求最小值 (原理关键!)

我们令误差函数对 a_0, a_1 的偏导数为 0, 找极小值点:

$$\frac{\partial E}{\partial a_0} = 0, \quad \frac{\partial E}{\partial a_1} = 0.$$

这是一个标准的“求极值”步骤。展开以后可以得到两个线性方程:

$$\begin{cases} na_0 + a_1 \sum x_i = \sum y_i, \\ a_0 \sum x_i + a_1 \sum x_i^2 = \sum x_i y_i. \end{cases}$$

直线拟合问题写成矩阵形式：

$$A\theta \approx y, \quad A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix}, \quad \theta = \begin{bmatrix} a \\ b \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

因为 $m > 2$ 一般无解，改为最小二乘，得到正规方程

$$A^T A \theta = A^T y,$$

即

$$\begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & m \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix}.$$

于是得到两条方程：

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i, \\ a \sum x_i + b m = \sum y_i, \end{cases}$$

其解 (a, b) 正是使残差平方和最小的拟合直线参数。

命题 13.3 (从矩阵理解和从导数理解的关系)

我们要最小化的目标函数是

$$E(\theta) = \|A\theta - y\|^2 = (A\theta - y)^T (A\theta - y),$$

这是最小二乘的定义。

2. 求偏导 = 0

对 θ 求梯度：

$$\nabla_{\theta} E = 2A^T (A\theta - y).$$

设导数 = 0，则

$$A^T (A\theta - y) = 0,$$

这就得到正规方程

$$A^T A \theta = A^T y.$$

因此，“偏导数 = 0”推出的条件，和“左乘 A^T 保证残差正交”其实是同一个式子。

命题 13.4 (如何从投影理解优化目标)

矩阵 A 的所有列向量可以生成一个列空间，向量 Ax 一定在这个列空间内，但向量 b 可能不在列空间内。 Ax 与 b 的差就是一个新的向量 r ，称为残差，可以用平行四边形法则理解。为了使 r 的长度最短，必须要求它垂直于由 A 的列向量生成的空间。如果满足这个条件，就有

$$A^T r = 0,$$

此时残差的长度也达到最小。几何上可以想象为：空间外一点到平面的最短距离就是垂直距离，而把 b 投影到由 A 的列向量张成的平面上后，就存在相应的 x ，使得 Ax 恰好等于这个投影向量。

东邪的 tip 13.4

从几何，代数和微积分都能理解这个方程的目的。没多一种维度你对问题的理解就更加深刻。

东邪的 tip 13.5

矩阵乘法意义

一句话：谁在左边，谁就“主导”最后结果落在哪个空间；右边的对象只决定“怎么组合”，因此，当我们同时左乘 A^T 时，等式的两边都被映射到了 A^T 所作用的空间中。选择左乘 A^T 不是随意的，而是因为它唯一能把几何条件 $r \perp C(A)$ （残差垂直列空间）翻译成代数形式。

列空间与行空间

$$(Ax) \cdot y = x \cdot (A^T y)$$

A 生成的空间是“输出空间里的列空间”——列向量为基生成的空间， A^T 生成的空间是“输入空间里的行空间”——行向量生成空间。它们的维数相等，但位置不同，通过点积保持联系。所以，输入空间就是 $\mathbb{R}^n$ （因为你只能送 n 维向量进去）也就是乘的 x 必须是一个 n 维列向量，输出空间就是 $\mathbb{R}^m$ （因为出来的永远是 m 维）。

13.3 QR 分解操作

QR分解的核心目的是将矩阵 A 分解为

$$A = QR,$$

其中 Q 是列向量正交归一的矩阵， R 是上三角矩阵。换句话说，QR 分解就是把原来可能不正交的基向量转化为一组正交单位向量，并将变化过程中的比例关系记录在 R 中。和前面的矩阵乘法概念类似，能清晰的看出 R 对原本的正交基有怎么样的拉伸和旋转。

命题 13.5

当 Q 是方阵且 $m = n$ 时，子空间就是整个空间。此时有

$$Q^T = Q^{-1},$$

并且

$$\hat{x} = Q^T b \quad \text{与} \quad x = Q^{-1} b$$

完全相同，因此这个解是确切的。

此外，向量 b 在整个空间上的投影就是它自身：

$$p = b,$$

并且投影矩阵满足

$$P = Q^T Q = I.$$

这样我们从原来的两边同乘 A^T 变成两边同乘 Q^T 乘 $A^T A$ 的方法会放大条件数（奇异值变成平方），误差更大。• 用 Q^T 投影不会改变长度或角度（正交变换保持数值稳定）。 R 作为上三角矩阵天然好解方程。

命题 13.6 (正交矩阵与傅立叶变换)

当 $p = b$ ，我们的公式把 b 从它的一维投影中聚集起来。若 $q_1, \dots, q_n$ 是整个空间的一组正交单位基底，则 Q 是方形，每个 $b = QQ^T b$ 是它沿着 q 's 的分量的总和：

$$b = q_1(q_1^T b) + \dots + q_n(q_n^T b).$$

转换 $QQ^T = I$ 是傅里叶级数的基础，也是所有应用数学中伟大“转换”的基础。他们把 b 或函数 $f(x)$ 打碎成垂直的片段，借由上式中的片段，逆转换把 b 与 $f(x)$ 再次一起恢复。

东邪的 tip 13.6 (施密特正交化)

看见转置要想到点积，看见点积要想到投影。如何把三个向量 a, b, c 正交化呢？方法就是：把每一个向量在其他向量方向上的分量减掉。

格莱姆-施密特的第一步：

$$B = b - \frac{A^T b}{A^T A} A, \quad (7)$$

其中投影系数为

$$\frac{\langle b, A \rangle}{\langle A, A \rangle} = \frac{A^T b}{A^T A}.$$

两边左乘 A 的转置即可验证 A 与 B 的正交。 c 同理去掉 a 和 B 的分量产生 C 。最后 a, B, C 再除以自己的模长即可。一句话总结

Gram-Schmidt 正交化 = 把 A 的列向量逐个「扣掉已有正交方向的分量」。这个扣除的系数恰好填入 R 的上三角位置，剩下的单位化结果填入 Q 的列。 $A = QR$ 不是额外的结论，而是 Gram-Schmidt 过程的直接重写。

命题 13.7 (格莱姆-施密特的关键步骤)

第一步是

$$q_1 = \frac{a}{\|a\|},$$

(其他向量不参与)。第二步是方程式 (7)，其中 b 是 A 与 B 的组合，在这一步骤 c 与 q_3 没有参与。下一个向量不参与正是格莱姆-施密特的关键点：这就是为什么下三角可以直接写 0。

- 向量 a 与 A 与 q_1 沿着同一条直线；
- 向量 a, b 与 A, B 与 q_1, q_2 在同一个平面；
- 向量 a, b, c 与 A, B, C 与 q_1, q_2, q_3 在同一个子空间（维度 3）。

在每一步， $a_1, \dots, a_k$ 是 $q_1, \dots, q_k$ 的组合，后面的 q 不参与。关联矩阵 R 是三角形，我们有

$$A = QR.$$

$$\begin{bmatrix} a & b & c \end{bmatrix} = \begin{bmatrix} q_1 & q_2 & q_3 \end{bmatrix} \begin{bmatrix} q_1^T a & q_1^T b & q_1^T c \\ 0 & q_2^T b & q_2^T c \\ 0 & 0 & q_3^T c \end{bmatrix}, \quad \text{或简记为 } A = QR. \quad (9)$$

 **笔记** 设 A 的列向量是 a_1, a_2, a_3 ，做 Gram-Schmidt 正交化：

$$q_1 = \frac{a_1}{\|a_1\|}, \quad q_2 = \frac{a_2 - (a_2 \cdot q_1)q_1}{\|\dots\|}, \quad q_3 = \frac{a_3 - (a_3 \cdot q_1)q_1 - (a_3 \cdot q_2)q_2}{\|\dots\|}$$

反过来用 q_i 表示原列向量：

$$a_1 = \|a_1\|q_1, \quad a_2 = (a_2 \cdot q_1)q_1 + \|\dots\|q_2, \quad a_3 = (a_3 \cdot q_1)q_1 + (a_3 \cdot q_2)q_2 + \|\dots\|q_3$$

写成矩阵形式即 $A = QR$ ：

$$\begin{bmatrix} | & | & | \\ a_1 & a_2 & a_3 \\ | & | & | \end{bmatrix} = \begin{bmatrix} | & | & | \\ q_1 & q_2 & q_3 \\ | & | & | \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

R 为何上三角？因为 a_k 正交化时只涉及前 k 个基向量，第 k 列仅 $r_{1k}, \dots, r_{kk}$ 非零，天然产生上三角结构。

定理 13.4 (格莱姆-施密特正交化)

从无关向量 $a_1, \dots, a_n$ 出发, 格莱姆-施密特方法能够构造出正交单位向量 $q_1, \dots, q_n$ 。这些列向量构成的矩阵 Q 满足

$$A = QR,$$

其中

$$R = Q^T A$$

是一个上三角矩阵, 因为后面的 q 与较早的 a 的内积为零 (正交性)。



$$\text{最小二乘: } R^T R \hat{x} = R^T Q^T b \quad \text{或} \quad R \hat{x} = Q^T b \quad \text{或} \quad \hat{x} = R^{-1} Q^T b \quad (10)$$

13.4 QR 分解迭代法-数值分析

基本的步骤是分解 A 成为 QR , 其中 A 的特征值是我们想要的。回忆格莱姆-施密特 (段落 4.4), Q 有正交单位列与 R 是三角形。特征值的关键概念是: 反转 Q 与 R 。新的矩阵 (相同 λ ' s) 是 $A_1 = RQ$, 因为 $A = QR$ 与 $A_1 = Q^{-1} A Q$ 相似, 所以 RQ 的特征值没有改变:

$$A_1 = RQ \text{ 有相同的特征值 } \lambda, \quad QRx = \lambda x \Rightarrow RQ(Q^{-1}x) = \lambda(Q^{-1}x). \quad (12)$$

继续这个程序。把新矩阵 A_1 分解成 $Q_1 R_1$, 然后反转因子变成 $R_1 Q_1$, 这是相似矩阵 A_2 而且再次没有改变特征值。惊人的是这些特征值开始出现在对角线, 很快的 A_4 的最后单元得到一个精确的特征值。这个情况下, 我们移除最后一行与最后一列得到一个较小的矩阵, 然后继续寻找下一个特征值。

这个现象的原因是

$$A_k = (Q_1 Q_2 \cdots Q_k)^T A_0 (Q_1 Q_2 \cdots Q_k).$$

记累计的正交变换

$$Q^{(k)} := Q_1 Q_2 \cdots Q_{k-1} \quad (\text{正交}),$$

于是

$$A_k = (Q^{(k)})^T A_0 Q^{(k)}.$$

这说明: 幂迭代是找的一个特征向量, QR 迭代是找一组特征向量做成的正交基

命题 13.8

两个额外的概念使得这个方法成功, 一个是在分解成 QR 之前, 把矩阵平移 I 的倍数, 然后 RQ 再平移回去得到 A_{k+1} :

$$A_k - c_k I = Q_k R_k, \quad A_{k+1} = R_k Q_k + c_k I.$$

于是 A_{k+1} 与 A_k 有相同的特征值, 而且与原始的 $A_0 = A$ 有相同的特征值。一个优质的平移是选择 c 靠近一个 (未知的) 特征值。这个更精确的特征值出现在 A_{k+1} 的对角线上——这告诉我们下一个到 A_{k+2} 的步骤应该选择的较好 c 。

—

第二个概念是在 QR 方法开始之前先获得非对角线的零, 一个消元步骤 E 会做这样的事情, 或者是一个吉文斯旋转 (Givens rotation)。但是不要忘了 E^{-1} (否则 λ 会改变):

$$EAE^{-1} = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & 4 \\ -1 & 1 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 7 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 5 & 3 \\ 1 & 9 & 5 \\ 0 & 4 & 2 \end{bmatrix} \quad (\text{相同的 } \lambda\text{'s}).$$

我们必须在次对角线留下非零数 1 与 4，更多的 E 可以移除它们，但是 E^{-1} 会再次填满。这是一个海森堡矩阵（一个非零的次对角线）。

在 QR 方法过程中，左下角的零仍然维持是零。因此，每个 QR 分解的运算次数可以从 $O(n^3)$ 降到 $O(n^2)$ 。

命题 13.9 (幂迭代与逆迭代)

最大特征值对应的向量会占据主导地位因为同时提出 λ_1^k ，除了最大的一项，其他的 λ_n/λ_1 比值都小于 1， k 趋紧无穷的时候他们都趋紧于 0。

从任意向量 u_0 开始。用 A 乘它得到 u_1 。再用 A 乘，得到 u_2 。如果 u_0 是特征向量的线性组合，那么 A 乘以每个特征向量 x_i 时会放大 λ_i 倍。经过 k 步之后，我们得到 $(\lambda_i)^k$ ：

$$u_k = A^k u_0 = c_1(\lambda_1)^k x_1 + \cdots + c_n(\lambda_n)^k x_n. \quad (10)$$

随着幂迭代的进行，最大的特征值会逐渐占据主导地位。向量 u_k 会逐渐指向那个主特征向量 x_1 。我们在马尔可夫矩阵中看到过这一点：

$$A = \begin{bmatrix} 0.9 & 0.3 \\ 0.1 & 0.7 \end{bmatrix} \quad \text{其最大特征值 } \lambda_{\max} = 1, \text{ 对应的特征向量为 } \begin{bmatrix} 0.75 \\ 0.25 \end{bmatrix}.$$

从 u_0 开始，并在每一步都用 A 相乘：

$$u_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad u_1 = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} 0.84 \\ 0.16 \end{bmatrix},$$

逐渐趋向于

$$u_{\infty} = \begin{bmatrix} 0.75 \\ 0.25 \end{bmatrix}.$$

收敛速度取决于第二大特征值 λ_2 与最大特征值 λ_1 的比值。我们并不希望 λ_1 太小，而是希望 λ_2/λ_1 较小。在这个例子中， $\lambda_2 = 0.6$ ， $\lambda_1 = 1$ ，因此收敛速度很好。

对于大矩阵，往往会发生 $|\lambda_2/\lambda_1|$ 很接近 1 的情况，这时幂迭代就太慢了。

有没有办法找到最小的特征值（这往往在应用中最重要）呢？答案是肯定的：用反幂迭代（inverse power method）：用 A^{-1} 而不是 A 去乘 u_0 。由于我们从不真的去计算 A^{-1} ，实际上我们解的是 $Au_1 = u_0$ 。通过保存 LU 分解，下一步 $Au_2 = u_1$ 也可以快速完成。在第 k 步时，有：

$$Au_k = u_{k-1}. \\ u_k = A^{-k} u_0 = \frac{c_1 x_1}{(\lambda_1)^k} + \cdots + \frac{c_n x_n}{(\lambda_n)^k}. \quad (11)$$

现在，最小的特征值 $\lambda_{\min}$ 起主导作用。当它非常小时，因子 $1/\lambda_{\min}^k$ 就会很大。为了加快速度，我们通过把矩阵平移为 $A - \lambda^* I$ 来使 $\lambda_{\min}$ 变得更小。

这种平移不会改变特征向量。 $(\lambda^*$ 可以取自 A 的对角元, 更好的选择是 Rayleigh 商

$$\frac{x^T Ax}{x^T x}.$$

)

如果 λ^* 接近 $\lambda_{\min}$, 那么 $(A - \lambda^* I)^{-1}$ 就有一个非常大的特征值 $(\lambda_{\min} - \lambda^*)^{-1}$ 。

每一步平移反幂迭代都会把该特征向量乘以这个大数, 从而使该特征向量迅速占据主导地位。

第 14 章 LLL 格基

定义 14.1 (格与格基)

设 $b_1, b_2, \dots, b_n \in \mathbb{R}^m$ 是 n 个线性无关向量。由它们生成的格定义为

$$L(b_1, b_2, \dots, b_n) = \left\{ z_1 b_1 + z_2 b_2 + \dots + z_n b_n \mid z_i \in \mathbb{Z} \right\}.$$

称 $B = (b_1, b_2, \dots, b_n)$ 为格 $L(B)$ 的格基。注意，一个格可以有多种不同的基。



注 与普通的向量空间基不同，格基规定系数必须是整数：

$$L(B) = \{ z_1 b_1 + \dots + z_n b_n \mid z_i \in \mathbb{Z} \}.$$

因此，格是离散点集，而不是连续的空间。

命题 14.1 (LLL 算法的核心思想)

LLL 算法不能追求完全正交（那样就不再是格基），只能在“整数保持”和“尽量正交”之间折中：

- 保证：所有变换都是整系数可逆矩阵（unimodular），因此格 $\mathcal{L}(B)$ 保持不变；
- 目标：基向量不要太“偏斜”，做到“近似正交”，同时“尽量短”。



输入与目标

设格基为

$$B = (b_1, b_2, \dots, b_n) \in \mathbb{Z}^{m \times n}.$$

目标：通过基变换得到“更短、更接近正交”的基。其中每一个向量里面全是整数，为最后的势函数 $\Phi(B)$ 提供约束。

$$B' = (b'_1, b'_2, \dots, b'_n),$$

满足一定的正规化条件（LLL-reduced）。

Gram-Schmidt 正交化

记 Gram-Schmidt 结果：

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{j,i} b_j^*, \quad \mu_{j,i} = \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}.$$

于是 b_i^* 是 b_i 在 $\text{span}\{b_1, \dots, b_{i-1}\}$ 的正交补上的投影。

东邪的 tip 14.1

$\mu_{j,i}$ 是 b_i 中含有 $\mu_{j,i}$ 倍的 b_j ，如果这个系数很大，说明 b_i 在 b_j^* 方向上“粘连”得厉害 $\rightarrow$ 基向量会偏斜。

步骤一：规模约化 (Size Reduction)

$$b_i \leftarrow b_i - \lfloor \mu_{j,i} \rfloor b_j, \quad \text{for } j < i,$$

其中 $\lfloor \cdot \rfloor$ 表示四舍五入到最近的整数。不断减去整数倍的前面向量，避免“过度粘连”，并且保证点仍在格内。让系数 $\mu_{j,i}$ 控制在

$$|\mu_{j,i}| \leq \frac{1}{2}.$$

这保证了基向量不会“偏斜”，得到更短正交。

定义 14.2 (Unimodular 矩阵)

一个整数矩阵 $U \in \mathbb{Z}^{n \times n}$ 若满足

$$\det(U) = \pm 1,$$

则称为 *unimodular* 矩阵。



命题 14.2 (Unimodular 矩阵及其作用)

若 $U \in \mathbb{Z}^{n \times n}$ 且 $\det(U) = \pm 1$ ，则称 U 为 *unimodular* 矩阵。它有以下性质：

1. 保持格不变. 若 B 是格基，则

$$\mathcal{L}(B) = \{Bz : z \in \mathbb{Z}^n\}, \quad B' = BU,$$

则有

$$\mathcal{L}(B') = \mathcal{L}(B).$$

因为 U 可逆且整系数，保证 $\mathbb{Z}^n$ 在变换下仍映射回 $\mathbb{Z}^n$ 。

2. 保持体积. 平行多面体体积满足

$$\det(BU) = \det(B) \det(U) = \pm \det(B),$$

因此 $|\det(B)|$ 保持不变。

3. LLL 算法中的意义. 在 LLL 算法中，所有基向量的调整都可表示为右乘 unimodular 矩阵：

- 规模约化 (Size Reduction)：

$$b_i \leftarrow b_i - \lfloor \mu_{i,j} \rfloor b_j, \quad j < i,$$

对应于右乘一个整系数初等矩阵，保证 $\mu_{i,j}$ 被控制在 $[-\frac{1}{2}, \frac{1}{2}]$ 。

- 交换步骤 (Swap)：

$$b_i \leftrightarrow b_{i-1},$$

对应于交换两列，也就是 unimodular 矩阵。

这些操作保证始终在同一个格中进行换基，而不会改变格的本质。

因此，unimodular 矩阵就是“合法的换基操作”：它能改变基向量的形状，但不会改变格本身，也不会改变体积。



步骤二：Lovász 条件与交换

在规模约化后，检查相邻两列是否满足 Lovász 条件：

$$\|b_i^*\|^2 \geq (\delta - \mu_{i-1,i}^2) \|b_{i-1}^*\|^2, \quad \delta \in \left(\frac{1}{4}, 1\right),$$

通常取 $\delta = \frac{3}{4}$ 。

- Lovasz 条件大致保证当前工作的向量足够大成为我们的临时基向量
在 Gram-Schmidt 正交化中，一个基向量 b_i 会被投影到前面的正交补 b_i^* 上。
 - 如果 b_i^* 太短，说明 b_i 在方向上几乎被前面基向量“解释掉了”；
 - 那么整个基就很“塌”，缺乏独立性，导致表示格点的效率很差。
- 于是 LLL 算法要求：

$\|b_i^*\|^2$ 不能太小，相对前一个正交向量 b_{i-1}^* 而言要有下界。

- 若条件成立：继续处理 $i+1$ 列；
- 若条件不成立：交换 b_{i-1} 与 b_i ，并令 $i \leftarrow \max(i-1, 2)$ ，然后重新规模约化。

命题 14.3 (LLL 算法的交换思想)

在施密特正交化中，按顺序处理基向量：前面的基先正交化，后面的基不断减去前面的投影。LLL 的交换条件 (Lovász 条件) 就是在检查：

- 前面的基向量是否“太差劲” (太长或太偏斜)，
- 如果条件不满足，就交换 b_k 和 b_{k+1} 。

这样做的结果是：

- 较短、方向更合适的向量被提前，作为参照基；
- 较差的向量被放到后面，被迫减去“好的”向量的分量。

Lovász 条件：

$$\delta \|b_k^*\|^2 \leq \|b_{k+1}^* + \mu_{k+1,k} b_k^*\|^2,$$

其中 b_i^* 是施密特正交向量， $\delta \in (1/4, 1)$ (通常取 $\delta = 3/4$)。

- 若条件成立，说明 b_k “够好”，无需交换；
- 若条件不成立，说明 b_k 太差，就与 b_{k+1} 交换。

类比：可以把这一过程看作“选队长”：

- 好的队员 (短而正交性好) 先当队长；
- 差的队员只能排在后面，被迫去配合前面更强的队员。

这样，整个队伍 (格基) 就会更整齐、更短、更接近正交。

命题 14.4 (LLL 与冒泡排序的类比)

在冒泡排序中，较大的元素 (若从大到小排序) 会一步步“冒”到前面：

- 每次比较相邻两个，如果顺序错误，就交换。
- 经过多轮，最终最“大”的元素会被送到最前面。

在 LLL 算法中，同样存在类似的“冒泡”现象：

- LLL 希望“更短、更正交”的向量排在前面。
- 每次检查 Lovász 条件

$$\delta \|b_k^*\|^2 \leq \|b_{k+1}^* + \mu_{k+1,k} b_k^*\|^2,$$

若不满足，就交换 $b_k \leftrightarrow b_{k+1}$ 。

- 交换后，那个“更好”的向量 (更短、更正交的) 会向前移动一步，就像冒泡排序里较大的元素逐步向前冒。
- 因此，经过不断交换，“最好的基向量”会逐渐被推到前面。

东邪的 tip 14.2 (类比：LLL 的 Lovász 条件 vs ODE 的 Lipschitz 条件)

LLL 格基约化	常微分方程 (ODE)
Lovász 条件：保证相邻正交向量不能“太塌”	Lipschitz 条件：保证函数变化不能“太陡”
$\delta \ b_k^*\ ^2 \leq \ b_{k+1}^* + \mu_{k+1,k} b_k^*\ ^2, \quad \delta \in (1/4, 1)$	$ f(x, y_1) - f(x, y_2) \leq L y_1 - y_2 $
避免基向量几乎线性相关，保证格的稳定性	避免解不唯一，保证 ODE 解的稳定性

步骤三：终止性与复杂度

定义一个势函数

$$\Phi(B) = \prod_{i=1}^n \|b_i^*\|^2,$$

这个势函数就所有 b_i 模长的平方的乘积。它在每次交换后都会严格减小一个有界的整数量，因此算法在有限步后必然终止。

复杂度结果：LLL 在输入长度的多项式时间内完成（这是它在密码学应用中的关键优势）。

命题 14.5 (LLL 势函数与格体积)

设 $B = [b_1, \dots, b_n] \in \mathbb{R}^{m \times n}$ ，Gram-Schmidt 正交化得 $b_1^*, \dots, b_n^*$ 。则

$$\det(B^T B) = \prod_{i=1}^n \|b_i^*\|^2.$$

因此定义的势函数

$$\Phi(B) := \prod_{i=1}^n \|b_i^*\|^2$$

正是格体积的平方。

由于 LLL 算法中的变换均为 unimodular 矩阵作用（保持体积不变）， $\Phi(B)$ 始终与格的几何结构对应，并在交换或约化时严格下降，保证算法收敛。



命题 14.6 (有限步终止性的直观理解)

在整格 $\mathcal{L}(B) \subseteq \mathbb{Z}^m$ 中，任意一组基 B 生成的平行多面体

$$P(B) = \left\{ \sum_{i=1}^n \alpha_i b_i : 0 \leq \alpha_i < 1 \right\}$$

的体积必为整数，因为它正好对应 $\mathbb{R}^m$ 中由单位立方格子堆叠出的“积木”个数。因此

$$\det(\mathcal{L}(B)) = |\det(B)| \in \mathbb{Z}_{>0}.$$

Gram-Schmidt 分解给出

$$\prod_{i=1}^n \|b_i^*\| = |\det(B)|,$$

于是势函数

$$\Phi(B) = \prod_{i=1}^n \|b_i^*\|^2 \in \mathbb{Z}_{>0}.$$

换句话说，每一个格基形成的平行多面体体积天然为整数，势函数就是它的“平方版”。每次交换都会严格减小 $\Phi(B)$ ，而 $\Phi(B)$ 只有有限多个整数值，因此 LLL 算法必在有限步后终止。



东邪的 tip 14.3

离散的好处就是有限，至少相对连续来说好很多

结果性质

输出基 $(b'_1, \dots, b'_n)$ 满足：

- 规模约化： $|\mu_{j,i}| \leq \frac{1}{2}, \forall j < i$;
- Lovász 条件： $\|b_i^*\|^2 \geq (\delta - \mu_{i-1,i}^2) \|b_{i-1}^*\|^2$;
- 基向量较短且接近正交。

14.1 LLL 最短向量的意义

用 LLL 从数值 α 识别整系数多项式：以 $\alpha \approx \sqrt{2}$ 为例

目标已知一个实数的近似值 α (例如 $\alpha = 1.414213$), 希望找到一个整系数多项式

$$f(x) = c_0 + c_1x + \cdots + c_dx^d \in \mathbb{Z}[x]$$

使得 $|f(\alpha)|$ 很小 (理想情况是 $f(\alpha) = 0$, 即找出 α 的最小多项式)。

格的构造 (How to build a lattice) 取一组尺度 $k \in \mathbb{N}$ (比如 $k = 6$), 并选择多项式次数上界 d 。构造如下 $(d+1) \times (d+1)$ 矩阵 B 的列向量作为格基:

$$B = \begin{bmatrix} 1 & & & 0 \\ & 1 & & 0 \\ & & \ddots & 0 \\ 0 & 0 & \cdots & 1 \\ 10^k & [10^k\alpha] & [10^k\alpha^2] & \cdots \end{bmatrix} \quad (\text{上方是 } I_{d+1}, \text{ 最后一行放 } 10^k\alpha^i \text{ 的取整——为了保证是整数 } k \text{ 的大小决定了精度}).$$

东邪的 tip 14.4

其中, 对角线上的元素依次代表 x^0 到 x^{d-1} 。它们的线性组合所形成的每一列, 都对应于一组 $(1, x, x^2, \dots, x^{d-1})$ 的系数。最后一行则是为了将这组线性组合的 x 直接代入 α , 使结果能够清晰直观地显示出来。我们的目的是让这组线性组合经可能的小最好是 0。这里还有一个放大的好处由于最后一行远大于前面的行所以最后一行对向量的大小起决定性作用, 最短的向量的最后一行必然是所有最后一行元素最小的。

* 用 LLL 找“短向量” $\Rightarrow$ 找多项式系数对基 B 运行 LLL 算法, 得到一个“较短且近似正交”的等价基。LLL 输出基中的最短向量或若干很短的向量, 其整数系数 $z = (c_0, \dots, c_d)$ 往往满足 $|f(\alpha)| = |\sum c_i\alpha^i|$ 很小。于是我们就得到了一个候选多项式

$$f(x) = c_0 + c_1x + \cdots + c_dx^d \in \mathbb{Z}[x], \quad \text{且 } |f(\alpha)| \text{ 很小.}$$

若 $f(\alpha) \approx 0$, 可尝试化简 (去公因数) 并检验是否确为最小多项式。

具体例子: $\alpha \approx \sqrt{2}, d = 2, k = 6$ 取

$$\alpha = 1.414213, \quad d = 2, \quad k = 6.$$

构造

$$B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 10^6 & [10^6\alpha] & [10^6\alpha^2] \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1000000 & 1414213 & 2000000 \end{bmatrix} \quad (\text{因为 } \alpha^2 \approx 2, [10^6\alpha^2] \approx 2000000).$$

对该 B 做 LLL 约化, 得到的一个最短向量 (或很短向量) 可能是

$$(-2, 0, 1)^\top.$$

它对应的多项式为

$$f(x) = (-2) + 0 \cdot x + 1 \cdot x^2 = x^2 - 2,$$

并且

$$f(\alpha) = \alpha^2 - 2 \approx 0.$$

这正是 $\sqrt{2}$ 的最小多项式。

总结

- 将“寻找整系数多项式 f 使 $f(\alpha)$ 小”转化为“寻找格中的短向量”;
- 通过构造含 I_{d+1} 与 $[10^k\alpha^i]$ 的基矩阵 B , 使每个整数系数向量 z 对应一个多项式;
- 用 LLL 在多项式时间内找到短向量, 其系数即为候选多项式;

- 在 $\alpha \approx \sqrt{2}$ 的例子中, LLL 返回 $(-2, 0, 1)$, 对应 $x^2 - 2$ 。

如果生成的不是最短多项式就是小数点后面精度不够。最短多项式就是, 项数最少的多项式, LLL 查找不同数字间的整数关系。整数本身就是一种简化。

例题 14.1 Merkle–Hellman 背包加密示例

1. 密钥生成

选择一个超递增序列:

$$w = (2, 3, 7, 14, 30).$$

取模数 $q = 61$ (大于 $\sum w_i = 56$), 并选取 $r = 17$ 与 q 互素。

计算公钥:

$$b_i \equiv r \cdot w_i \pmod{q},$$

得到

$$b = (34, 51, 58, 54, 22).$$

因此: - 私钥为 (w, q, r) ; - 公钥为 $b = (34, 51, 58, 54, 22)$ 。

明文消息为比特串 $(1, 0, 1, 0, 1)$ 。

计算密文:

$$c = \sum_{i=1}^5 m_i b_i = 1 \cdot 34 + 0 \cdot 51 + 1 \cdot 58 + 0 \cdot 54 + 1 \cdot 22 = 114.$$

因此密文为:

$$c = 114.$$

3. 解密

首先计算 r 在模 q 下的逆元:

$$r^{-1} \equiv 18 \pmod{61}, \quad \text{因为 } 17 \cdot 18 \equiv 1 \pmod{61}.$$

然后:

$$c' \equiv c \cdot r^{-1} \pmod{q} = 114 \cdot 18 \equiv 37 \pmod{61}.$$

在超递增序列 $w = (2, 3, 7, 14, 30)$ 中解方程

$$\sum_{i=1}^5 m_i w_i = 37.$$

使用贪心法: $-30 \leq 37 \implies m_5 = 1$, 余数 7; $-14 > 7 \implies m_4 = 0$; $-7 = 7 \implies m_3 = 1$, 余数 0; - 余下均为 0。

因此恢复的明文为:

$$(m_1, m_2, m_3, m_4, m_5) = (1, 0, 1, 0, 1).$$

注 Merkle–Hellman 背包加密体系最初被认为是安全的, 因为从公钥 $b = (b_1, \dots, b_n)$ 和密文 $c = \sum m_i b_i$ 恢复比特向量 $(m_1, \dots, m_n)$ 等价于求解

$$\sum_{i=1}^n m_i b_i = c, \quad m_i \in \{0, 1\},$$

这正是著名的子集和问题 (subset sum problem), 在一般情况下是 NP-困难的。

然而, LLL 格基约化算法提供了一种有效的攻击途径: 通过构造一个 $(n+1)$ 维格, 其中包含向量

$$(a_1, \dots, a_n, c),$$

LLL 可以在多项式时间内找到短向量。这些短向量往往正好对应于满足上述子集和关系的比特向量 $(m_1, \dots, m_n)$ 。

因此, 原本依赖于子集和难解性的 Merkle–Hellman 背包体系, 在 LLL 算法出现之后不再安全, 很快被实际

攻破。这也是格密码学对现代密码体系影响的重要历史事件之一。

定义 14.3 (时间复杂度)

在计算机科学中，算法的时间复杂度是指：算法在最坏情况或平均情况下所需的基本运算次数，作为输入规模 n 的函数。换句话说，时间复杂度研究的是当 n 越来越大时，算法运行时间如何随 n 增长。

设算法 A 在输入规模为 n 时，所需的基本操作次数为 $T(n)$ 。如果存在常数 $c > 0$ 和 n_0 ，使得对所有 $n \geq n_0$ 都有

$$T(n) \leq c \cdot f(n),$$

则称该算法的时间复杂度是 $O(f(n))$ 。其中 $f(n)$ 是一个常见的函数，例如 $n, n^2, n \log n$ 等。



14.2 LLL 的时间复杂度

定理 14.1 (LLL 算法的时间复杂度 (经典结论))

设输入是 n 维整数格的基 $B \in \mathbb{Z}^{n \times n}$ ，其元素绝对值不超过 B 。对固定的 $\delta \in (1/4, 1)$ ，标准 LLL 算法在位操作意义下的时间复杂度满足

$$T(n, B, \delta) = \tilde{O}(n^4 \cdot \log B),$$

更精细的经典上界常写作

$$O(n^4 \cdot \log^3 B),$$

其中 $\tilde{O}$ 省略了多项式对数因子。实际实现常优于理论上界。



GSO 就是施密特正交化

步骤	动作	单次代价	备注
1	初始化 GSO (计算 $\mu_{i,j}, \ b_j^*\ ^2$)	$O(n^3)$	一次性操作
2	尺寸约化 (对 $j < i$: 算 $\mu_{i,j}$, 更新 b_i)	$O(n^2)$	内积 $O(n)$ + 向量更新 $O(n)$, 共 $i-1 \leq n$ 次
3	Lovász 条件检查	$O(1)$	只用已存的 $\mu, \ b^*\ ^2$
4	交换并更新 GSO (若条件不满足)	$O(n^2)$	增量更新避免 $O(n^3)$
循环次数	交换步数 $\leq O(n^2 \log B)$	—	势函数下降法保证
总复杂度	$O(n^4 \log^3 B)$ (其中 $\log^3 B$ 来自大整数算术位长)		

表 14.1: LLL 算法每步操作及时间复杂度

定理 14.2 (LLL 算法的复杂度上界)

设输入基为 $B \in \mathbb{Z}^{n \times n}$ ，其元素的绝对值不超过 B 。则 Lenstra–Lenstra–Lovász (1982) 的分析表明：

- 所有中间数的位长增长是多项式有界，不会发生指数爆炸；
- 在保守估计下，每次算术操作的位复杂度需要额外考虑 $(\log B)^2$ 或 $(\log B)^3$ 。

因此，LLL 算法的总时间复杂度满足

$$T(n, B) = O(n^4 \cdot \log^3 B).$$



命题 14.7 (对 $\log^3 B$ 的拆解理解)

复杂度中的 $\log^3 B$ 可以解释为以下三个因素的叠加：

1. 第一个 $\log B$ ：来自输入规模，即最大数 B 的比特长度；
2. 第二个 $\log B$ ：来自大整数算术运算（乘法、除法）的代价；
3. 第三个 $\log B$ ：来自迭代过程中中间数的膨胀，虽数值变大但仍在多项式范围内。

因此最终得到 $\log^3 B$ 。



第 15 章 Attention Is All You Need

15.1 Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

15.2 Introduction

Recurrent neural networks, long short-term memory [hochreiter1997long] and gated recurrent [cho2014learning] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [sutskever2014sequence, cho2014learning, bahdanau2014neural]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [wu2016google, luong2015effective, jozefowicz2016exploring].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t , as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [kuchaiev2017factorization] and conditional computation [shazeer2017outrageously], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [bahdanau2014neural, structuredattentionnetworks]. In all but a few cases [decomposableattentionmodel], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

15.3 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [extendedneuralgpu], ByteNet [kalchbrenner2016neural] and ConvS2S [gehring2017convolutional], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these

models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [hochreiter2001gradient]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in Section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [cheng2016long, decomposableattentionmodel, paulus2017deep, lin2017structured].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [sukhbaatar2015].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [sutskever2014sequence, bahdanau2014neural] and [recurrentleastsquares, lecun2015deep].

15.4 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [cho2014learning, bahdanau2014neural, sutskever2014sequence]. Here, the encoder maps an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \dots, z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time. At each step the model is auto-regressive [graves2013generating], consuming the previously generated symbols as additional input when generating the next.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

15.4.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [he2016deep] around each of the two sub-layers, followed by layer normalization [ba2016layer]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

15.4.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

15.4.2.1 Scaled Dot-Product Attention

We call our particular attention “Scaled Dot-Product Attention” (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The two most commonly used attention functions are additive attention [bahdanau2014neural], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [DBLP:journals/corr/BritzGLL17]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

15.4.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

15.4.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In “encoder-decoder attention” layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [wu2016google, bahdanau2014neural, decomposableattentionmodel].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input to the softmax which correspond to illegal connections.

15.4.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{\text{ff}} = 2048$.

15.4.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [press2016using]. In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.

15.4.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add “positional encodings” to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [gehring2017convolutional].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

We also experimented with using learned positional embeddings [gehring2017convolutional] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

15.5 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, \dots, x_n)$ to another sequence of equal length

$(z_1, \dots, z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [hochreiter2001gradient]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [wu2016google] and byte-pair [sennrich2015neural] representations.

To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(\log_k(n))$ in the case of dilated convolutions [yu2015multi], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k . Separable convolutions [chollet2016xception], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

15.6 Training

This section describes the training regime for our models.

15.6.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [sennrich2015neural], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [wu2016google]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

15.6.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000

steps or 12 hours. For our big models, step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

15.6.3 Optimizer

We used the Adam optimizer [kingma2014adam] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

This corresponds to increasing the learning rate linearly for the first *warmup_steps* training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used *warmup_steps* = 4000.

15.6.4 Regularization

We employ three types of regularization during training:

Residual Dropout. We apply dropout [srivastava2014dropout] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

Label Smoothing. During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [szegedy2015rethinking]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

15.7 Results

15.7.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models including ensembles by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [wu2016google]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length +50, but terminate early when possible [wu2016google].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU.

15.7.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [gehring2017convolutional], and observe nearly identical results to the base model.

15.7.3 English Constituency Parsing

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [vinyals2015grammar].

We trained a 4-layer transformer with $d_{\text{model}} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [marcus1993building], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkeleyParser corpora from with approximately 17M sentences [vinyals2015grammar, zhu2013fast]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we increased the maximum output length to input length +300. We used a beam size of 21 and $\alpha = 0.3$ for both WSJ only and the semi-supervised setting.

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [dyer2016recurrent].

In contrast to RNN sequence-to-sequence models [vinyals2015grammar], the Transformer outperforms the BerkeleyParser [petrov2006learning] even when training only on the WSJ training set of 40K sentences.

15.8 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. We achieved a new state of the art on both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goal of ours.

The code we used to train and evaluate our models is available at <https://github.com/tensorflow/tensor2tensor>.

第 16 章 《Deep Neural Networks for YouTube Recommendations》中文版

16.1 摘要

YouTube 是现存规模最大、最复杂的工业级推荐系统之一。本文从高层视角描述该系统，重点介绍深度学习所带来的显著性能提升。全文按照经典的两阶段信息检索范式组织：首先详细介绍一个深度候选生成模型，然后描述一个独立的深度排序模型。我们还将分享在设计、迭代和维护这一对用户产生巨大影响的大规模推荐系统过程中所积累的实用经验与洞见。

关键词：推荐系统；深度学习；可扩展性

16.2 引言

YouTube 是全球最大的视频创作、分享与发现平台。YouTube 推荐系统负责帮助超过十亿用户从持续增长的视频库中发现个性化内容。本文重点关注深度学习近期对 YouTube 视频推荐系统所产生的巨大影响。图 1 展示了 YouTube 移动应用首页上的推荐界面。

YouTube 视频推荐面临以下三大主要挑战：

- **规模 (Scale)：**许多在小规模问题上表现良好的推荐算法在我们的规模下无法正常工作。处理 YouTube 庞大的用户群体和内容库，需要高度专业化的分布式学习算法和高效的服务系统。
- **时效性 (Freshness)：**YouTube 的内容库极为动态，每秒都有大量视频上传。推荐系统需要足够灵敏，既能为新上传的内容建模，又能捕捉用户的最新行为。在探索/利用 (exploration/exploitation) 视角下，新内容与成熟视频之间的平衡是一个核心问题。
- **噪声 (Noise)：**YouTube 用户的历史行为由于稀疏性及各种不可观测的外部因素，天然难以预测。我们几乎无法获得用户满意度的真实标签，只能对含噪的隐式反馈信号建模。此外，内容相关的元数据结构混乱，缺乏明确的本体定义。算法需要对训练数据的这些特性保持鲁棒性。

与谷歌旗下其他产品领域协同，YouTube 已完成向深度学习的根本性范式转变，将其作为几乎所有学习问题的通用解决方案。我们的系统构建在 Google Brain 之上，该平台近期以 TensorFlow 的形式开源。TensorFlow 提供了一个灵活的框架，可在大规模分布式训练环境下实验各种深度神经网络架构。我们的模型学习约十亿个参数，并在数千亿个样本上进行训练。

与大量矩阵分解方法的研究相比，将深度神经网络应用于推荐系统的工作相对较少。神经网络已被用于新闻推荐、引文推荐和评分预测；协同过滤被构建为深度神经网络，自编码器亦被用于此目的。Elkahrky 等人利用深度学习进行跨域用户建模；在基于内容的场景中，Borges 等人将深度神经网络用于音乐推荐。

本文的组织结构如下：第 2 节简要介绍系统概述；第 3 节更详细地描述候选生成模型，包括其训练方式以及如何用于服务推荐，实验结果将展示该模型如何从更深的隐藏层和额外的异质信号中受益；第 4 节详细介绍排序模型，包括如何改进经典逻辑回归以训练预测期望观看时长（而非点击概率）的模型，实验结果将表明隐藏层深度在此同样有益；最后，第 5 节呈现我们的结论与经验教训。

16.3 系统概述

我们推荐系统的整体架构如图 2 所示。该系统由两个神经网络组成：一个用于候选生成 (candidate generation)，另一个用于排序 (ranking)。

候选生成网络以用户在 YouTube 上的活动历史作为输入，从庞大的视频库中检索出一小部分（数百个）可能与用户相关的视频。这些候选视频具有较高的精确率，能够总体上满足用户的相关性需求。候选生成网络仅

通过协同过滤提供粗粒度的个性化。用户之间的相似性通过粗粒度特征来表达，例如观看视频的 ID、搜索词令牌以及人口统计信息。

在列表中呈现少量“最佳”推荐，需要对候选视频的相对重要性进行细粒度的区分，以实现高召回率。排序网络通过使用描述视频与用户的丰富特征集，依据目标函数为每个视频分配一个分数来完成这一任务。得分最高的视频按分数排序后呈现给用户。

两阶段推荐方法使我们能够从数百万规模的庞大视频库中生成推荐，同时确保最终呈现在用户设备上的少量视频是个性化的、能够吸引用户的。此外，这一设计还支持融合来自其他来源的候选视频，例如早期工作中描述的那些来源。

在开发过程中，我们大量使用离线指标（精确率、召回率、排序损失等）来指导系统的迭代改进。然而，对于算法或模型有效性的最终判断，我们依赖于在线 A/B 实验。在线实验可以衡量点击率、观看时长以及其他用户参与度指标的细微变化。这一点至关重要，因为在线 A/B 实验结果并不总是与离线实验结果相关。

16.4 候选生成

在候选生成阶段，庞大的 YouTube 视频库被精简为数百个可能与用户相关的视频。本文所描述的推荐系统的前身是一种在排序损失下训练的矩阵分解方法。我们神经网络模型的早期迭代版本通过仅嵌入用户历史观看记录的浅层网络来模拟这种分解行为。从这个角度来看，我们的方法可以视为分解技术的非线性推广。

16.4.1 将推荐建模为分类

我们将推荐问题建模为极端多分类问题，预测任务转化为：给定用户 U 和上下文 C ，在包含数百万视频 i （类别）的视频库 V 中，精确分类用户在时刻 t 将观看的特定视频 w_t ，

$$P(w_t = i | U, C) = \frac{e^{v_i^u}}{\sum_{j \in V} e^{v_j^u}}$$

其中 $u \in \mathbb{R}^N$ 表示用户与上下文对的高维“嵌入（embedding）”， $v_j \in \mathbb{R}^N$ 表示各候选视频的嵌入。在此设定下，嵌入即是稀疏实体（单个视频、用户等）映射到 $\mathbb{R}^N$ 中稠密向量的映射。深度神经网络的任务是以用户的历史记录和上下文为输入，学习用户嵌入 u ，使其能够通过 softmax 分类器对视频进行有效区分。

尽管 YouTube 上存在显式反馈机制（点赞/点踩、产品内调查等），我们仍选择使用观看行为这一隐式反馈来训练模型，以用户看完视频作为正样本。这一选择基于以下考量：可用的隐式用户历史数据在量级上远超显式反馈，使我们能够为长尾内容生成推荐，而在这些领域显式反馈极为稀疏。

高效的极端多分类（Efficient Extreme Multiclass）

为了在数百万类别下高效训练此类模型，我们采用从背景分布中采样负类（“候选采样”）的技术，并通过重要性加权对采样偏差进行校正。对于每个样本，交叉熵损失在真实标签和采样的负类上同时被最小化。实践中采样数千个负样本，相比传统 softmax 实现了超过 100 倍的加速。层次化 softmax 是另一种常见替代方案，但我们未能达到可比的精度。在层次化 softmax 中，遍历树中每个节点需要在通常不相关的类别集合之间进行判别，这使分类问题更加困难，导致性能下降。

在服务阶段，需要计算最可能的 N 个类别（视频）以选出排名前 N 的视频呈现给用户。在数十毫秒的严格服务延迟下对数百万候选项打分，需要一种次线性复杂度的近似评分方案。YouTube 此前的系统依赖哈希方法，本文描述的分类器采用了类似的方案。由于服务时不需要 softmax 输出层的校准似然值，评分问题退化为点积空间中的最近邻搜索，可使用通用库实现。我们发现 A/B 实验结果对最近邻搜索算法的选择并不十分敏感。

16.4.2 模型架构

受连续词袋语言模型的启发，我们在固定词表中为每个视频学习高维嵌入，并将这些嵌入输入前馈神经网络。用户的观看历史由一个变长的稀疏视频 ID 序列表示，通过嵌入映射为稠密向量表示。网络需要固定大小的稠密输入，实验表明对嵌入取平均在多种策略（求和、逐元素最大值等）中表现最佳。重要的是，嵌入与其他所

有模型参数通过正常的梯度下降反向传播联合学习。特征被拼接成一个宽的第一层，随后是若干全连接整流线性单元（ReLU）层。图 3 展示了加入以下所述非视频观看特征后的通用网络架构。

16.4.3 异质信号

将深度神经网络作为矩阵分解推广的一个关键优势在于，任意连续型和类别型特征都可以方便地加入模型。搜索历史的处理方式与观看历史类似——每个查询被切分为 unigram 和 bigram，每个 token 被嵌入。取平均后，用户经过分词和嵌入处理的查询序列代表了一份压缩的稠密搜索历史。人口统计特征对于为新用户提供先验信息、使推荐行为合理化至关重要。用户的地理区域和设备被嵌入后拼接。性别、登录状态和年龄等简单二值型和连续型特征以归一化到 $[0, 1]$ 的实数值直接输入网络。

“样本年龄”特征（“Example Age” Feature）

每秒都有大量视频上传至 YouTube。推荐新上传的（“新鲜”）内容对 YouTube 作为产品而言极为重要。我们持续观察到用户偏好新鲜内容，但不以牺牲相关性为代价。除了简单推荐用户想看的新视频这一一阶效应外，还存在引导和传播病毒式内容这一关键的二阶现象。

机器学习系统通常对历史存在隐式偏差，因为它们是从历史样本中预测未来行为训练而来的。视频流行度的分布高度非平稳，但我们推荐系统所产生的语料库上的多项分布将反映数周训练窗口内的平均观看可能性。为了纠正这一偏差，我们在训练时将训练样本的年龄（age）作为特征输入。在服务时，该特征被设为零（或略为负值），以体现模型是在训练窗口末端做出预测。

图 4 展示了该方法在一个任意选取的视频上的有效性。

16.4.4 标签与上下文选择

需要特别强调的是，推荐往往涉及求解一个代理问题（*surrogate problem*）并将结果迁移到特定场景。一个经典例子是“准确预测评分可以带来有效电影推荐”这一假设。我们发现，代理学习问题的选择对 A/B 测试中的性能影响极大，但在离线实验中却很难衡量。

训练样本来自所有 YouTube 观看记录（包括嵌入在其他网站上的观看），而非仅限于我们推荐产生的观看。否则，新内容将极难浮现，推荐系统也会对利用（*exploitation*）产生过度偏倚。若用户通过推荐以外的途径发现视频，我们希望能够通过协同过滤迅速将这一发现传播给其他用户。另一个改善在线指标的关键洞见是：为每位用户生成固定数量的训练样本，从而在损失函数中对用户进行等权重处理，防止少数高活跃用户主导损失。

有些违反直觉的是，为防止模型利用网站结构并对代理问题过拟合，必须谨慎地对分类器隐藏某些信息。举例来说，若用户刚刚搜索了“taylor swift”，由于问题被建模为预测下一个观看视频，给定该信息的分类器会预测最可能被观看的视频正是对应搜索结果页上出现的视频。不出所料，将用户最近的搜索页面作为首页推荐的表现极差。通过丢弃序列信息、以无序词袋表示搜索查询，分类器不再能直接感知标签的来源。

视频的自然消费模式通常导致非常不对称的共同观看概率。系列剧通常按顺序观看，用户往往从某类型中最广为人知的内容开始，逐渐深入到更细分的领域。因此，我们发现预测用户的下一次观看，比预测随机留出的观看效果更好（图 5）。许多协同过滤系统通过留出随机一项并从用户历史中的其他项进行预测来隐式选择标签和上下文（图 5a），这会泄露未来信息并忽略非对称的消费模式。与之相反，我们通过选择一次随机观看来“回滚”用户历史，仅将用户在留出标签观看之前的行为作为输入（图 5b）。

16.4.5 特征与深度实验

如图 6 所示，增加特征和深度能显著提升留出数据上的精确率。在这些实验中，100 万个视频词表和 100 万个搜索词令牌词表各以 256 个浮点数嵌入，最大词袋大小为 50 条最近观看和 50 条最近搜索。softmax 层在相同的 100 万视频类别上输出维度为 256 的多项分布（可视为一个独立的输出视频嵌入）。这些模型在所有 YouTube 用户上训练至收敛，对应在数据上进行若干个 epoch。网络结构遵循常见的“塔形”模式——网络底层最宽，每层隐藏层的单元数减半（类似图 3）。深度为零的网络实际上是一个线性分解方案，与前代系统的表现非常相似。

不断增加宽度和深度，直到增量收益减弱、收敛变得困难为止：

- 深度 0：一个线性层，将拼接层变换至 softmax 维度 256
- 深度 1：256 ReLU
- 深度 2：512 ReLU → 256 ReLU
- 深度 3：1024 ReLU → 512 ReLU → 256 ReLU
- 深度 4：2048 ReLU → 1024 ReLU → 512 ReLU → 256 ReLU

16.5 排序

排序的核心职责是利用曝光数据，针对特定用户界面对候选预测结果进行专项化校准。例如，某用户总体上以较高概率观看某视频，但由于缩略图的选择，未必会点击该视频在首页上的特定曝光位。在排序阶段，由于只需对数百个视频打分（而非候选生成阶段的数百万个），我们能够获取大量描述视频与用户关系的特征。排序对于融合来自不同候选来源、分数不可直接比较的结果同样至关重要。

我们使用与候选生成架构类似的深度神经网络，通过逻辑回归（图 7）为每个视频曝光独立打分，然后按分数排序返回给用户。最终排序目标依据在线 A/B 测试结果持续调整，但通常是每次曝光期望观看时长的简单函数。按点击率排序往往会推广用户不会看完的欺骗性视频（“标题党”），而观看时长能更好地衡量用户参与度。

16.5.1 特征表示

我们的特征遵循传统的类别型与连续型/有序型特征分类体系。所使用的类别型特征在基数上差异悬殊——有些是二值型（如用户是否登录），另一些则有数百万个可能取值（如用户的最近一次搜索词）。特征还按照是贡献单一值（“单值型”）还是一组值（“多值型”）进一步区分。单值类别型特征的典型例子是当前打分的视频 ID，对应的多值特征则可以是用户观看的最近 N 个视频 ID 的词袋。此外，特征还按是否描述候选项属性（“曝光”特征）或用户/上下文属性（“查询”特征）进行分类。查询特征每次请求只计算一次，而曝光特征则需对每个打分候选项分别计算。

特征工程 (Feature Engineering)

我们的排序模型通常使用数百个特征，类别型与连续型特征大致各占一半。尽管深度学习有望减轻手工特征工程的负担，但我们的原始数据性质使其无法直接输入前馈神经网络。我们仍然投入大量工程资源将用户和视频数据转化为有用的特征，核心挑战在于如何表示用户行为的时序序列，以及这些行为与当前打分视频曝光之间的关系。

我们观察到最重要的信号，是描述用户与候选项本身及相似候选项的历史交互情况的特征，与其他排序广告经验相吻合。例如，考虑用户与当前打分视频所属频道的历史关系——用户观看了该频道多少视频？用户最近一次观看该主题内容是什么时候？这些描述用户在相关内容上历史行为的连续型特征尤为强大，因为它们能在不同内容之间良好泛化。我们还发现，将候选生成阶段的信息以特征形式传递给排序阶段至关重要，例如：哪些来源提名了这个候选视频？它们分别给出了怎样的分数？

描述历史视频曝光频率的特征对于在推荐中引入“扰动 (churn)”同样关键——确保连续请求不会返回完全相同的列表。若某视频最近被推荐给用户但未被观看，模型将在下次页面加载时自然地降低该曝光的排名。实时提供最新的曝光和观看历史本身就是一项重大的工程挑战，超出了本文讨论范围，但对于提供响应灵敏的推荐至关重要。

嵌入类别型特征 (Embedding Categorical Features)

与候选生成类似，我们使用嵌入将稀疏类别型特征映射为适合神经网络的稠密表示。每个唯一 ID 空间（“词表”）有一个独立的可学习嵌入，其维度大致与唯一取值数量的对数成正比。这些词表是在训练开始前对数据进行一次遍历后构建的简单查找表。极高基数的 ID 空间（如视频 ID 或搜索词）通过仅保留按点击曝光频率排序的前 N 个进行截断。词表外的取值简单地映射到零嵌入。与候选生成相同，多值类别型特征的嵌入在输入网络前取平均。

重要的是，同一 ID 空间内的类别型特征共享底层嵌入。例如，存在一个单一的全局视频 ID 嵌入，被许多不同特征使用（曝光的视频 ID、用户最近观看的视频 ID、“种子”推荐的视频 ID 等）。尽管共享嵌入，每个特征仍单独输入网络，使上层网络能够为每个特征学习专门的表示。共享嵌入有助于改善泛化、加速训练并降低内存需求。绝大多数模型参数都集中在这些高基数嵌入空间中——例如，100 万个 ID 在 32 维空间中嵌入的参数量是 2048 单元全连接层的 7 倍。

连续型特征归一化 (Normalizing Continuous Features)

神经网络对输入的尺度和分布极为敏感，而决策树集成等方法则对单个特征的缩放不变。我们发现，对连续型特征进行合理归一化对收敛至关重要。对于分布为 f 的连续型特征 x ，通过累积分布函数 $\tilde{x} = \int_{-\infty}^x df$ 将其缩放至在 $[0, 1)$ 上均匀分布，得到归一化特征 $\tilde{x}$ 。该积分在训练开始前对数据进行一次遍历，通过特征值分位数上的线性插值近似。

除原始归一化特征 $\tilde{x}$ 外，我们还将其幂次 $\tilde{x}^2$ 和 $\sqrt{\tilde{x}}$ 一并输入，赋予网络更强的表达能力，使其能够轻松构建特征的超线性和次线性函数。实验表明，输入连续型特征的幂次能够改善离线准确率。

16.5.2 期望观看时长建模

我们的目标是根据正样本（视频曝光被点击）和负样本（曝光未被点击）的训练样本，预测期望观看时长。正样本被标注了用户观看该视频所花费的时间。为预测期望观看时长，我们采用加权逻辑回归技术，该技术正是为此目的而开发的。

模型在交叉熵损失下以逻辑回归训练（图 7）。然而，正样本（被点击的曝光）以该视频的实际观看时长加权，负样本（未被点击的曝光）则统一赋予单位权重。由此，逻辑回归所学习的赔率为 $\frac{\sum T_i}{N-k}$ ，其中 N 为训练样本总数， k 为正样本数， T_i 为第 i 个样本的观看时长。假设正样本比例较小（在我们的场景下确实如此），学习到的赔率近似为 $E[T](1+P)$ ，其中 P 为点击概率， $E[T]$ 为该曝光的期望观看时长。由于 P 较小，该乘积接近于 $E[T]$ 。在推断时，我们使用指数函数 e^x 作为最终激活函数，以产生对期望观看时长的近似估计。

16.5.3 隐藏层实验

表 1 展示了我们在不同隐藏层配置下的次日留出数据结果。每种配置的指标值（“加权逐用户损失”）通过考察单次页面加载中呈现给用户的正样本（被点击）和负样本（未被点击）曝光对来计算。我们首先用模型对两个曝光打分：若负样本得分高于正样本，则认为正样本的观看时长被错误预测。加权逐用户损失即为留出曝光对中，被错误预测的观看时长占总观看时长的比例。

结果表明，增加隐藏层宽度和深度均能改善性能。然而，代价是推断所需的服务端 CPU 时间。 $1024 \rightarrow 512 \rightarrow 256$ 的 ReLU 配置在满足服务 CPU 预算的前提下取得了最佳结果。

对于 $1024 \rightarrow 512 \rightarrow 256$ 模型，我们尝试仅输入归一化连续型特征而不加入其幂次，损失增加了 0.2%。在相同隐藏层配置下，我们还训练了一个对正负样本等权重处理的模型，正如预期，观看时长加权损失显著上升了 4.1%。

16.6 结论

我们描述了用于 YouTube 视频推荐的深度神经网络架构，将其拆分为两个独立问题：候选生成与排序。

我们的深度协同过滤模型能够有效融合多种信号，并通过多层深度对信号间的交互建模，优于 YouTube 此前使用的矩阵分解方法。在为推荐选择代理问题时，艺术性多于科学性——我们发现，预测用户的未来观看在线指标上表现优异，这得益于它能捕捉非对称的共同观看行为，并防止未来信息的泄露。对分类器隐藏判别性信号同样是取得良好结果的关键——否则模型会对代理问题过拟合，无法良好迁移到首页场景。

我们证明，将训练样本的年龄作为输入特征，能够消除模型对历史的隐式偏倚，使模型能够表示视频流行度的时间依赖行为。这改善了离线留出精确率，并在 A/B 测试中显著提升了近期上传视频的观看时长。

排序是一个更为经典的机器学习问题，但我们的深度学习方法在观看时长预测上优于此前的线性和基于树的方法。推荐系统尤其受益于描述用户对候选项历史行为的专项特征。深度神经网络要求对类别型和连续型特征进行特殊表示，我们分别通过嵌入和分位数归一化进行处理。多层深度被证明能够有效建模数百个特征间的非线性交互。

逻辑回归通过对正样本以观看时长加权、对负样本赋予单位权重进行改进，使我们能够学习到对期望观看时长精确建模的赔率。该方法在以观看时长加权的排序评估指标上，远优于直接预测点击率的方法。

第 17 章 Deep Neural Networks for YouTube Recommendations

17.1 Abstract

YouTube represents one of the largest scale and most sophisticated industrial recommendation systems in existence. In this paper, we describe the system at a high level and focus on the dramatic performance improvements brought by deep learning. The paper is split according to the classic two-stage information retrieval dichotomy: first, we detail a deep candidate generation model and then describe a separate deep ranking model. We also provide practical lessons and insights derived from designing, iterating and maintaining a massive recommendation system with enormous user-facing impact.

Keywords: recommender system; deep learning; scalability

17.2 Introduction

YouTube is the world's largest platform for creating, sharing and discovering video content. YouTube recommendations are responsible for helping more than a billion users discover personalized content from an ever-growing corpus of videos. In this paper we will focus on the immense impact deep learning has recently had on the YouTube video recommendations system. Figure 1 illustrates the recommendations on the YouTube mobile app home.

Recommending YouTube videos is extremely challenging from three major perspectives:

- **Scale:** Many existing recommendation algorithms proven to work well on small problems fail to operate on our scale. Highly specialized distributed learning algorithms and efficient serving systems are essential for handling YouTube's massive user base and corpus.
- **Freshness:** YouTube has a very dynamic corpus with many hours of video are uploaded per second. The recommendation system should be responsive enough to model newly uploaded content as well as the latest actions taken by the user. Balancing new content with well-established videos can be understood from an exploration/exploitation perspective.
- **Noise:** Historical user behavior on YouTube is inherently difficult to predict due to sparsity and a variety of unobservable external factors. We rarely obtain the ground truth of user satisfaction and instead model noisy implicit feedback signals. Furthermore, metadata associated with content is poorly structured without a well defined ontology. Our algorithms need to be robust to these particular characteristics of our training data.

In conjugation with other product areas across Google, YouTube has undergone a fundamental paradigm shift towards using deep learning as a general-purpose solution for nearly all learning problems. Our system is built on Google Brain [dean2012large] which was recently open sourced as TensorFlow [tensorflow2015]. TensorFlow provides a flexible framework for experimenting with various deep neural network architectures using large-scale distributed training. Our models learn approximately one billion parameters and are trained on hundreds of billions of examples.

In contrast to vast amount of research in matrix factorization methods [su2009survey], there is relatively little work using deep neural networks for recommendation systems. Neural networks are used for recommending news in [oh2014personalized], citations in [huang2015neural] and review ratings in [tang2015user]. Collaborative filtering is formulated as a deep neural network in [wang2015collaborative] and autoencoders in [sedhain2015autorec]. Elkahky et al. used deep learning for cross domain user modeling [elkahky2015multi]. In a content-based setting, Burges et al. used deep neural networks for music recommendation [vandennoord2013deep].

The paper is organized as follows: A brief system overview is presented in Section 2. Section 3 describes the candidate generation model in more detail, including how it is trained and used to serve recommendations. Experimental results will

show how the model benefits from deep layers of hidden units and additional heterogeneous signals. Section 4 details the ranking model, including how classic logistic regression is modified to train a model predicting expected watch time (rather than click probability). Experimental results will show that hidden layer depth is helpful as well in this situation. Finally, Section 5 presents our conclusions and lessons learned.

17.3 System Overview

The overall structure of our recommendation system is illustrated in Figure 2. The system is comprised of two neural networks: one for *candidate generation* and one for *ranking*.

The candidate generation network takes events from the user’s YouTube activity history as input and retrieves a small subset (hundreds) of videos from a large corpus. These candidates are intended to be generally relevant to the user with high precision. The candidate generation network only provides broad personalization via collaborative filtering. The similarity between users is expressed in terms of coarse features such as IDs of video watches, search query tokens and demographics.

Presenting a few “best” recommendations in a list requires a fine-level representation to distinguish relative importance among candidates with high recall. The ranking network accomplishes this task by assigning a score to each video according to a desired objective function using a rich set of features describing the video and user. The highest scoring videos are presented to the user, ranked by their score.

The two-stage approach to recommendation allows us to make recommendations from a very large corpus (millions) of videos while still being certain that the small number of videos appearing on the device are personalized and engaging for the user. Furthermore, this design enables blending candidates generated by other sources, such as those described in an earlier work [davidson2010youtube].

During development, we make extensive use of offline metrics (precision, recall, ranking loss, etc.) to guide iterative improvements to our system. However for the final determination of the effectiveness of an algorithm or model, we rely on A/B testing via live experiments. In a live experiment, we can measure subtle changes in click-through rate, watch time, and many other metrics that measure user engagement. This is important because live A/B results are not always correlated with offline experiments.

17.4 Candidate Generation

During candidate generation, the enormous YouTube corpus is winnowed down to hundreds of videos that may be relevant to the user. The predecessor to the recommender described here was a matrix factorization approach trained under rank loss [weston2011wsabie]. Early iterations of our neural network model mimicked this factorization behavior with shallow networks that only embedded the user’s previous watches. From this perspective, our approach can be viewed as a non-linear generalization of factorization techniques.

17.4.1 Recommendation as Classification

We pose recommendation as extreme multiclass classification where the prediction problem becomes accurately classifying a specific video watch w_t at time t among millions of videos i (classes) from a corpus V based on a user U and context C ,

$$P(w_t = i \mid U, C) = \frac{e^{v_i u}}{\sum_{j \in V} e^{v_j u}}$$

where $u \in \mathbb{R}^N$ represents a high-dimensional “embedding” of the user, context pair and the $v_j \in \mathbb{R}^N$ represent embeddings of each candidate video. In this setting, an embedding is simply a mapping of sparse entities (individual videos, users etc.)

into a dense vector in $\mathbb{R}^N$. The task of the deep neural network is to learn user embeddings u as a function of the user’s history and context that are useful for discriminating among videos with a softmax classifier.

Although explicit feedback mechanisms exist on YouTube (thumbs up/down, in-product surveys, etc.) we use the implicit feedback [oard1998implicit] of watches to train the model, where a user completing a video is a positive example. This choice is based on the orders of magnitude more implicit user history available, allowing us to produce recommendations deep in the tail where explicit feedback is extremely sparse.

Efficient Extreme Multiclass

To efficiently train such a model with millions of classes, we rely on a technique to sample negative classes from the background distribution (“candidate sampling”) and then correct for this sampling via importance weighting [jean2014using]. For each example the cross-entropy loss is minimized for the true label and the sampled negative classes. In practice several thousand negatives are sampled, corresponding to more than 100 times speedup over traditional softmax. A popular alternative approach is hierarchical softmax [morin2005hierarchical], but we weren’t able to achieve comparable accuracy. In hierarchical softmax, traversing each node in the tree involves discriminating between sets of classes that are often unrelated, making the classification problem much more difficult and degrading performance.

At serving time we need to compute the most likely N classes (videos) in order to choose the top N to present to the user. Scoring millions of items under a strict serving latency of tens of milliseconds requires an approximate scoring scheme sublinear in the number of classes. Previous systems at YouTube relied on hashing [weston2013label] and the classifier described here uses a similar approach. Since calibrated likelihoods from the softmax output layer are not needed at serving time, the scoring problem reduces to a nearest neighbor search in the dot product space for which general purpose libraries can be used [liu2004investigation]. We found that A/B results were not particularly sensitive to the choice of nearest neighbor search algorithm.

17.4.2 Model Architecture

Inspired by continuous bag of words language models [mikolov2013distributed], we learn high dimensional embeddings for each video in a fixed vocabulary and feed these embeddings into a feedforward neural network. A user’s watch history is represented by a variable-length sequence of sparse video IDs which is mapped to a dense vector representation via the embeddings. The network requires fixed-sized dense inputs and simply averaging the embeddings performed best among several strategies (sum, component-wise max, etc.). Importantly, the embeddings are learned jointly with all other model parameters through normal gradient descent backpropagation updates. Features are concatenated into a wide first layer, followed by several layers of fully connected Rectified Linear Units (ReLU) [glorot2011deep]. Figure 3 shows the general network architecture with additional non-video watch features described below.

17.4.3 Heterogeneous Signals

A key advantage of using deep neural networks as a generalization of matrix factorization is that arbitrary continuous and categorical features can be easily added to the model. Search history is treated similarly to watch history — each query is tokenized into unigrams and bigrams and each token is embedded. Once averaged, the user’s tokenized, embedded queries represent a summarized dense search history. Demographic features are important for providing priors so that the recommendations behave reasonably for new users. The user’s geographic region and device are embedded and concatenated. Simple binary and continuous features such as the user’s gender, logged-in state and age are input directly into the network as real values normalized to $[0, 1]$.

“Example Age” Feature

Many hours worth of videos are uploaded each second to YouTube. Recommending this recently uploaded (“fresh”) content is extremely important for YouTube as a product. We consistently observe that users prefer fresh content, though not at the expense of relevance. In addition to the first-order effect of simply recommending new videos that users want to

watch, there is a critical secondary phenomenon of bootstrapping and propagating viral content [jiang2014viral].

Machine learning systems often exhibit an implicit bias towards the past because they are trained to predict future behavior from historical examples. The distribution of video popularity is highly non-stationary but the multinomial distribution over the corpus produced by our recommender will reflect the average watch likelihood in the training window of several weeks. To correct for this, we feed the age of the training example as a feature during training. At serving time, this feature is set to zero (or slightly negative) to reflect that the model is making predictions at the very end of the training window.

Figure 4 demonstrates the efficacy of this approach on an arbitrarily chosen video [zayn2016pillowtalk].

17.4.4 Label and Context Selection

It is important to emphasize that recommendation often involves solving a *surrogate problem* and transferring the result to a particular context. A classic example is the assumption that accurately predicting ratings leads to effective movie recommendations [amatriain2012building]. We have found that the choice of this surrogate learning problem has an outsized importance on performance in A/B testing but is very difficult to measure with offline experiments.

Training examples are generated from all YouTube watches (even those embedded on other sites) rather than just watches on the recommendations we produce. Otherwise, it would be very difficult for new content to surface and the recommender would be overly biased towards exploitation. If users are discovering videos through means other than our recommendations, we want to be able to quickly propagate this discovery to others via collaborative filtering. Another key insight that improved live metrics was to generate a fixed number of training examples per user, effectively weighting our users equally in the loss function. This prevented a small cohort of highly active users from dominating the loss.

Somewhat counter-intuitively, great care must be taken to *withhold information from the classifier* in order to prevent the model from exploiting the structure of the site and overfitting the surrogate problem. Consider as an example a case in which the user has just issued a search query for “taylor swift”. Since our problem is posed as predicting the next watched video, a classifier given this information will predict that the most likely videos to be watched are those which appear on the corresponding search results page for “taylor swift”. Unsurprisingly, reproducing the user’s last search page as homepage recommendations performs very poorly. By discarding sequence information and representing search queries with an unordered bag of tokens, the classifier is no longer directly aware of the origin of the label.

Natural consumption patterns of videos typically lead to very asymmetric co-watch probabilities. Episodic series are usually watched sequentially and users often discover artists in a genre beginning with the most broadly popular before focusing on smaller niches. We therefore found much better performance predicting the user’s next watch, rather than predicting a randomly held-out watch (Figure 5). Many collaborative filtering systems implicitly choose the labels and context by holding out a random item and predicting it from other items in the user’s history (5a). This leaks future information and ignores any asymmetric consumption patterns. In contrast, we “rollback” a user’s history by choosing a random watch and only input actions the user took before the held-out label watch (5b).

17.4.5 Experiments with Features and Depth

Adding features and depth significantly improves precision on holdout data as shown in Figure 6. In these experiments, a vocabulary of 1M videos and 1M search tokens were embedded with 256 floats each in a maximum bag size of 50 recent watches and 50 recent searches. The softmax layer outputs a multinomial distribution over the same 1M video classes with a dimension of 256 (which can be thought of as a separate output video embedding). These models were trained until convergence over all YouTube users, corresponding to several epochs over the data. Network structure followed a common “tower” pattern in which the bottom of the network is widest and each successive hidden layer halves the number of units (similar to Figure 3). The depth zero network is effectively a linear factorization scheme which performed very similarly to the predecessor system. Width and depth were added until the incremental benefit diminished and convergence became

difficult:

- Depth 0: A linear layer simply transforms the concatenation layer to match the softmax dimension of 256
- Depth 1: 256 ReLU
- Depth 2: 512 ReLU $\rightarrow$ 256 ReLU
- Depth 3: 1024 ReLU $\rightarrow$ 512 ReLU $\rightarrow$ 256 ReLU
- Depth 4: 2048 ReLU $\rightarrow$ 1024 ReLU $\rightarrow$ 512 ReLU $\rightarrow$ 256 ReLU

17.5 Ranking

The primary role of ranking is to use impression data to specialize and calibrate candidate predictions for the particular user interface. For example, a user may watch a given video with high probability generally but is unlikely to click on the specific homepage impression due to the choice of thumbnail image. During ranking, we have access to many more features describing the video and the user’s relationship to the video because only a few hundred videos are being scored rather than the millions scored in candidate generation. Ranking is also crucial for ensembling different candidate sources whose scores are not directly comparable.

We use a deep neural network with similar architecture as candidate generation to assign an independent score to each video impression using logistic regression (Figure 7). The list of videos is then sorted by this score and returned to the user. Our final ranking objective is constantly being tuned based on live A/B testing results but is generally a simple function of expected watch time per impression. Ranking by click-through rate often promotes deceptive videos that the user does not complete (“clickbait”) whereas watch time better captures engagement [meyerson2012watchtime, yi2014beyond].

17.5.1 Feature Representation

Our features are segregated with the traditional taxonomy of categorical and continuous/ordinal features. The categorical features we use vary widely in their cardinality — some are binary (e.g. whether the user is logged-in) while others have millions of possible values (e.g. the user’s last search query). Features are further split according to whether they contribute only a single value (“univalent”) or a set of values (“multivalent”). An example of a univalent categorical feature is the video ID of the impression being scored, while a corresponding multivalent feature might be a bag of the last N video IDs the user has watched. We also classify features according to whether they describe properties of the item (“impression”) or properties of the user/context (“query”). Query features are computed once per request while impression features are computed for each item scored.

Feature Engineering

We typically use hundreds of features in our ranking models, roughly split evenly between categorical and continuous. Despite the promise of deep learning to alleviate the burden of engineering features by hand, the nature of our raw data does not easily lend itself to be input directly into feedforward neural networks. We still expend considerable engineering resources transforming user and video data into useful features. The main challenge is in representing a temporal sequence of user actions and how these actions relate to the video impression being scored.

We observe that the most important signals are those that describe a user’s previous interaction with the item itself and other similar items, matching others’ experience in ranking ads [he2014practical]. As an example, consider the user’s past history with the channel that uploaded the video being scored — how many videos has the user watched from this channel? When was the last time the user watched a video on this topic? These continuous features describing past user actions on related items are particularly powerful because they generalize well across disparate items. We have also found it crucial to propagate information from candidate generation into ranking in the form of features, e.g. which sources nominated this video candidate? What scores did they assign?

Features describing the frequency of past video impressions are also critical for introducing “churn” in recommen-

dations (successive requests do not return identical lists). If a user was recently recommended a video but did not watch it then the model will naturally demote this impression on the next page load. Serving up-to-the-second impression and watch history is an engineering feat onto itself outside the scope of this paper, but is vital for producing responsive recommendations.

Embedding Categorical Features

Similar to candidate generation, we use embeddings to map sparse categorical features to dense representations suitable for neural networks. Each unique ID space (“vocabulary”) has a separate learned embedding with dimension that increases approximately proportional to the logarithm of the number of unique values. These vocabularies are simple look-up tables built by passing over the data once before training. Very large cardinality ID spaces (e.g. video IDs or search query terms) are truncated by including only the top N after sorting based on their frequency in clicked impressions. Out-of-vocabulary values are simply mapped to the zero embedding. As in candidate generation, multivalent categorical feature embeddings are averaged before being fed in to the network.

Importantly, categorical features in the same ID space also share underlying embeddings. For example, there exists a single global embedding of video IDs that many distinct features use (video ID of the impression, last video ID watched by the user, video ID that “seeded” the recommendation, etc.). Despite the shared embedding, each feature is fed separately into the network so that the layers above can learn specialized representations per feature. Sharing embeddings is important for improving generalization, speeding up training and reducing memory requirements. The overwhelming majority of model parameters are in these high-cardinality embedding spaces — for example, one million IDs embedded in a 32 dimensional space have 7 times more parameters than fully connected layers 2048 units wide.

Normalizing Continuous Features

Neural networks are notoriously sensitive to the scaling and distribution of their inputs [ioffe2015batch] whereas alternative approaches such as ensembles of decision trees are invariant to scaling of individual features. We found that proper normalization of continuous features was critical for convergence. A continuous feature x with distribution f is transformed to $\tilde{x}$ by scaling the values such that the feature is equally distributed in $[0, 1)$ using the cumulative distribution, $\tilde{x} = \int_{-\infty}^x df$. This integral is approximated with linear interpolation on the quantiles of the feature values computed in a single pass over the data before training begins.

In addition to the raw normalized feature $\tilde{x}$, we also input powers $\tilde{x}^2$ and $\sqrt{\tilde{x}}$, giving the network more expressive power by allowing it to easily form super- and sub-linear functions of the feature. Feeding powers of continuous features was found to improve offline accuracy.

17.5.2 Modeling Expected Watch Time

Our goal is to predict expected watch time given training examples that are either positive (the video impression was clicked) or negative (the impression was not clicked). Positive examples are annotated with the amount of time the user spent watching the video. To predict expected watch time we use the technique of weighted logistic regression, which was developed for this purpose.

The model is trained with logistic regression under cross-entropy loss (Figure 7). However, the positive (clicked) impressions are weighted by the observed watch time on the video. Negative (unclicked) impressions all receive unit weight. In this way, the odds learned by the logistic regression are $\frac{\sum T_i}{N-k}$ where N is the number of training examples, k is the number of positive impressions, and T_i is the watch time of the i th impression. Assuming the fraction of positive impressions is small (which is true in our case), the learned odds are approximately $E[T](1 + P)$, where P is the click probability and $E[T]$ is the expected watch time of the impression. Since P is small, this product is close to $E[T]$. For inference we use the exponential function e^x as the final activation function to produce these odds that closely estimate expected watch time.

17.5.3 Experiments with Hidden Layers

Table 1 shows the results we obtained on next-day holdout data with different hidden layer configurations. The value shown for each configuration (“weighted, per-user loss”) was obtained by considering both positive (clicked) and negative (unclicked) impressions shown to a user on a single page. We first score these two impressions with our model. If the negative impression receives a higher score than the positive impression, then we consider the positive impression’s watch time to be *mispredicted watch time*. Weighted, per-user loss is then the total amount mispredicted watch time as a fraction of total watch time over heldout impression pairs.

These results show that increasing the width of hidden layers improves results, as does increasing their depth. The trade-off, however, is server CPU time needed for inference. The configuration of a 1024-wide ReLU followed by a 512-wide ReLU followed by a 256-wide ReLU gave us the best results while enabling us to stay within our serving CPU budget.

For the $1024 \rightarrow 512 \rightarrow 256$ model we tried only feeding the normalized continuous features without their powers, which increased loss by 0.2%. With the same hidden layer configuration, we also trained a model where positive and negative examples are weighted equally. Unsurprisingly, this increased the watch time-weighted loss by a dramatic 4.1%.

17.6 Conclusions

We have described our deep neural network architecture for recommending YouTube videos, split into two distinct problems: candidate generation and ranking.

Our deep collaborative filtering model is able to effectively assimilate many signals and model their interaction with layers of depth, outperforming previous matrix factorization approaches used at YouTube [weston2011wsabie]. There is more art than science in selecting the surrogate problem for recommendations and we found classifying a future watch to perform well on live metrics by capturing asymmetric co-watch behavior and preventing leakage of future information. Withholding discriminative signals from the classifier was also essential to achieving good results — otherwise the model would overfit the surrogate problem and not transfer well to the homepage.

We demonstrated that using the age of the training example as an input feature removes an inherent bias towards the past and allows the model to represent the time-dependent behavior of popular of videos. This improved offline holdout precision results and increased the watch time dramatically on recently uploaded videos in A/B testing.

Ranking is a more classical machine learning problem yet our deep learning approach outperformed previous linear and tree-based methods for watch time prediction. Recommendation systems in particular benefit from specialized features describing past user behavior with items. Deep neural networks require special representations of categorical and continuous features which we transform with embeddings and quantile normalization, respectively. Layers of depth were shown to effectively model non-linear interactions between hundreds of features.

Logistic regression was modified by weighting training examples with watch time for positive examples and unity for negative examples, allowing us to learn odds that closely model expected watch time. This approach performed much better on watch-time weighted ranking evaluation metrics compared to predicting click-through rate directly.

第 18 章 XGBoost: A Scalable Tree Boosting System

 **笔记**完整英文原文见本地 PDF: `papers/xgboost_a_scalable_tree_boosting_system.pdf`。本章保留原论文标题、章节结构、核心公式、算法与图表位置, 便于和 PDF 原文逐节对照。

18.1 ABSTRACT

原文位置: PDF p.1, ABSTRACT。

结构要点:

- Tree boosting is highly effective and widely used.
- The paper describes XGBoost as a scalable end-to-end tree boosting system.
- Main contributions include sparsity-aware learning, weighted quantile sketch, cache-aware access, compression and sharding.

18.2 1. INTRODUCTION

原文位置: PDF p.1–p.2, Section 1。

结构要点:

- Large-scale machine learning needs both effective statistical models and scalable learning systems.
- Gradient tree boosting is strong in classification, ranking, ad click-through rate prediction and machine learning competitions.
- XGBoost is motivated by scalability in single-machine, distributed and out-of-core settings.
- The paper lists four major contributions:
 - scalable end-to-end tree boosting system;
 - weighted quantile sketch;
 - sparsity-aware parallel tree learning;
 - cache-aware block structure for out-of-core learning.

18.3 2. TREE BOOSTING IN A NUTSHELL

18.3.1 2.1 Regularized Learning Objective

原文位置: PDF p.2, Section 2.1。

原文公式:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}.$$

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k).$$

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2.$$

结构要点:

- Model prediction is the sum of K regression trees.

- The objective is training loss plus tree complexity penalty.
- Regularization controls overfitting by penalizing leaf number and leaf weights.

18.3.2 2.2 Gradient Tree Boosting

原文位置：PDF p.2–p.3, Section 2.2。

原文公式：

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t).$$

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t).$$

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)}), \quad h_i = \partial_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}^{(t-1)}).$$

$$I_j = \{i \mid q(x_i) = j\}.$$

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}.$$

$$\tilde{\mathcal{L}}^{(t)}(q) = - \frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T.$$

$$L_{\text{split}} = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i \right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma.$$

结构要点：

- Each iteration adds a new tree f_t .
- XGBoost uses second-order Taylor approximation.
- The optimal leaf weight can be computed from first-order and second-order gradient statistics.
- Split gain is derived from the regularized objective.

18.3.3 2.3 Shrinkage and Column Subsampling

原文位置：PDF p.3, Section 2.3。

结构要点：

- Shrinkage scales newly added tree weights by a factor η .
- Column subsampling borrows an idea from Random Forest.
- Both techniques help prevent overfitting; column subsampling can also speed up computation.

18.4 3. SPLIT FINDING ALGORITHMS

18.4.1 3.1 Basic Exact Greedy Algorithm

原文位置：PDF p.3, Section 3.1, Algorithm 1。

结构要点：

- Enumerates possible splits over all features.

- Data must be sorted by feature values.
- Gradient statistics are accumulated in sorted order.

18.4.2 3.2 Approximate Algorithm

原文位置：PDF p.3–p.4, Section 3.2, Algorithm 2, Figure 3。

结构要点：

- Exact greedy search is expensive when data is too large.
- Approximate algorithm proposes candidate split points by feature percentiles.
- Candidate proposal can be global or local.
- With enough candidates, approximate splitting can match exact greedy accuracy.

18.4.3 3.3 Weighted Quantile Sketch

原文位置：PDF p.4, Section 3.3。

原文公式：

$$r_k(z) = \frac{1}{\sum_{(x,h) \in D_k} h} \sum_{(x,h) \in D_k, x < z} h.$$

$$|r_k(s_{k,j}) - r_k(s_{k,j+1})| < \epsilon.$$

结构要点：

- Approximate splitting needs candidate split points.
- Instance weights are represented by second-order statistics h_i .
- XGBoost proposes a distributed weighted quantile sketch with theoretical guarantee.

18.4.4 3.4 Sparsity-aware Split Finding

原文位置：PDF p.4–p.5, Section 3.4, Algorithm 3, Figure 4, Figure 5。

结构要点：

- Sparsity can come from missing values, zero entries and one-hot encoding.
- Each split learns a default direction for missing values.
- The algorithm visits only non-missing entries.
- Computation becomes linear in the number of non-missing entries.

18.5 4. SYSTEM DESIGN

18.5.1 4.1 Column Block for Parallel Learning

原文位置：PDF p.5–p.6, Section 4.1, Figure 6。

结构要点：

- Data is stored in compressed column format.
- Columns in a block are sorted by feature value.
- The block structure supports parallel split finding and column subsampling.

18.5.2 4.2 Cache-aware Access

原文位置：PDF p.6–p.7, Section 4.2, Figure 7, Figure 8, Figure 9。

结构要点：

- Split finding needs indirect access to gradient statistics.
- Cache misses slow down training on large datasets.
- Cache-aware prefetching and proper block size improve performance.

18.5.3 4.3 Blocks for Out-of-core Computation

原文位置：PDF p.7, Section 4.3。

结构要点：

- Data is divided into blocks and stored on disk.
- Independent prefetching overlaps disk reading with computation.
- Block compression and block sharding improve disk I/O throughput.

18.6 5. RELATED WORKS

原文位置：PDF p.7–p.8, Section 5。

结构要点：

- XGBoost implements gradient boosting in functional space.
- It adds regularization, sparsity-aware learning and system-level optimizations.
- Weighted quantile sketch extends classical quantile summary to weighted data.

18.7 6. END TO END EVALUATIONS

18.7.1 6.1 System Implementation

原文位置：PDF p.8, Section 6.1。

结构要点：

- XGBoost is implemented as an open-source package.
- It supports Python, R, Julia and user-defined objectives.
- Distributed XGBoost supports multiple big-data ecosystems.

18.7.2 6.2 Dataset and Setup

原文位置：PDF p.8–p.9, Section 6.2, Table 2。

结构要点：

- Datasets include Allstate, Higgs Boson, Yahoo LTRC and Criteo.
- Tasks include classification, learning to rank and click-through rate prediction.
- Default experimental settings include max depth 8 and shrinkage 0.1.

18.7.3 6.3 Classification

原文位置：PDF p.9, Section 6.3, Table 3。

结构要点：

- XGBoost is compared with scikit-learn and R GBM on Higgs data.
- XGBoost achieves comparable accuracy and much faster training than scikit-learn.

18.7.4 6.4 Learning to Rank

原文位置：PDF p.9–p.10, Section 6.4, Table 4, Figure 10。

结构要点：

- XGBoost is evaluated on Yahoo learning-to-rank data.
- Column subsampling reduces runtime and can improve ranking performance.

18.7.5 6.5 Out-of-core Experiment

原文位置：PDF p.10, Section 6.5, Figure 11。

结构要点：

- Compression gives large speedup.
- Sharding across disks further improves throughput.
- XGBoost can process 1.7 billion examples on a single machine.

18.7.6 6.6 Distributed Experiment

原文位置：PDF p.10–p.11, Section 6.6, Figure 12, Figure 13。

结构要点：

- XGBoost is compared with Spark MLlib and H2O.
- XGBoost scales smoothly to full Criteo data by using out-of-core computation.
- Performance scales with the number of machines.

18.8 7. CONCLUSION

原文位置：PDF p.11, Section 7。

结构要点：

- XGBoost is a scalable tree boosting system widely used by data scientists.
- The paper proposes sparsity-aware learning and weighted quantile sketch.
- Cache access, data compression and sharding are essential for scalable tree boosting.

第 19 章 《XGBoost: A Scalable Tree Boosting System》中文版

19.1 摘要

树提升是一种效果很强、应用很广的机器学习方法。本文介绍了一个可扩展的端到端树提升系统 XGBoost。该系统被数据科学家广泛使用，并在许多机器学习竞赛中取得领先结果。论文提出了面向稀疏数据的稀疏感知算法，以及用于近似树学习的加权分位数草图。此外，论文还从缓存访问模式、数据压缩和数据分片等角度总结了如何构建可扩展的树提升系统。综合这些设计，XGBoost 能够用比已有系统更少的资源扩展到十亿级样本。

19.2 1. 引言

大规模机器学习应用的成功依赖两个因素：第一，模型本身要足够有效，能够捕捉复杂的数据依赖；第二，学习系统要足够可扩展，能够从大规模数据中训练出目标模型。梯度树提升在实际应用中表现突出，可用于分类、排序、广告点击率预测等任务。XGBoost 的出现，是为了把树提升方法做成一个在单机、分布式和内存受限场景下都能高效运行的系统。

本文的主要贡献包括：构建可扩展的端到端树提升系统；提出有理论保证的加权分位数草图；提出用于并行树学习的稀疏感知算法；提出适合外存训练的缓存友好块结构。

19.3 2. 树提升简述

19.3.1 2.1 正则化学习目标

XGBoost 使用加法树模型进行预测。对于第 i 个样本，预测值是多棵回归树输出的总和：

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}.$$

模型优化的目标函数由训练损失和正则化惩罚组成：

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k).$$

其中正则化项为：

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2.$$

这里 T 是叶子节点数量， w 是叶子节点权重向量。这个正则项会惩罚过多的叶子节点和过大的叶子权重，从而缓解过拟合。

19.3.2 2.2 梯度树提升

XGBoost 采用逐轮加树的方式训练模型。第 t 轮会在已有预测 $\hat{y}_i^{(t-1)}$ 的基础上加入一棵新树 f_t ：

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t).$$

为了高效优化该目标，XGBoost 使用二阶泰勒展开：

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t).$$

其中 g_i 是一阶梯度， h_i 是二阶梯度：

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)}), \quad h_i = \partial_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}^{(t-1)}).$$

对于固定树结构，落入叶子 j 的样本集合记为：

$$I_j = \{i \mid q(x_i) = j\}.$$

叶子节点的最优权重为：

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}.$$

对应树结构的目标函数得分为：

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i\right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T.$$

节点分裂带来的损失下降为：

$$L_{\text{split}} = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i\right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i\right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i\right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma.$$

这个分裂增益公式是 XGBoost 选择分裂点的核心依据。

19.3.3 2.3 收缩与列采样

除了正则化目标函数，XGBoost 还使用收缩和列采样来进一步防止过拟合。收缩会在每一步树提升后用学习率 η 缩放新加入的树权重，使单棵树的影响更保守。列采样借鉴随机森林思想，每次只使用部分特征，有助于防止过拟合并加速并行计算。

19.4 3. 分裂查找算法

19.4.1 3.1 基本精确贪婪算法

树学习的关键问题之一是寻找最佳分裂点。精确贪婪算法会枚举所有特征上的可能分裂点。为了高效完成这一过程，算法需要先按照特征值对数据排序，然后按排序顺序累积梯度统计量，并根据分裂增益公式选择最佳切分。

19.4.2 3.2 近似算法

当数据无法完整放入内存或处于分布式场景时，精确枚举所有分裂点效率很低。近似算法先根据特征分布的分位点提出候选切分点，然后将连续特征映射到这些候选切分点形成的桶中，在聚合统计量后只在候选分裂点中寻找最优解。候选点可以在建树初始阶段全局提出，也可以在每次分裂后局部重新提出。

19.4.3 3.3 加权分位数草图

近似算法的重要步骤是提出候选切分点。XGBoost 使用二阶梯度统计量作为样本权重，定义特征上的加权秩函数：

$$r_k(z) = \frac{1}{\sum_{(x,h) \in D_k} h} \sum_{(x,h) \in D_k, x < z} h.$$

候选切分点需要满足：

$$|r_k(s_{k,j}) - r_k(s_{k,j+1})| < \epsilon.$$

为处理带权数据的近似分位数问题，论文提出了有理论保证的分布式加权分位数草图算法。

19.4.4 3.4 稀疏感知分裂查找

实际数据中稀疏性很常见，来源包括缺失值、统计中的零值以及 one-hot 编码等特征工程结果。XGBoost 在每个树节点中加入默认方向。当分裂所需的特征值缺失时，样本会走向默认方向。这个默认方向由数据学习得到。算法只遍历非缺失项，因此计算复杂度与非缺失项数量线性相关。

19.5 4. 系统设计

19.5.1 4.1 用于并行学习的列块

树学习中最耗时的部分是让数据进入排序状态。XGBoost将数据存储为压缩列格式的块，每一列按照对应特征值排序。这个结构只需要在训练前构建一次，之后的迭代可以重复使用。列块结构支持并行分裂查找，也支持列采样。

19.5.2 4.2 缓存感知访问

虽然列块结构降低了分裂查找的计算复杂度，但算法需要按照行索引间接访问梯度统计量。这种非连续内存访问可能导致缓存未命中，从而拖慢训练。XGBoost使用缓存感知预取，并为近似算法选择合适的块大小，在并行效率和缓存命中之间取得平衡。

19.5.3 4.3 用于外存计算的块

为了处理无法放入主内存的数据，XGBoost将数据划分为多个块并存储在磁盘上。训练时通过独立线程预取数据块，使磁盘读取和计算并行进行。论文还使用块压缩和多磁盘分片，以降低磁盘 I/O 成本并提升吞吐量。

19.6 5. 相关工作

XGBoost实现的是函数空间中的梯度提升。与已有方法相比，XGBoost 加入正则化以防止过拟合，引入列采样和稀疏感知学习，并把缓存访问、外存计算等系统优化纳入端到端系统设计。加权分位数草图则把经典分位数摘要问题扩展到了带权数据场景。

19.7 6. 端到端评估

19.7.1 6.1 系统实现

XGBoost被实现为开源包，支持多种加权分类、排序目标函数以及用户自定义目标函数。它可用于 Python、R、Julia 等语言，并能够与 scikit-learn 等数据科学工具链集成。分布式版本还可以运行在 Hadoop、MPI、Spark、Flink 等生态中。

19.7.2 6.2 数据集与实验设置

论文使用 Allstate、Higgs Boson、Yahoo LTRC 和 Criteo 四个数据集。这些任务分别覆盖保险索赔分类、高能物理事件分类、学习排序和点击率预测。实验通常使用最大深度 8、收缩率 0.1 的树提升配置。

19.7.3 6.3 分类

在 Higgs 数据集上, XGBoost 与 scikit-learn 和 R GBM 进行比较。实验结果表明, XGBoost 能达到与 scikit-learn 相近的精度, 同时训练速度明显更快。

19.7.4 6.4 学习排序

在 Yahoo LTRC 排序数据集上, XGBoost 与 pGBRT 比较。结果显示 XGBoost 训练更快, 列采样不仅减少运行时间, 也可能提升排序表现。

19.7.5 6.5 外存实验

在 Criteo 数据上, XGBoost 评估了外存计算能力。实验表明, 压缩可以显著加速计算, 多磁盘分片可以进一步提升速度。最终系统能够在单机上处理 17 亿样本。

19.7.6 6.6 分布式实验

论文还在分布式场景下比较 XGBoost、Spark MLlib 和 H2O。XGBoost 借助外存计算能够平滑扩展到完整 Criteo 数据, 并且随着机器数量增加表现出良好扩展性。

19.8 7. 结论

本文总结了构建 XGBoost 这一可扩展树提升系统的经验。XGBoost 被广泛用于数据科学任务, 并在许多问题上取得领先结果。论文提出了处理稀疏数据的稀疏感知算法和用于近似学习的加权分位数草图。实践经验表明, 缓存访问模式、数据压缩和数据分片是构建端到端可扩展树提升系统的关键要素。

第 20 章 LightGBM: A Highly Efficient Gradient Boosting Decision Tree

 **笔记**完整英文原文见本地 PDF: `papers/lightgbm_a_highly_efficient_gradient_boosting_decision_tree.pdf`。
本章保留原论文标题、章节结构、核心公式、算法与图表位置，便于和 PDF 原文逐节对照。

20.1 Abstract

原文位置：PDF p.1, Abstract。

结构要点：

- GBDT is popular and effective, with implementations such as XGBoost and pGBRT.
- Existing implementations are inefficient when feature dimension and data size are large.
- The paper proposes Gradient-based One-Side Sampling (GOSS).
- The paper proposes Exclusive Feature Bundling (EFB).
- LightGBM combines GOSS and EFB and can speed up conventional GBDT by over 20 times with almost the same accuracy.

20.2 1 Introduction

原文位置：PDF p.1–p.2, Section 1。

结构要点：

- GBDT is widely used because of efficiency, accuracy and interpretability.
- Big data brings new challenges in both number of features and number of instances.
- Conventional GBDT scans all data instances for all possible split points, so complexity depends on both data size and feature size.
- The natural acceleration direction is to reduce data instances and reduce features.
- GOSS reduces data instances.
- EFB reduces effective feature number.
- The new algorithm with GOSS and EFB is called LightGBM.

20.3 2 Preliminaries

20.3.1 2.1 GBDT and Its Complexity Analysis

原文位置：PDF p.2, Section 2.1, Algorithm 1。

结构要点：

- GBDT is an ensemble model of decision trees trained in sequence.
- Each iteration fits negative gradients, also known as residual errors.
- The main cost is learning decision trees.
- The most time-consuming part is finding best split points.
- Histogram-based algorithm buckets continuous feature values into discrete bins.
- Histogram building costs $O(\#data \times \#feature)$.

- Split point finding costs $O(\#bin \times \#feature)$.
- Since $\#bin$ is much smaller than $\#data$, histogram building dominates computational complexity.

20.3.2 2.2 Related Work

原文位置：PDF p.2–p.3, Section 2.2。

结构要点：

- Existing GBDT implementations include XGBoost, pGBRT, scikit-learn and R gbm.
- XGBoost supports both pre-sorted and histogram-based algorithms.
- Random data sampling can hurt accuracy in GBDT.
- Feature filtering depends on redundancy assumptions and may hurt accuracy.
- Sparse data are common in large-scale real applications.
- Histogram-based GBDT lacks efficient sparse optimization in previous systems.
- LightGBM addresses these limits with GOSS and EFB.

20.4 3 Gradient-based One-Side Sampling

20.4.1 3.1 Algorithm Description

原文位置：PDF p.3–p.4, Section 3.1, Algorithm 2。

结构要点：

- AdaBoost has sample weights, but GBDT has no native sample weights.
- Gradients in GBDT provide useful information for sampling.
- Small-gradient instances are already well trained.
- Simply dropping small-gradient instances changes data distribution.
- GOSS keeps all large-gradient instances and randomly samples small-gradient instances.
- Sampled small-gradient instances are multiplied by $\frac{1-a}{b}$ when computing information gain.

20.4.2 3.2 Theoretical Analysis

原文位置：PDF p.4–p.5, Section 3.2, Definition 3.1, Theorem 3.2。

原文公式：

$$\tilde{V}_j(d) = \frac{1}{n} \left[\frac{(\sum_{x_i \in A_l} g_i + \frac{1-a}{b} \sum_{x_i \in B_l} g_i)^2}{n_l^j(d)} + \frac{(\sum_{x_i \in A_r} g_i + \frac{1-a}{b} \sum_{x_i \in B_r} g_i)^2}{n_r^j(d)} \right].$$

结构要点：

- GOSS uses an estimated variance gain $\tilde{V}_j(d)$ on subset $A \cup B$.
- A contains large-gradient instances.
- B contains sampled small-gradient instances.
- The coefficient $\frac{1-a}{b}$ normalizes the small-gradient subset back to the scale of the original small-gradient data.
- Theorem 3.2 bounds the approximation error of GOSS.
- Random sampling is a special case of GOSS when $a = 0$.
- GOSS can outperform random sampling under the same sampling ratio.

20.5 4 Exclusive Feature Bundling

原文位置：PDF p.5–p.6, Section 4, Algorithm 3, Algorithm 4, Theorem 4.1。

结构要点：

- High-dimensional data are usually sparse.
- Many sparse features are mutually exclusive.
- Mutually exclusive features rarely take nonzero values simultaneously.
- Exclusive features can be bundled into a single feature bundle.
- Histogram building complexity changes from $O(\#data \times \#feature)$ to $O(\#data \times \#bundle)$.
- Finding the optimal bundling strategy is NP-hard.
- LightGBM reduces the bundling problem to graph coloring and uses a greedy algorithm.
- Features in one bundle are merged by assigning different bin ranges through offsets.

20.6 5 Experiments

原文位置：PDF p.6–p.8, Section 5, Table 1, Table 2, Table 3, Figure 1, Figure 2。

结构要点：

- Experiments use five public datasets: Allstate, Flight Delay, LETOR, KDD10 and KDD12.
- Datasets cover sparse features, dense features, binary classification and ranking.
- Baselines include XGBoost and LightGBM without GOSS/EFB.
- LightGBM is the fastest while maintaining almost the same accuracy.
- Compared with lgb_baseline, LightGBM speeds up training by 21x, 6x, 1.6x, 14x and 13x on five datasets.
- XGBoost histogram version runs out of memory on KDD10 and KDD12.
- GOSS and EFB do not hurt accuracy while bringing speed-up.

20.6.1 5.1 Overall Comparison

原文位置：PDF p.6–p.8, Section 5.1。

结构要点：

- XGBoost and lgb_baseline are used as baselines.
- xgb_exa uses pre-sorted algorithm.
- xgb_his uses histogram-based algorithm.
- LightGBM combines lgb_baseline with GOSS and EFB.
- LightGBM achieves large training speed-up with nearly unchanged accuracy.

20.6.2 5.2 Analysis on GOSS

原文位置：PDF p.8, Section 5.2, Table 4。

结构要点：

- GOSS alone can bring nearly 2x speed-up using 10%–20% data.
- GOSS still needs full-data prediction and gradient computation, so speed-up is not linear in sampling ratio.
- GOSS is compared with Stochastic Gradient Boosting (SGB).
- Under the same sampling ratio, GOSS is consistently more accurate than SGB.

20.6.3 5.3 Analysis on EFB

原文位置：PDF p.8, Section 5.3。

结构要点：

- EFB contribution is measured by comparing lgb_baseline and EFB_only.

- EFB gives significant speed-up on large-scale sparse datasets.
- EFB merges one-hot and implicitly exclusive sparse features into fewer features.
- Bundling improves spatial locality and cache hit rate.

20.7 6 Conclusion

原文位置：PDF p.8, Section 6。

结构要点：

- LightGBM is a new GBDT algorithm with GOSS and EFB.
- GOSS handles large numbers of data instances.
- EFB handles large numbers of features.
- Theory and experiments show that LightGBM improves speed and memory consumption while maintaining accuracy.
- Future work includes choosing optimal a and b in GOSS and improving EFB.

第 21 章 《LightGBM: A Highly Efficient Gradient Boosting Decision Tree》中文版

21.1 摘要

梯度提升决策树 (GBDT) 是一种流行的机器学习算法, 并且已经有 XGBoost 和 pGBRT 等有效实现。尽管这些实现采用了许多工程优化, 但当特征维度很高、数据规模很大时, 它们的效率和可扩展性仍然不够理想。主要原因是: 对于每个特征, 算法都需要扫描所有数据样本, 以估计所有可能切分点的信息增益, 这非常耗时。

为了解决这个问题, 论文提出了两种新技术: 基于梯度的单边采样 (Gradient-based One-Side Sampling, GOSS) 和互斥特征捆绑 (Exclusive Feature Bundling, EFB)。GOSS 会排除大量小梯度样本, 只使用剩余样本估计信息增益。由于大梯度样本在信息增益计算中更重要, GOSS 可以用更小的数据量获得较准确的信息增益估计。EFB 将互斥特征捆绑在一起, 从而减少特征数量。结合 GOSS 和 EFB 的新 GBDT 实现被称为 LightGBM。实验表明, LightGBM 在几乎保持相同精度的同时, 可以使传统 GBDT 训练速度提升超过 20 倍。

21.2 1 引言

GBDT 因效率、精度和可解释性而被广泛使用, 在多分类、点击预测、学习排序等任务中表现突出。随着数据规模和特征数量不断增大, GBDT 面临精度与效率之间的新挑战。传统 GBDT 实现通常需要对每个特征扫描所有样本, 以估计所有候选切分点的信息增益。因此, 其计算复杂度同时依赖样本数量和特征数量, 当数据很大时会非常耗时。

一个直接的加速思路是减少样本数量和特征数量, 但这并不简单。对于样本, GBDT 没有像 AdaBoost 那样的天然样本权重, 因此不能直接使用 AdaBoost 的采样方法。LightGBM 观察到, 梯度大小可以反映样本对信息增益的重要性, 于是提出 GOSS: 保留大梯度样本, 只随机丢弃小梯度样本。对于特征, 实际高维特征空间通常很稀疏, 很多特征几乎不会同时非零, 因此可以将这些互斥特征捆绑成更少的特征, 这就是 EFB。

21.3 2 预备知识

21.3.1 2.1 GBDT 及其复杂度分析

GBDT 是由按顺序训练的多棵决策树组成的集成模型。每次迭代中, GBDT 通过拟合损失函数的负梯度来训练新的决策树。GBDT 的主要计算成本在于学习决策树, 而学习决策树最耗时的部分是寻找最佳分裂点。

预排序算法会在排好序的特征值上枚举所有可能分裂点, 虽然可以找到最优分裂点, 但训练速度和内存消耗都较差。直方图算法将连续特征值离散为若干桶, 并在训练中构建特征直方图。该算法在内存和速度上更高效, 因此 LightGBM 在直方图算法基础上展开。

直方图构建的复杂度为 $O(\#data \times \#feature)$, 寻找分裂点的复杂度为 $O(\#bin \times \#feature)$ 。由于桶数量通常远小于样本数量, 直方图构建主导了整体计算复杂度。因此, 如果能减少样本数或特征数, 就能显著加速 GBDT 训练。

21.3.2 2.2 相关工作

已有 GBDT 实现包括 XGBoost、pGBRT、scikit-learn 和 R 中的 gbm。scikit-learn 和 R gbm 使用预排序算法, pGBRT 使用直方图算法, XGBoost 同时支持预排序和直方图算法。为了减少训练数据规模, 常见方法是下采样样本, 但随机采样可能损害 GBDT 精度。为了减少特征数量, 也可以过滤弱特征, 但这依赖特征冗余假设, 实

践中不一定成立。

实际大规模数据通常非常稀疏。预排序算法可以通过忽略零值降低训练成本，但直方图算法之前缺少有效的稀疏优化方案。LightGBM 正是为了解决这些限制，提出了 GOSS 和 EFB。

21.4 3 基于梯度的单边采样

21.4.1 3.1 算法描述

在 AdaBoost 中，样本权重可以表示样本重要性。但 GBDT 没有天然样本权重，因此 AdaBoost 的采样方法不能直接使用。LightGBM 注意到，GBDT 中每个样本的梯度提供了有用的采样信息。如果一个样本梯度很小，说明该样本训练误差较小，已经被模型学得比较好。

直接丢弃小梯度样本会改变数据分布，从而损害模型精度。GOSS 的做法是：保留所有大梯度样本，并从小梯度样本中随机采样一部分。为了补偿数据分布变化，在计算信息增益时，对被采样的小梯度样本乘以常数 $\frac{1-a}{b}$ 。这样既能更关注欠训练样本，又不会过度改变原始数据分布。

21.4.2 3.2 理论分析

GBDT 使用决策树从输入空间学习到梯度空间的函数。对某个固定节点，特征 j 在切分点 d 上的信息增益可以用分裂后的方差增益表示。GOSS 将样本按梯度绝对值降序排列，保留前 $a \times 100\%$ 的大梯度样本形成集合 A ，再从剩余小梯度样本中随机采样比例为 b 的集合 B 。

GOSS 在 $A \cup B$ 上估计方差增益：

$$\tilde{V}_j(d) = \frac{1}{n} \left[\frac{(\sum_{x_i \in A_l} g_i + \frac{1-a}{b} \sum_{x_i \in B_l} g_i)^2}{n_l^j(d)} + \frac{(\sum_{x_i \in A_r} g_i + \frac{1-a}{b} \sum_{x_i \in B_r} g_i)^2}{n_r^j(d)} \right].$$

其中 $\frac{1-a}{b}$ 用于把小梯度采样集合的梯度和归一化回原始小梯度数据规模。论文的定理表明，GOSS 的近似误差可以被控制，并且随机采样是 $a = 0$ 时的特殊情形。在许多情况下，GOSS 在相同采样率下可以优于随机采样。

21.5 4 互斥特征捆绑

高维数据通常非常稀疏，这为近似无损地减少特征数量提供了可能。在稀疏特征空间中，很多特征是互斥的，也就是说它们几乎不会同时取非零值。这些互斥特征可以被安全地捆绑成一个特征，称为互斥特征束。

通过精心设计的特征扫描算法，可以从特征束中构建出与单独特征相同的直方图。这样，直方图构建复杂度可以从 $O(\#data \times \#feature)$ 降为 $O(\#data \times \#bundle)$ ，其中 $\#bundle$ 远小于 $\#feature$ 。

EFB 需要解决两个问题：第一，哪些特征应该捆绑在一起；第二，如何构造捆绑后的特征。论文证明，寻找最少互斥特征束的最优划分是 NP-hard 问题。因此，LightGBM 将该问题转化为图着色问题：把特征看作顶点，如果两个特征不互斥，就在它们之间连边；然后使用贪婪算法得到近似解。

对于特征合并，关键是要保证原始特征值仍能从特征束中识别出来。由于直方图算法存储的是离散桶，LightGBM 可以通过给不同特征的桶添加偏移量，让多个互斥特征占据不同的桶区间，从而安全地合并成一个特征束。

21.6 5 实验

论文使用五个公开数据集：Allstate、Flight Delay、LETOR、KDD10 和 KDD12。这些数据集覆盖稀疏特征、稠密特征、二分类和排序任务。实验环境是一台 Linux 服务器，使用两颗 E5-2670 v3 CPU，共 24 核和 256GB 内存。

21.6.1 5.1 整体比较

实验使用 XGBoost 和不含 GOSS/EFB 的 LightGBM 版本作为基线。XGBoost 分为预排序版本 `xgb_exa` 和直方图版本 `xgb_his`。LightGBM 则是在 `lgb_baseline` 基础上加入 GOSS 和 EFB。

结果表明, LightGBM 在几乎保持相同测试精度的同时训练最快。与 `lgb_baseline` 相比, LightGBM 在 Allstate、Flight Delay、LETOR、KDD10、KDD12 上的训练加速比分别为 21 倍、6 倍、1.6 倍、14 倍和 13 倍。XGBoost 的直方图版本在 KDD10 和 KDD12 上由于内存不足无法运行。这些结果说明, GOSS 和 EFB 在带来显著加速的同时不会损害精度。

21.6.2 5.2 GOSS 分析

通过比较 LightGBM 和 EFB_only, 论文分析 GOSS 的加速能力。结果显示, 在只使用 10%–20% 数据的情况下, GOSS 本身可以带来接近 2 倍的加速。不过 GOSS 仍然需要在全量数据上进行预测和计算梯度, 因此整体加速并不会与采样比例完全线性相关。

论文还将 GOSS 与随机梯度提升 (SGD) 比较。在相同采样率下, GOSS 的精度始终高于 SGD。这说明 GOSS 是比普通随机采样更有效的采样方法。

21.6.3 5.3 EFB 分析

论文通过比较 `lgb_baseline` 和 EFB_only 来分析 EFB 的贡献。结果显示, EFB 在大规模稀疏数据集上可以带来显著加速。这是因为 EFB 将大量 one-hot 特征和隐式互斥稀疏特征合并成更少的特征。此外, 由于许多原本分散的特征被捆绑在一起, 空间局部性和缓存命中率也会提升。因此, EFB 是直方图算法中利用稀疏性的有效方法。

21.7 6 结论

本文提出了一种新的 GBDT 算法 LightGBM, 其中包含两项新技术: 基于梯度的单边采样 (GOSS) 和互斥特征捆绑 (EFB)。GOSS 用于处理大量样本, EFB 用于处理大量特征。论文对两项技术进行了理论分析和实验验证。实验结果与理论分析一致, 表明在 GOSS 和 EFB 的帮助下, LightGBM 可以在计算速度和内存消耗方面显著优于 XGBoost 和 SGD, 同时保持相近精度。未来工作包括研究 GOSS 中 a 和 b 的最优选择, 以及继续提升 EFB 在更多特征场景下的表现。

第 22 章 模型

22.1 BPR 模型

东邪的 tip 22.1

双塔模型训练模型是为了损失函数最小，也就是所有第 u 个人对第 i 件事比第 j 件事更喜欢的概率，因为加了负号所以他越大损失函数越小。本来应该是概率相乘但是因为太小了就取 $\ln$ 变相加了。由于概率不好算直接拿 Sigmoid 函数近似了，近似的方法就是用 $\hat{y}_{ui} - \hat{y}_{uj}$ 的差带入 Sigmoid 函数。 $\hat{y}_{ui}$ 和 $\hat{y}_{uj}$ 分别是 UserTower 和 ItemTower 带入用一个 x_u 再分别带入物品 i 和物品 j 得出的分数。

定义 22.1 (模型打分)

$$\hat{y}_{ui} = f_{\theta}(x_u)^{\top} g_{\phi}(x_i)$$

其中 $\hat{y}_{ui}$ 为用户 u 对物品 i 的预测偏好分数； $f_{\theta}(\cdot)$ 为 User Tower，参数为 θ ，输入用户特征 x_u ，输出用户向量； $g_{\phi}(\cdot)$ 为 Item Tower，参数为 ϕ ，输入物品特征 x_i ，输出物品向量； $\top$ 表示向量内积。

 **笔记** 训练阶段整个 batch 以矩阵形式输入：

$$X_U \in \mathbb{R}^{B \times d_u}, \quad X_V \in \mathbb{R}^{B \times d_v}$$

其中 B 为 batch size，每一行对应一个用户/物品的特征向量。

单个样本为向量形式：

$$x_u \in \mathbb{R}^{d_u}, \quad x_i \in \mathbb{R}^{d_v}$$

公式中 $f_{\theta}(x_u)$ 严格指单个样本的向量输入。

 **笔记** embedding 是嵌入的意思，说的很好 transformer 架构用 attention 公式就是把用户信息和用户的兴趣偏好嵌入了一个矩阵

 **笔记** f_{θ} 由多层堆叠构成，上一层输出作为下一层输入：

$$x_u \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \mathbf{u}$$

每一层做一次：ReLU 就是保留正数部分负数部分归零

$$h_l = \text{ReLU}(W_l h_{l-1} + b_l)$$

最终输出即为用户 embedding $\mathbf{u} \in \mathbb{R}^d$ 。

定义 22.2 (BPR 损失函数)

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j) \in \mathcal{D}} \log \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\Theta\|^2$$

其中 $\mathcal{D}$ 为训练三元组集合，每条样本为 (u, i, j) ，即用户 u 、正样本 i 、负样本 j ； $\log \sigma(\hat{y}_{ui} - \hat{y}_{uj})$ 为对数似然，分数差越大则损失越小； $\lambda \|\Theta\|^2$ 为 L2 正则项， λ 控制正则化强度，防止过拟合。

东邪的 tip 22.2

训练是为了损失函数最小

例题 22.1 假设模型只有一层，参数为：

$$W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

则 L2 正则项为：

$$\|\Theta\|^2 = 1^2 + 2^2 + 3^2 + 4^2 + 1^2 + (-1)^2 = 1 + 4 + 9 + 16 + 1 + 1 = 32$$

即将 W 和 b 中所有元素平方后求和，得到一个标量。

参数越多正则项越大。

 **笔记** 原始目标是最大化所有三元组的概率之积（连乘），但连乘结果数值极小，计算机无法存储，因此取对数将连乘变为连加：

$$\prod_{(u,i,j)} P(i >_u j | \Theta) \xrightarrow{\log} \sum_{(u,i,j)} \log P(i >_u j | \Theta)$$

再加负号转为最小化损失，数值稳定，结果与原始目标等价。

 **笔记** 每个三元组 (u, i, j) 对应一个事件：用户 u 更喜欢 i 而不是 j 。训练数据中有很多这样的三元组，我们希望模型对所有这些事件同时成立的概率最大，因此对所有三元组的概率连乘——这正是最大似然估计的思想。

 **笔记** $\hat{y}_{ui} - \hat{y}_{uj}$ 越大， σ 越接近 1， $\log \sigma$ 越接近 0，加负号后损失越小。即正样本分数远高于负样本时，模型损失趋近于 0，说明排序正确。

定义 22.3 (贝叶斯后验概率近似方法)

$$P(i >_u j | \Theta) = \sigma(\hat{y}_{ui} - \hat{y}_{uj})$$

其中 $i >_u j$ 表示用户 u 对物品 i 的偏好强于物品 j ； Θ 为模型全部参数； $\hat{y}_{ui}$ 与 $\hat{y}_{uj}$ 分别为用户 u 对正样本 i （已点击）和负样本 j （未点击）的预测分数； $\sigma(\cdot) = \frac{1}{1 + e^{-x}}$ 为 Sigmoid 函数，将分数差压缩至 $(0, 1)$ 区间作为概率。

 **笔记** 贝叶斯后验概率 $P(i >_u j | \Theta) = \sigma(\hat{y}_{ui} - \hat{y}_{uj})$ 并非推导所得，而是 BPR 作者的建模假设：直接用 Sigmoid 函数将正负样本的分数差映射为概率。就是为了把他映射到 0 到 1 之间

 **笔记** Θ 是全部样本空间是一个集合

22.1.1 WBPR

定义 22.4 (WBPR 损失函数)

$$\mathcal{L}_{\text{WBPR}} = - \sum_{(u,i,j) \in \mathcal{D}} w_j \cdot \log \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\Theta\|^2$$

其中 w_j 为负样本 j 的流行度权重，越流行的 item 未被点击则权重越大，损失越大，梯度信号越强。

就是在 BPR 的基础上给 j 加了权重

第 23 章 基础知识

定义 23.1 (广告投放核心指标)

点击率 **CTR** (Click-Through Rate):

$$\text{CTR} = \frac{\text{点击数}}{\text{展示数}}$$

单次点击成本 **CPC** (Cost Per Click):

$$\text{CPC} = \frac{\text{花费}}{\text{点击数}}$$

千次展示成本 **CPM** (Cost Per Mille):

$$\text{CPM} = \frac{\text{花费}}{\text{展示数}} \times 1000$$

投资回报率 **ROI** (Return on Investment):

$$\text{ROI} = \frac{\text{收入} - \text{成本}}{\text{成本}}$$

广告支出回报 **ROAS** (Return on Ad Spend):

$$\text{ROAS} = \frac{\text{广告收入}}{\text{广告花费}}$$

单次安装成本 **CPI** (Cost Per Install):

$$\text{CPI} = \frac{\text{花费}}{\text{安装数}}$$



东邪的 tip 23.1

p 都是 per 就是前面除以后面当分母。

定义 23.2 (转化漏斗类指标)

转化率 **CVR** (Conversion Rate):

$$\text{CVR} = \frac{\text{转化数}}{\text{点击数}}$$

千次展示安装数 **IPM** (Installs Per Mille):

$$\text{IPM} = \frac{\text{安装数}}{\text{展示数}} \times 1000$$

第 N 日留存率 (Day N Retention Rate):

$$\text{Retention}_N = \frac{\text{第 } N \text{ 天仍活跃用户数}}{\text{首日新增用户数}}$$



定义 23.3 (预算与出价类指标)

日预算消耗率 (Daily Budget Consumption Rate):

$$\text{消耗率} = \frac{\text{实际消耗}}{\text{计划预算}}$$

有效千次展示收益 **eCPM** (Effective Cost Per Mille):

$$\text{eCPM} = \frac{\text{总收入}}{\text{展示数}} \times 1000$$

出价反推 (Bid Estimation):

$$\text{出价} = \text{目标 CPI} \times \text{预期 CVR}$$



定义 23.4 (用户价值类指标)

用户生命周期价值 **LTV** (Lifetime Value):

$$LTV = ARPU \times \text{平均生命周期}$$

每用户平均收入 **ARPU** (Average Revenue Per User):

$$ARPU = \frac{\text{总收入}}{\text{活跃用户数}}$$

ROI 回收判断准则: 当 $LTV > CPI$ 时, 该渠道具备投放价值。



定义 23.5 (效果对比类指标)

同比增长率 (Year-over-Year Growth Rate, YoY):

$$YoY = \frac{\text{本期} - \text{同期}}{\text{同期}}$$

环比增长率 (Month-over-Month Growth Rate, MoM):

$$MoM = \frac{\text{本期} - \text{上期}}{\text{上期}}$$

渠道占比 (Channel Share):

$$\text{Channel Share} = \frac{\text{某渠道花费}}{\text{总花费}}$$



笔记 [广告投放漏斗归因一览]

漏斗层级	核心指标	出问题找谁
曝光 → 点击	CTR (Click-Through Rate, 点击率)	素材组/定向策略
点击 → 完播	VTR (View-Through Rate, 完播率)	视频内容质量
完播 → 安装	CVR (Conversion Rate, 转化率) IPM (Installs Per Mille, 千次展示安装数)	落地页、游戏包体
安装 → 付费	付费转化率 (Payment Conversion Rate) ARPU (Average Revenue Per User, 每用户平均收入)	人群质量、游戏内设计
整体盈利	ROAS (Return on Ad Spend, 广告支出回报) LTV/CPI (Lifetime Value, 用户生命周期价值 / Cost Per Install, 单次安装成本)	投放策略

东邪的 tip 23.2

OK 你按照我这个方式再梳理一下, 我可不可以这样理解我们投放广告会经过, 挑选用户, 用户经过点击, 观看, 下载, 付费四个过程大部分指标就是看这个些的。点击率低可能封面不好看, 完播率低可能是内容不精彩, 完播率也高但是下载不高可能是下载麻烦, 落地页不好看, 下载率高但是付费少可能是人群有问题对吗

笔记 [广告分析常用工具]

一、网站/产品流量分析

- **Google Analytics 4 (GA4)**: 看用户来源、访问、转化、留存, 用于理解网站和 App 整体表现。
 - **Microsoft Clarity**: 看热力图、用户录屏、页面点击行为, 免费行为分析工具。
- 对应岗位: 数据分析、增长分析、产品分析、网站运营分析。

二、广告投放分析

- **Meta Ads Manager** (即 Facebook / Instagram 广告后台): 查看 Campaign、Ad Set、Ad 层级的数据表现。
 - **Google Ads + GA4**: 看广告带来的点击、转化、ROI。
 - 国内对应工具: 巨量引擎、腾讯广告、快手磁力引擎、百度营销。
- 对应岗位: 广告投放、商业化数据分析、增长运营、效果广告优化师。

分析对象	代表工具
广告投放效果	Meta Ads Manager、Google Ads
用户行为分析	GA4、Microsoft Clarity

定义 23.6 (付费转化率)

付费转化率 **PCR** (Payment Conversion Rate):

$$\text{PCR} = \frac{\text{付费用户数}}{\text{安装数}}$$

定义 23.7 (广告转化概率分解)

广告系统的核心目标是预估用户最终付费的概率，按漏斗结构分解为三层条件概率的连乘：

$$p(\text{付费}) = p(\text{点击}|\text{曝光}) \times p(\text{安装}|\text{点击}) \times p(\text{付费}|\text{安装})$$

即：

$$p(\text{付费}) = \text{CTR} \times \text{CVR} \times \text{PCR}$$

 笔记 [对数似然与数值稳定性]

模型训练时采用对数似然损失 (Log Loss):

$$\mathcal{L} = - \sum [y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

其中 $y \in \{0, 1\}$ 为真实标签， $\hat{p}$ 为模型预测概率。

将概率连乘转化为对数累加：

$$\log(p_1 \times p_2 \times p_3) = \log p_1 + \log p_2 + \log p_3$$

这样做的原因是避免数值下溢问题：多个小概率直接连乘会趋近于零，超出计算机浮点数的表示范围。

东邪的 tip 23.3

cpi 是为数不多比一大

23.1 attention

 **笔记** 每一个 token，比如 doing = do + ing，都有一个固定的映射向量。把一句话的所有 token（比如 10 个）竖着排列，得到矩阵

$$X \in \mathbb{R}^{n \times d}$$

其中 n 是 token 数量，也是模型一次能处理的上限； d 是每个 token 向量的维度。

用训练好的三个权重矩阵 W_Q, W_K, W_V 分别右乘 X ， $X * W_Q$ 按照 Attention 公式

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

将 X 处理成一个新矩阵，融合了整个序列的上下文信息。

东邪的 tip 23.4

 **笔记** 因为 $QK^\top$ 的方差是 d_k ，所以要除以 $\sqrt{d_k}$ ，本质还是标准化。

定义 23.8 (Softmax)

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad \text{如 } [3, 1, 0] \xrightarrow{e^z} [20.1, 2.7, 1] \xrightarrow{\text{归一化}} [0.84, 0.11, 0.05]$$

指数 e^z 放大分数差距，实现“带竞争的归一化”，使得分最高的 token 获得大部分注意力权重。

**东邪的 tip 23.5**

softmax 就归一化只不过为了突出重要信息先变成 e 的指数次方再归一

- **Query (W_Q):** “我在找什么”，当前 token 根据自己的需求检索其他位置。
- **Key (W_K):** “我有什么、我在哪”，提供每个 token 的标识/地址，供 Query 匹配。
- **Value (W_V):** “我要传递什么信息”，真正被加权聚合的内容，不参与注意力权重计算。

东邪的 tip 23.6

key 里面有着每一个 token 的坐标信息用于分开我喜欢猫和猫喜欢我

东邪的 tip 23.7

QKV 决定了现实世界的信息到数学世界映射的准确性，FNN 决定了输入到你想要的结果也就是我们给 ai 提一个问题他最终给我们答案的好坏。Attention 决定了模型知不知道该关注什么，FFN 决定了模型能不能基于关注到的信息推算出正确答案。前者决定“理解方向”对不对，后者决定“计算结果”好不好。

然后将这个新矩阵与未处理过的自己 X 相加（残差连接），对每一行（即每个 token）单独归一化，使均值为 0、方差为 1，记结果为 x_1 ：

$$x_1 = \text{LayerNorm}(X + \text{Attention}(X))$$

x_1 经过 FFN 处理得到 $\text{FFN}(x_1)$ ，再将 $\text{FFN}(x_1)$ 与 x_1 相加，再次归一化，进入下一层：

$$x_2 = \text{LayerNorm}(x_1 + \text{FFN}(x_1))$$



笔记 残差连接 $x + \text{Attention}(x)$ 把原始输入加回来，保证梯度能直接流回前面的层，防止网络过深时梯度消失。



笔记 F 然后 FFN 先将维度升高把粘在一起的信息分开，ReLU relu 就是用来拟合函数的，这里可以理解为保留重要信息，然后再把矩阵压缩回去。

定义 23.9 (前馈网络 FFN)

前馈网络 (Feed-Forward Network, FFN) 是 Transformer 每层中紧跟 Attention 的两层线性变换，对每个 token 的向量独立进行非线性精加工。设输入向量为 $x \in \mathbb{R}^d$ ，则

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

其中 $W_1 \in \mathbb{R}^{d \times 4d}$ ， $W_2 \in \mathbb{R}^{4d \times d}$ 。输入输出维度均为 d ，中间层先升至 $4d$ （扩大特征间隙，便于分离），经 ReLU 激活后再压回 d 维。



- W_1 : up-projection（升维矩阵 / 扩张矩阵）
- W_2 : down-projection（降维矩阵 / 收缩矩阵）

定义 23.10 (ReLU 激活函数)

ReLU (Rectified Linear Unit) 定义为

$$\text{ReLU}(x) = \max(0, x)$$

即保留正数、将负数清零。其作用是在 FFN 中引入非线性，使多层线性变换无法合并为单一矩阵乘法，从

而赋予网络更强的表达能力。



东邪的 tip 23.8

Transformer 提供了一个通用的序列处理框架，后续所有创新本质上都是在这个框架上做文章：更好地训练它（BERT、GPT）、处理更长序列（稀疏 Attention、FlashAttention）、提升效率（蒸馏、量化）、注入更多知识（预训练 + 微调、RLHF）、扩展到图像和音频（ViT、Whisper）。Transformer 是地基，后续所有大模型都是在这块地基上盖的不同建筑。

- **因果掩码 (Causal Masking) —— GPT 类 (Decoder)**：每个 token 只允许关注自己及左侧的 token，不能看到右侧。注意力矩阵的上三角部分被设为 $-\infty$ ，经过 softmax 后权重为 0：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

其中 $M_{ij} = 0$ ($i \geq j$)， $M_{ij} = -\infty$ ($i < j$)。保证生成时只能利用过去的上下文。

- **双向注意力 —— BERT 类 (Encoder)**：每个 token 可以关注序列中所有位置（包括左右两侧），不加因果掩码，捕获全局上下文信息。